



Jenkins

Table of Contents

1. Introduction to Jenkins
 - 1.1. What is Jenkins?
 - 1.2. Continuous Integration and Continuous Delivery (CI/CD) Explained
 - 1.3. Benefits of Using Jenkins for CI/CD
2. Jenkins Features for Better Management
 - 2.1. Timeout
 - 2.2. Timestamp
 - 2.3. Disable/Enable Job
 - 2.4. Build a Job Concurrently
 - 2.5. Retry Count
 - 2.6. Throttle Builds
3. Creating Your First Jenkins Job
 - 3.1. Click on New Item
 - 3.2. Enter the Job Name and Choose Freestyle Project
 - 3.3. Follow the Steps
 - 3.4. Click on Build Now
 - 3.5. Viewing the Console Output
4. Changing Jenkins Theme using Plugin
 - 4.1. Installing and Configuring the Simple Theme Plugin
 - 4.2. Changing the Jenkins URL
5. User Management in Jenkins
 - 5.1. Creating a New User in Jenkins
 - 5.2. Jenkins Role-Based Access Control (RBAC)
 - 5.3. Managing Roles and Assigning Permissions
6. Jenkins Integration with GitHub
 - 6.1. Building a Job without GitHub Plugin

- 6.2. Building a Job with GitHub Plugin
 - 6.3. Building a Job Using Trigger Builds Remotely (Authentication Token)
- 7. Jenkins Build Triggers
 - 7.1. Build After Other Projects are Built
 - 7.2. Build Job Periodically
 - 7.3. Poll SCM (Source Code Management)
- 8. Jenkins Job Configuration and Customization
 - 8.1. Defining Variables Globally
 - 8.2. Parameterized Jobs
 - 8.3. Custom Workspace
 - 8.4. Changing Display Name and Project Name
- 9. Building Upstream and Downstream Projects
 - 9.1. Blocking Build When Upstream Project is Building
 - 9.2. Blocking Build When Downstream Project is Building
- 10. Jenkins Pipelines
 - 10.1. Creating Jenkins Pipeline Using Build Pipeline
 - 10.2. Understanding Continuous Deployment vs. Continuous Delivery
 - 10.3. Running Two Jobs in Parallel in Jenkins Pipeline
 - 10.4. Deploying WAR to Tomcat Server Through Jenkins (Automation)
- 11. Creating Jenkins Slaves
 - 11.1. Configuring Jenkins Slaves
 - 11.2. Running Commands in Pipeline as Code
 - 11.3. Setting Environment Variables in Pipeline as Code
 - 11.4. Taking User Input in Pipeline

Conclusion

Jenkins

Jenkins is a powerful application that allows continuous integration and continuous delivery of projects, regardless of the platform you are working on. It is a free source that can handle any kind of build or continuous integration. You can integrate Jenkins with a number of testing and deployment technologies. In this notes, we will explain how you can use Jenkins to build and test your software projects continuously.

Continuous Integration and Continuous Delivery (CI/CD) are essential practices in modern software development that aim to streamline and automate the process of building, testing, and deploying code changes. These practices help development teams deliver software more efficiently, with higher quality, and at a faster pace.

Continuous Integration (CI):

CI is the practice of automatically integrating code changes from multiple developers into a shared repository on a frequent basis, typically multiple times a day. The main goal of CI is to detect integration issues early in the development cycle, thereby reducing the chances of conflicts and bugs that arise when multiple developers work on the same codebase.

Key features of CI:

Automatic code integration: Developers frequently merge their code changes into a central repository, where automated build and testing processes are triggered.

Automated build and tests: CI systems automatically compile the code, run unit tests, and perform other validation checks to ensure the codebase remains stable.

Early feedback: By catching issues early, CI allows developers to fix problems quickly, preventing the accumulation of bugs.

Continuous Delivery (CD):

CD is an extension of CI that focuses on automating the software release process. It ensures that code changes are always in a deployable state and ready for production release. The ultimate goal of CD is to enable frequent and reliable software releases to end-users with minimal manual intervention.

Key features of CD

- **Automated deployment:** The CD pipeline automates the deployment of code changes to various environments, including staging and production, with consistent configurations.
- **Release orchestration:** CD pipelines manage the entire release process, coordinating tasks such as database updates and infrastructure provisioning.
- **Rollbacks and monitoring:** CD ensures that automated rollbacks are possible if any issues arise during deployment. Monitoring is also integrated to track the application's health post-deployment.

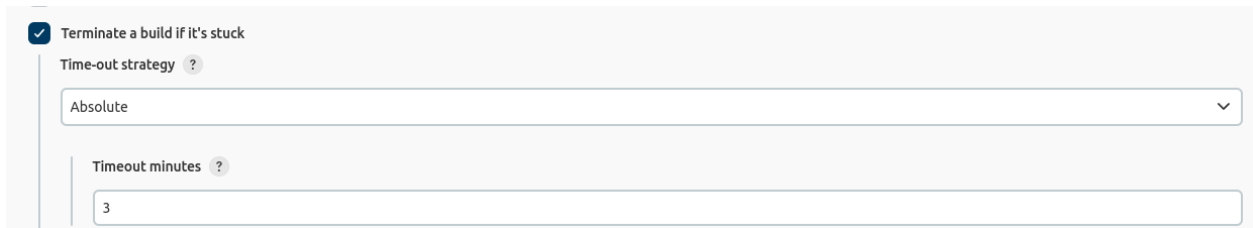
Benefits of CI/CD

- **Faster time-to-market:** CI/CD streamlines the development and deployment process, reducing the time it takes to deliver new features and updates to end-users.
- **Higher software quality:** Automated testing and validation catch bugs early, leading to a more stable and reliable codebase.
- **Risk reduction:** Frequent integration and automated deployment enable faster bug identification and recovery, minimizing risks associated with manual processes.
- **Increased collaboration:** CI/CD encourages collaboration among developers, testers, and operations teams, fostering a culture of continuous improvement.

Jenkins features of every tool:

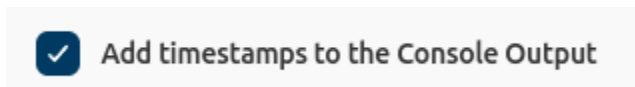
1. **Timeout-** It will be found under the Build Environment section-> Terminate a build if it's stuck.

When the system sleeps for some time e.g. 240 seconds we define the Time-out strategy time e.g. 3 minutes. It means that a job will not build or fail.



The screenshot shows the 'Build Environment' configuration page in Jenkins. Under the 'Timeout' section, there is a checkbox labeled 'Terminate a build if it's stuck' which is checked. Below it, there is a 'Time-out strategy' dropdown menu set to 'Absolute'. Further down, there is a 'Timeout minutes' input field with the value '3'.

2. **Timestamp-** It will be found under the Build Environment section->Add timestamps to the Console Output which means it will notify the time whatever command is running at what time.



The screenshot shows the 'Build Environment' configuration page in Jenkins. Under the 'Timestamp' section, there is a checkbox labeled 'Add timestamps to the Console Output' which is checked.

3. **Disable/Enable Job-** It will be found under the status of a specific project. You can use this feature to disable or enable the job build. If you want that nobody can build the job except you then, you can disable the job. And whenever you want to enable it you can do that too.



The screenshot shows the Jenkins project status interface. On the left is a blue button labeled 'Disable Project'. In the center is a yellow warning triangle icon followed by the text 'This project is currently disabled'. On the right is a blue button labeled 'Enable'.

4. **Build a Job concurrently-** As you are aware you couldn't build a job concurrently. But if you want to do this then, you have to enable the feature that will be found under the section of Description named -> Execute concurrent builds if necessary.

☐ Execute concurrent builds if necessary ?

5. **Retry count-** When your job is not built successfully. So you can retry as many as you want but for that, you have to enable it to define time and number of counts.

☒ Retry Count ?

6. **Throttle Builds-** When you want that there should be a limited number of builds asked then, you will use throttle builds. For eg, the Number of builds is 3 and the Time period is minute. Then there are only 3 builds that will be successful and possible, not more than that.

☒ Throttle builds ?

Number of builds ?

3

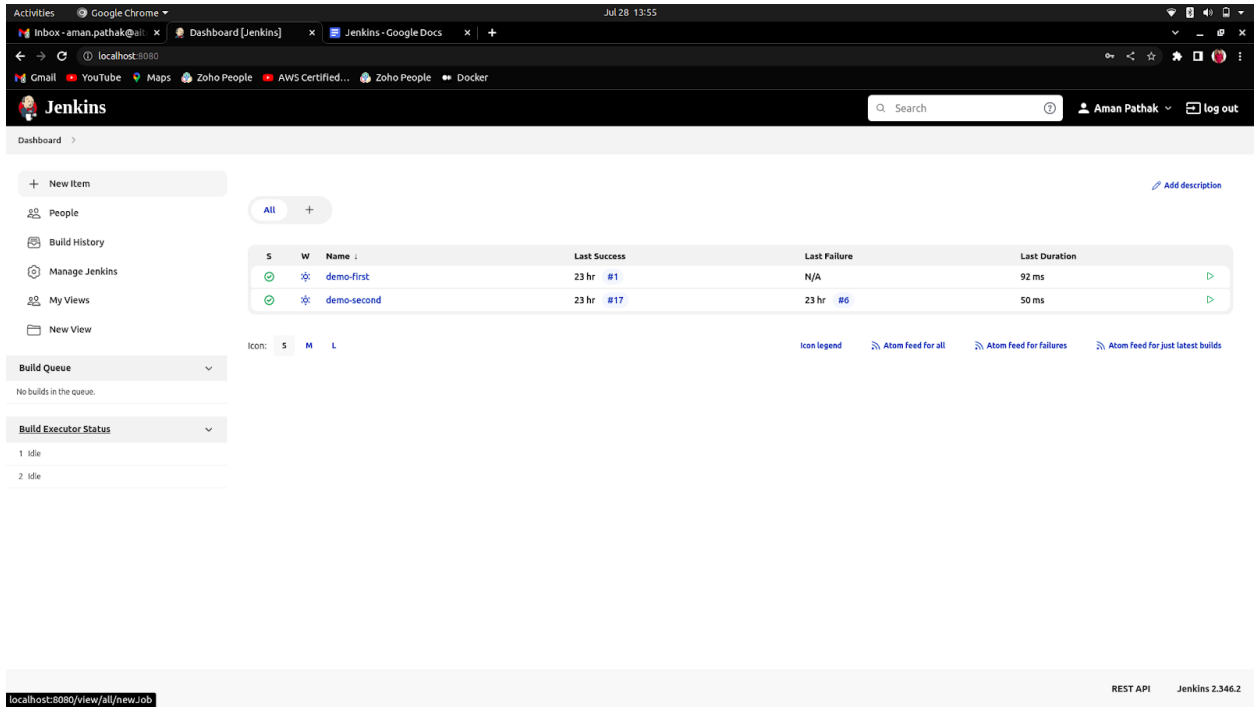
Approximately 20 seconds between builds

Time period ?

Minute

Creating First Job

1. Click on New Item.



The screenshot shows the Jenkins Dashboard in a Google Chrome browser. The dashboard includes a sidebar with navigation links: New Item, People, Build History, Manage Jenkins, My Views, and New View. The main content area displays a table of jobs. The 'Build Queue' section shows 'No builds in the queue.' and the 'Build Executor Status' section shows two idle executors. The footer indicates the URL 'localhost:8080/view/all/newJob', the REST API, and Jenkins version 2.346.2.

S	W	Name	Last Success	Last Failure	Last Duration
✓	✱	demo-first	23 hr #1	N/A	92 ms
✓	✱	demo-second	23 hr #17	23 hr #6	50 ms

2. Enter the job name and choose freestyle project.

Activities Google Chrome Jul 28 13:56

New Item [Jenkins]

localhost:8080/newall/newJob

Dashboard > All >

Enter an item name

first-job

Required field

Freestyle project
This is the central feature of Jenkins. Jenkins will build your project, combining any SCM with any build system, and this can be even used for something other than software build.

Pipeline
Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.

Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

Folder
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

Multibranch Pipeline
Creates a set of Pipeline projects according to detected branches in one SCM repository.

Organization Folder
Creates a set of multibranch project subfolders by scanning for repositories.

If you want to create a new item from other existing, you can use this option:

Copy from

Type to autocomplete

OK

3. Follow the below steps.

Activities Google Chrome Jul 28 13:57

first-job Config [jenkins]

localhost:8080/job/first-job/configure

Dashboard > first-job >

General Source Code Management **Build Triggers** Build Environment Build Post-build Actions

☐ Build periodically ?

☐ GitHub hook trigger for GITSCM polling ?

☐ Poll SCM ?

Build Environment

☐ Delete workspace before build starts

☐ Use secret text(s) or file(s) ?

☐ Add timestamps to the Console Output

☐ Inspect build log for published Gradle build scans

☐ Terminate a build if it's stuck

☐ With Ant ?

Build

Add build step +

Filter

Execute Windows batch command

Execute shell

Invoke Ant

Invoke Gradle script

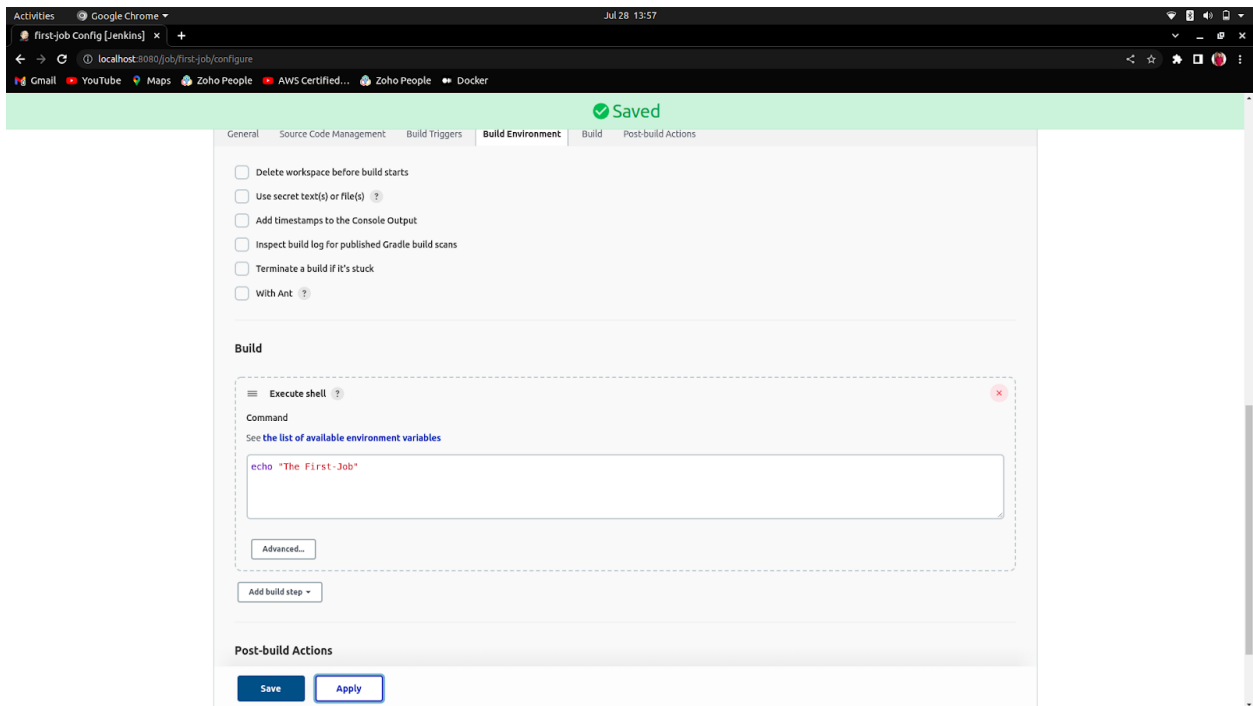
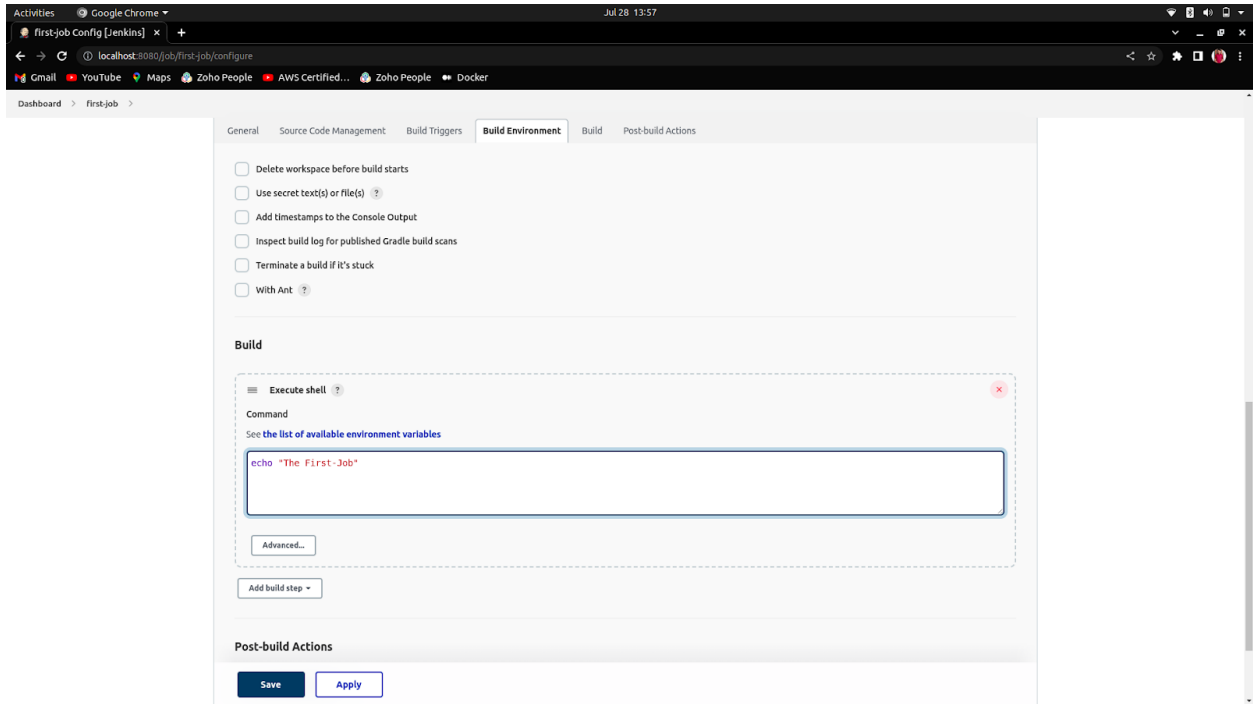
Invoke top-level Maven targets

Run with timeout

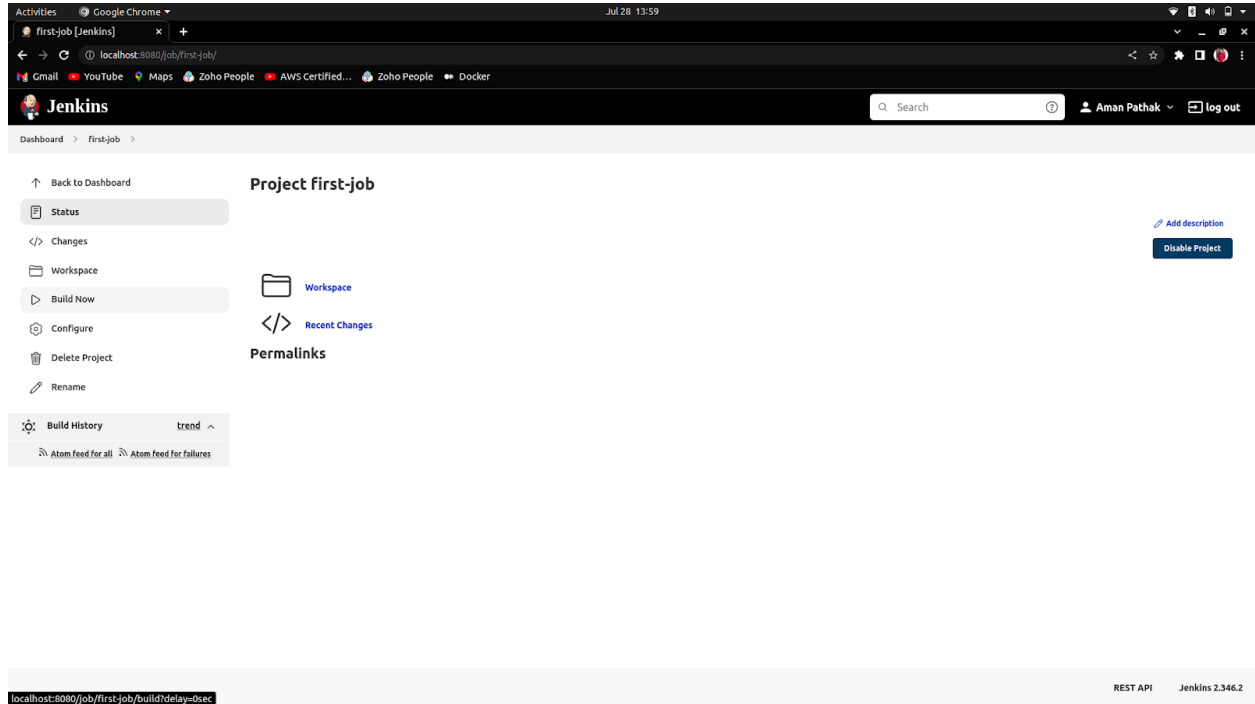
Set build status to "pending" on GitHub commit

Save Cancel

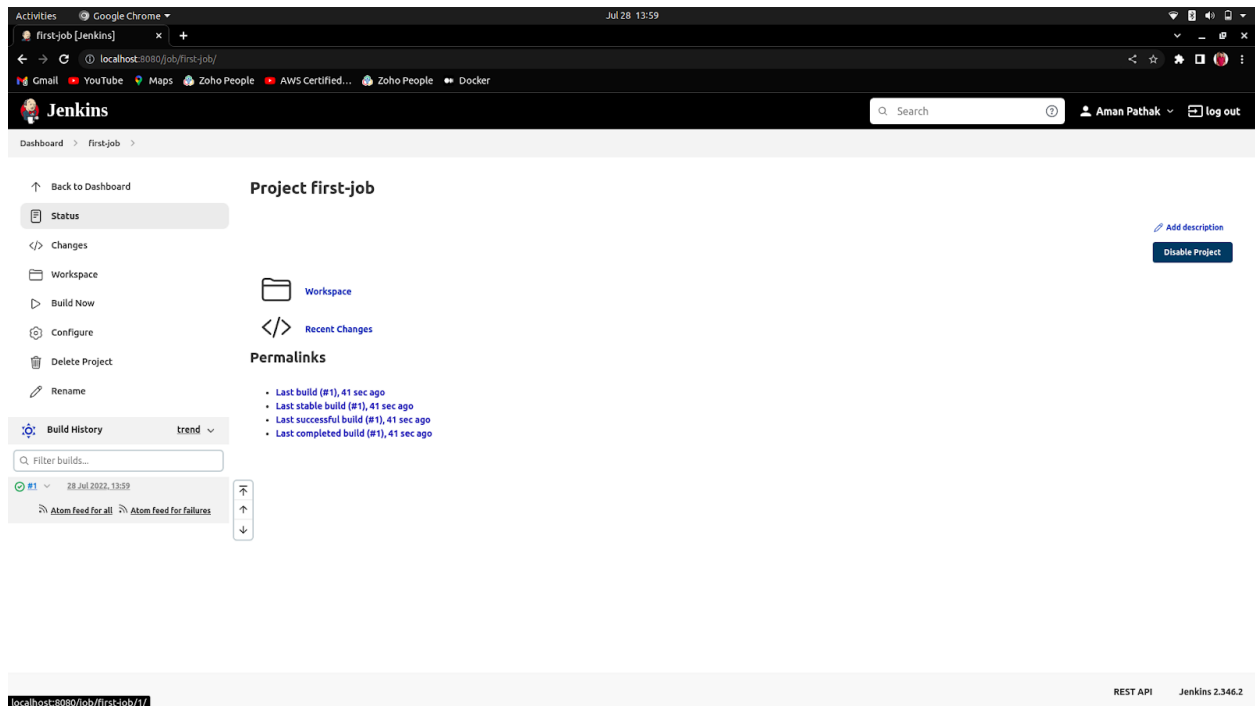
localhost:8080/job/first-job/configure#



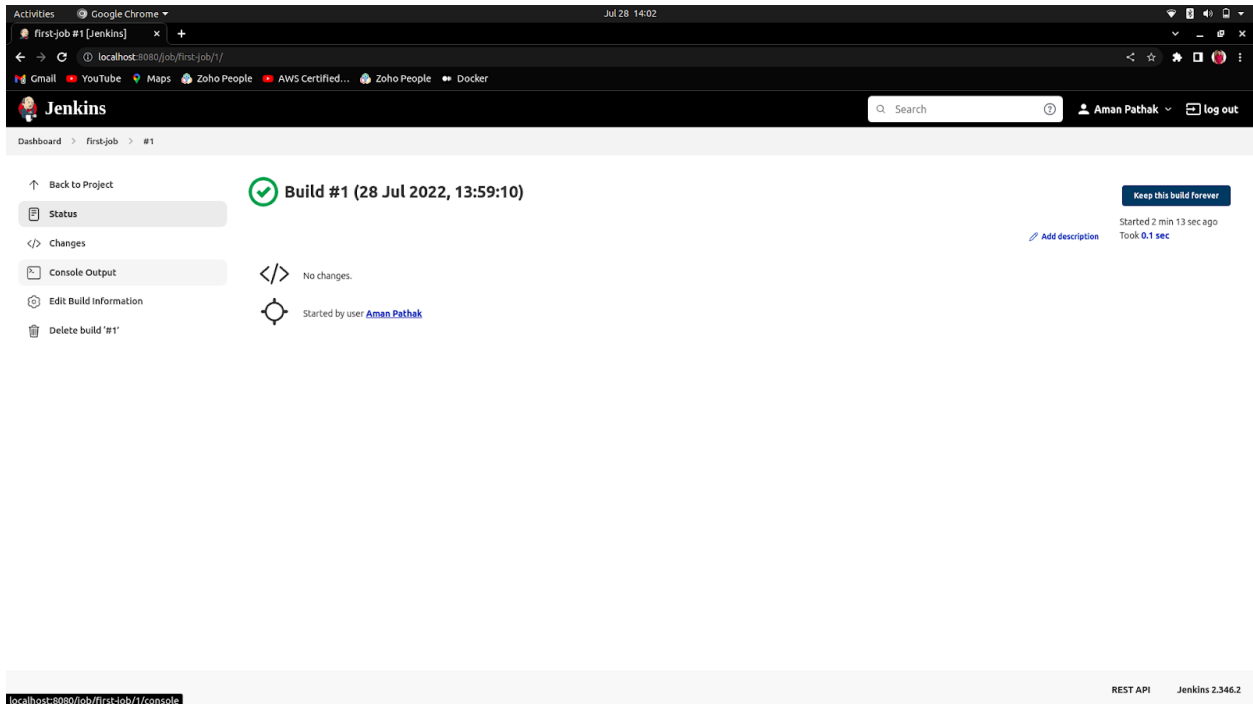
4. Click on Build Now



5. As the build is successful. Now click on the green color text (#1).

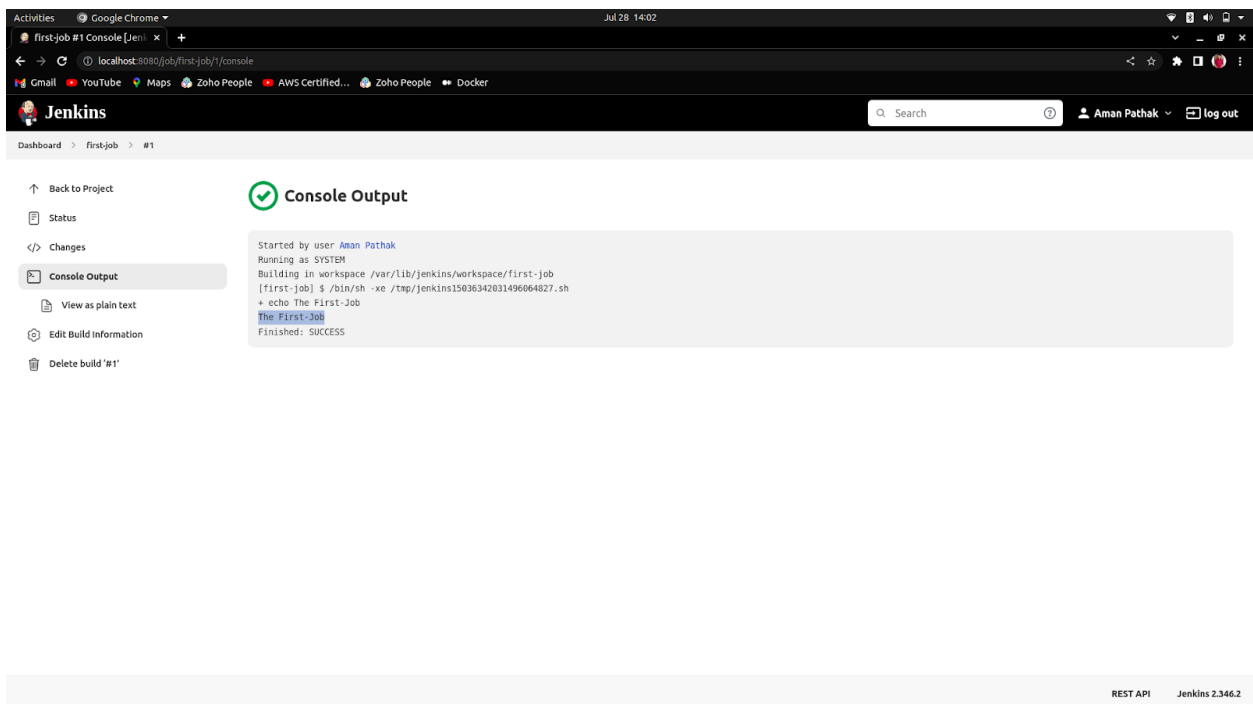


6. Now, To see the output of your created job. Click on the Console Output.



The screenshot shows the Jenkins web interface in a Google Chrome browser. The address bar shows the URL `localhost:8080/job/first-job/1/`. The Jenkins dashboard displays the status of 'Build #1' (28 Jul 2022, 13:59:10) as successful. The left sidebar has a menu with options: 'Back to Project', 'Status', 'Changes', 'Console Output' (selected), 'Edit Build Information', and 'Delete build #1'. The main content area shows the 'Console Output' tab with a green checkmark icon and the text 'No changes.' and 'Started by user Aman Pathak'. A 'Keep this build forever' button is visible in the top right. The bottom status bar shows 'localhost:8080/job/first-job/1/console', 'REST API', and 'Jenkins 2.346.2'.

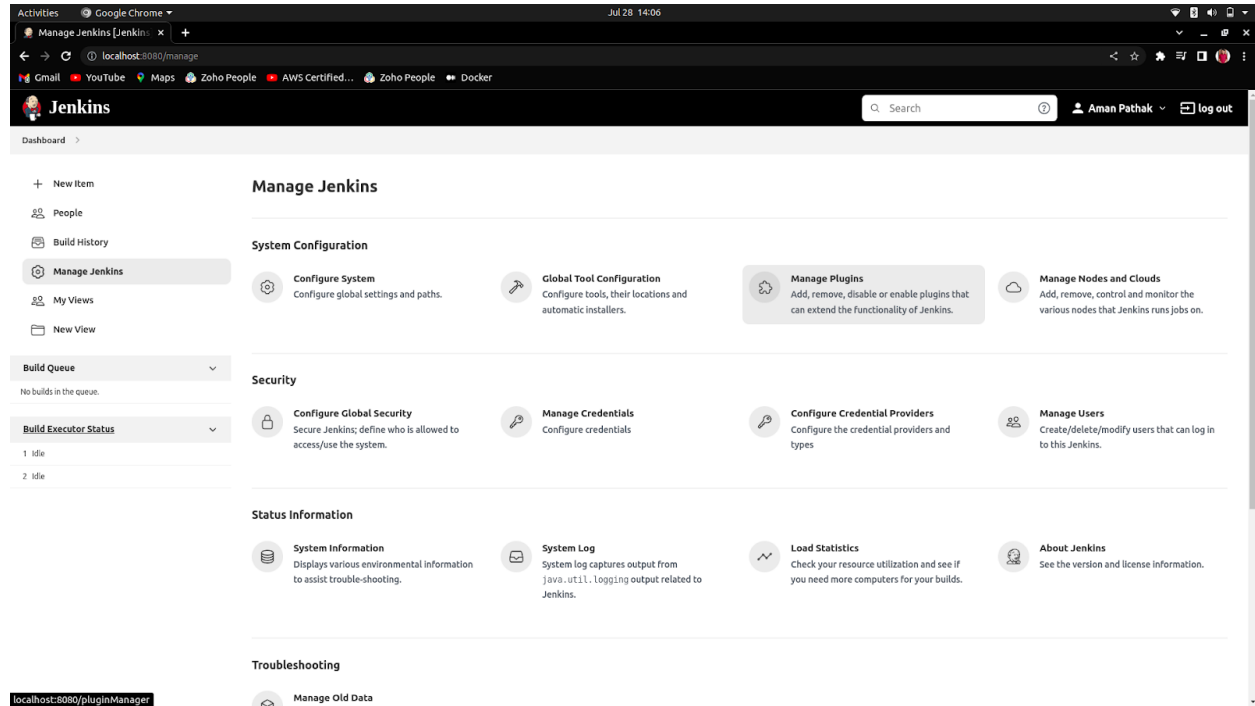
7. The Output:



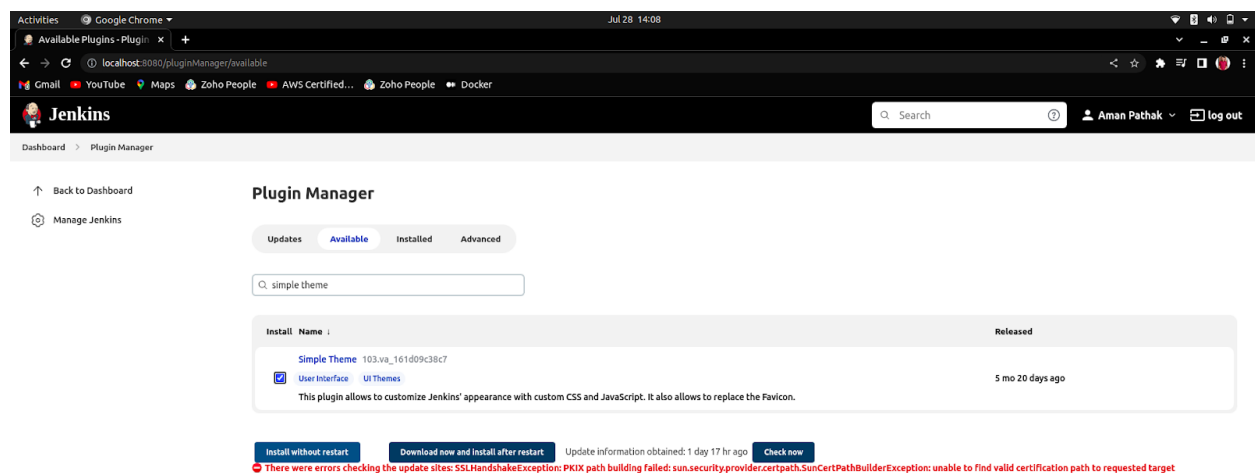
The screenshot shows the Jenkins web interface with the 'Console Output' tab selected. The main content area displays the build output, which includes the text 'Started by user Aman Pathak', 'Running as SYSTEM', 'Building in workspace /var/lib/jenkins/workspace/first-job', the command `[first-job] $ /bin/sh -xe /tmp/jenkins15036342031496064827.sh`, and the output `+ echo The First-Job`. The output is followed by a link to 'The First-Job' and the status 'Finished: SUCCESS'. The left sidebar is the same as in the previous screenshot. The bottom status bar shows 'localhost:8080/job/first-job/1/console', 'REST API', and 'Jenkins 2.346.2'.

Change Jenkins Theme using Plugin

1. Click on Manage Jenkins -> Manage Plugins.



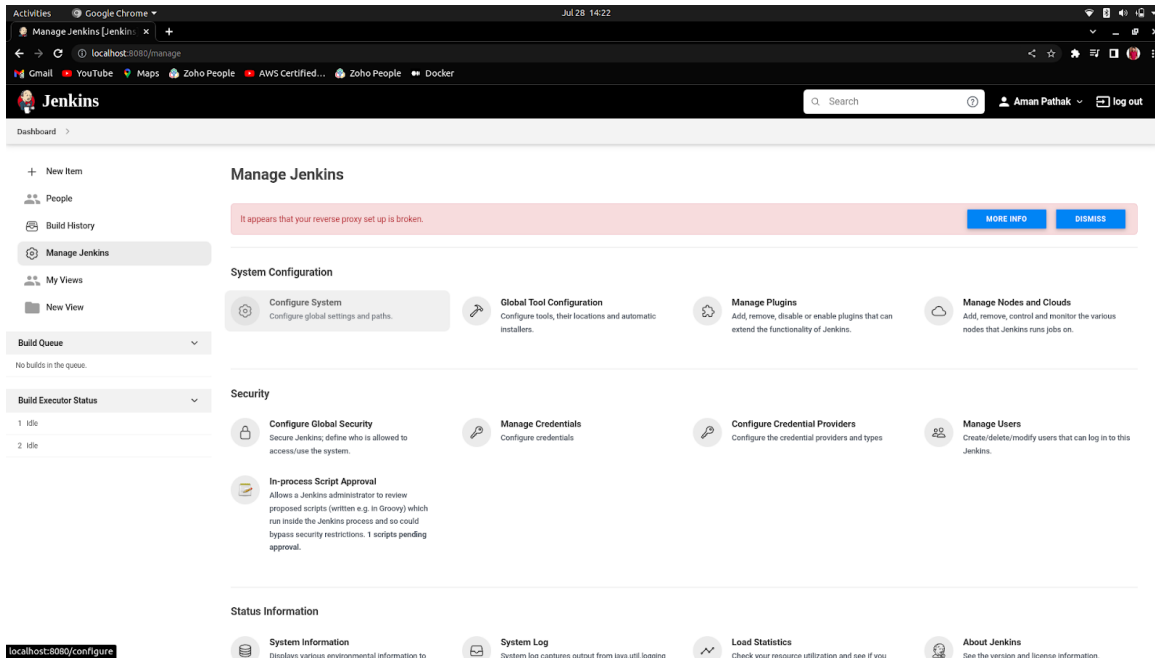
2. Now search the Simple theme plugin, checked it and install without restart.



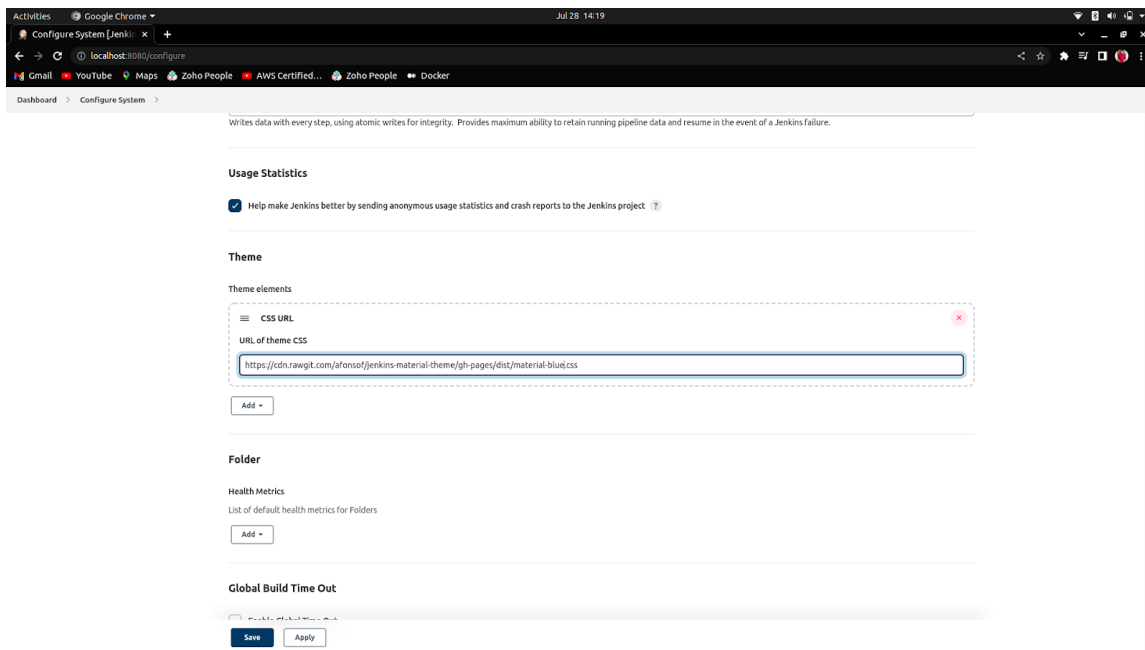
3. Now, that the plugin has been installed check the restart Jenkins server and log in again.

The image shows two screenshots of the Jenkins web interface. The top screenshot displays the 'Installing Plugins/Upgrades' page, indicating that the 'Simple Theme' and 'Loading plugin extensions' were installed successfully. It includes a 'Go back to the top page' link and a checkbox for 'Restart Jenkins when installation is complete and no jobs are running'. The bottom screenshot shows the 'Sign in' page with the Jenkins logo, a 'Welcome to Jenkins!' message, and login fields for 'aman19' and a password. A 'Keep me signed in' checkbox is checked, and a 'Sign in' button is visible. The browser address bar shows 'localhost:8080/login?from=%2F'.

4. Now, go to the Manage Jenkins again and configure the Plugin by clicking on Configure System.



5. Now, enter the URL of CSS to change the theme and apply then, then save it. URL Link-> [Themes Link](https://cdn.rawgit.com/atlassian/jenkins-material-theme/gh-pages/dist/material-blud.css)



To change the Jenkins URL

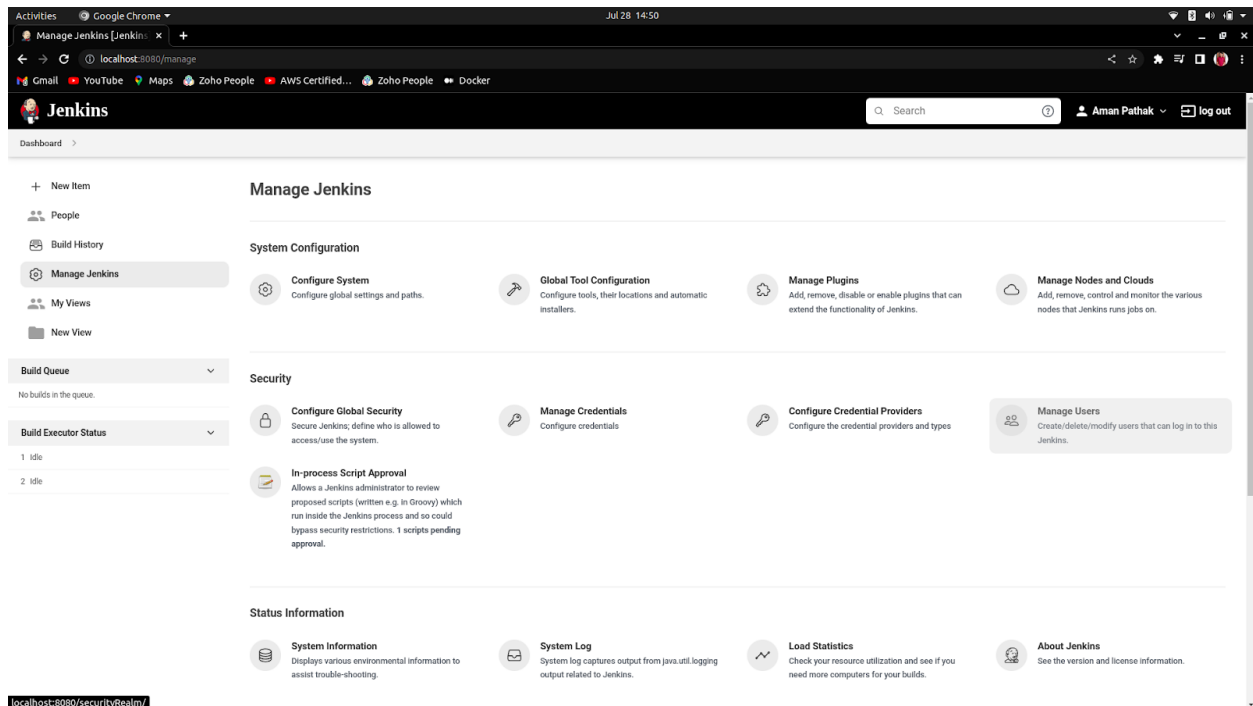
Manage Jenkins -> Configure System

Enter the desired location or URL.

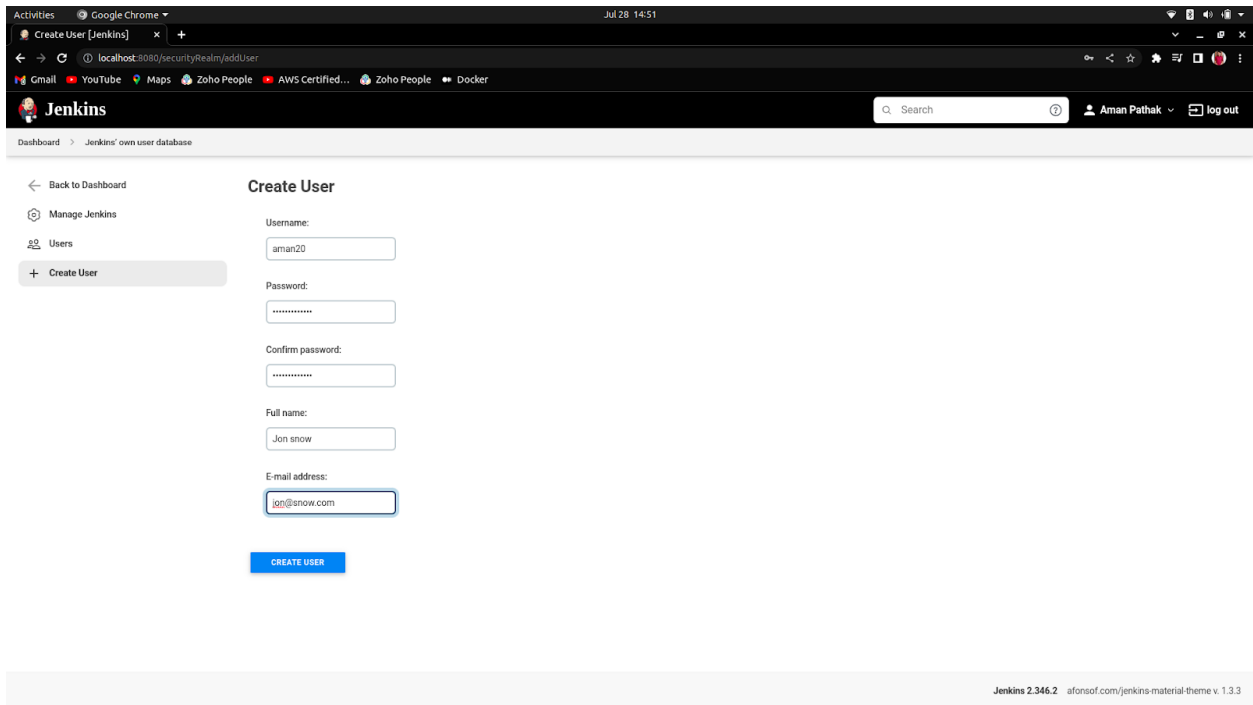
By Default URL-> <http://localhost:8080/>

+Create a new user in Jenkins

1. Go to Manage Plugins -> Manage Users.



2. Click on Create user. And enter the further details.

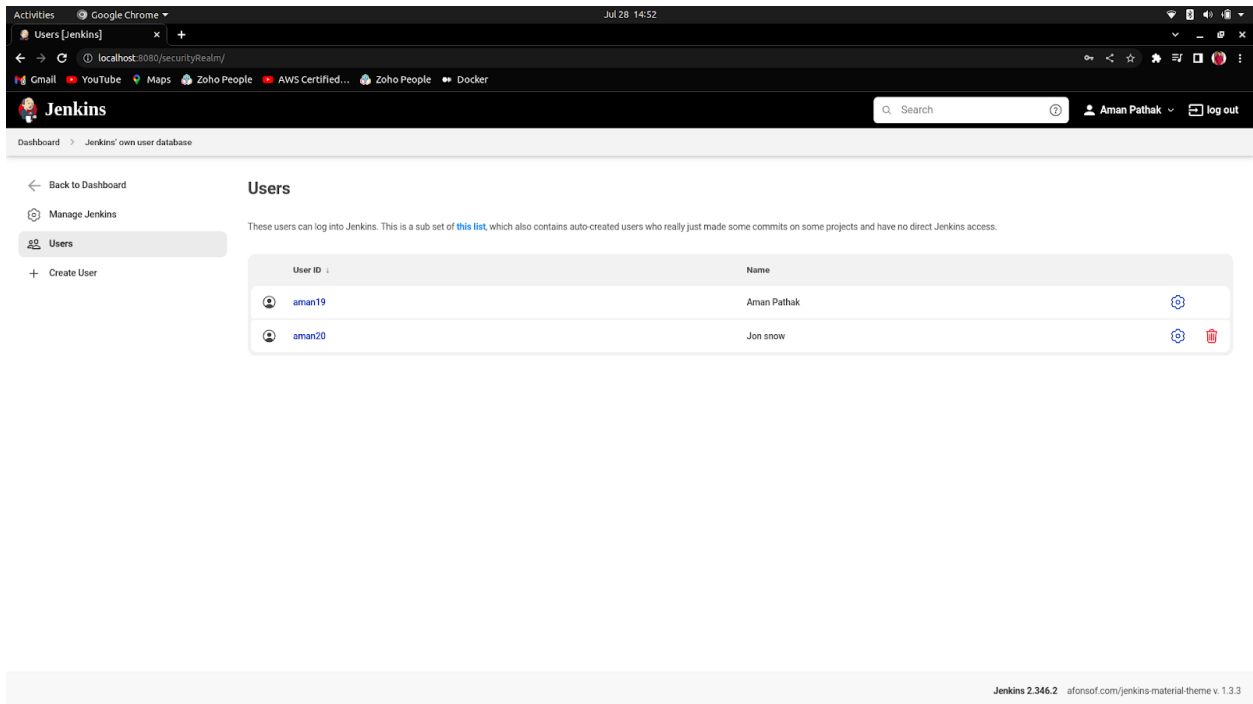


The screenshot shows the Jenkins 'Create User' form in a web browser. The browser's address bar shows 'localhost:8080/securityRealm/addUser'. The Jenkins header includes a search bar and the user 'Aman Pathak' with a 'log out' button. The left sidebar has links for 'Back to Dashboard', 'Manage Jenkins', 'Users', and 'Create User' (which is highlighted). The main form area is titled 'Create User' and contains the following fields:

- Username: aman20
- Password: (masked with dots)
- Confirm password: (masked with dots)
- Full name: Jon snow
- E-mail address: jgn@snow.com

A blue 'CREATE USER' button is located at the bottom of the form. The footer of the page indicates 'Jenkins 2.346.2' and 'afonsof.com/jenkins-material-theme v. 1.3.3'.

3. New User



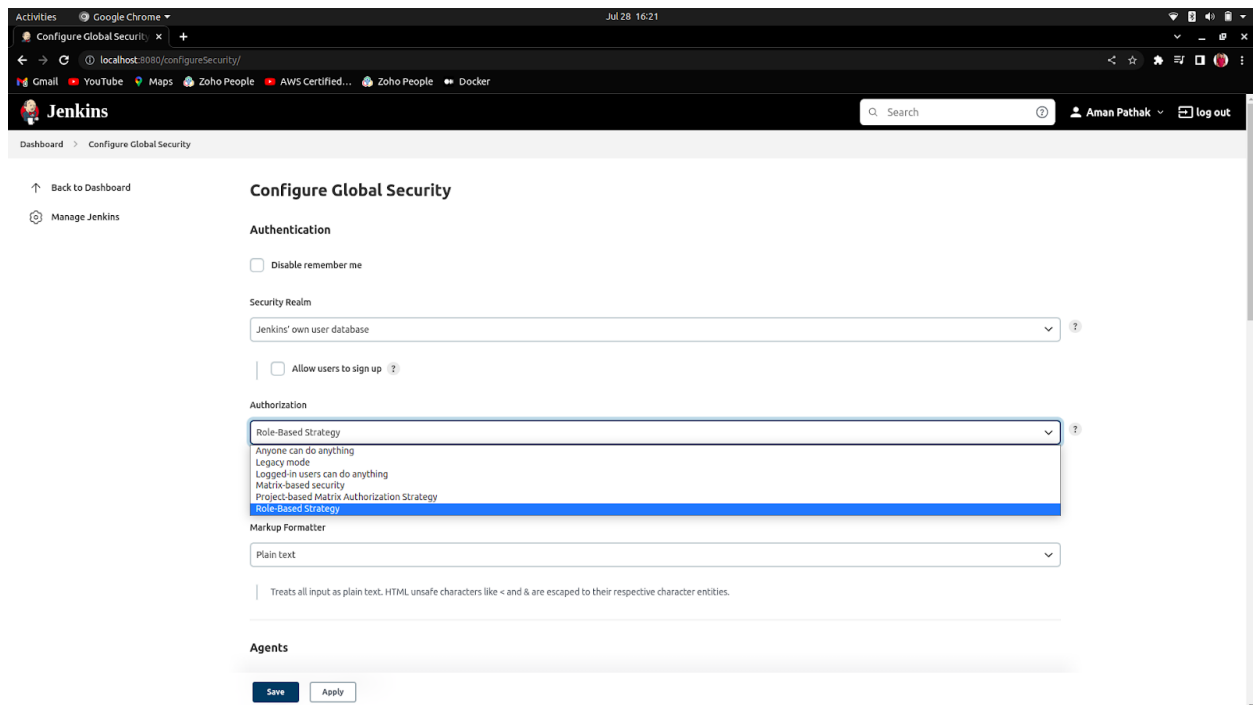
The screenshot shows the Jenkins 'Users' page. The browser's address bar shows 'localhost:8080/securityRealm/'. The Jenkins header is the same as in the previous screenshot. The left sidebar highlights 'Users'. The main content area is titled 'Users' and includes a descriptive text: 'These users can log into Jenkins. This is a sub set of [this list](#), which also contains auto-created users who really just made some commits on some projects and have no direct Jenkins access.'

User ID	Name	
aman19	Aman Pathak	
aman20	Jon snow	

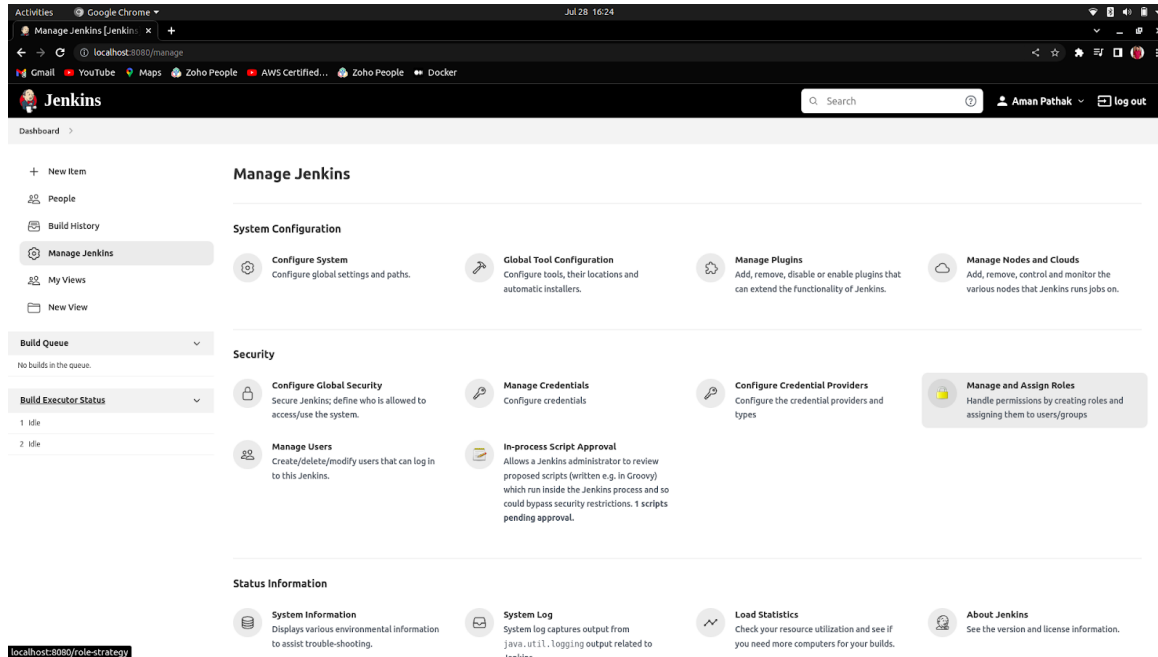
The footer of the page indicates 'Jenkins 2.346.2' and 'afonsof.com/jenkins-material-theme v. 1.3.3'.

Jenkins Role-Based Access Control (RBAC)

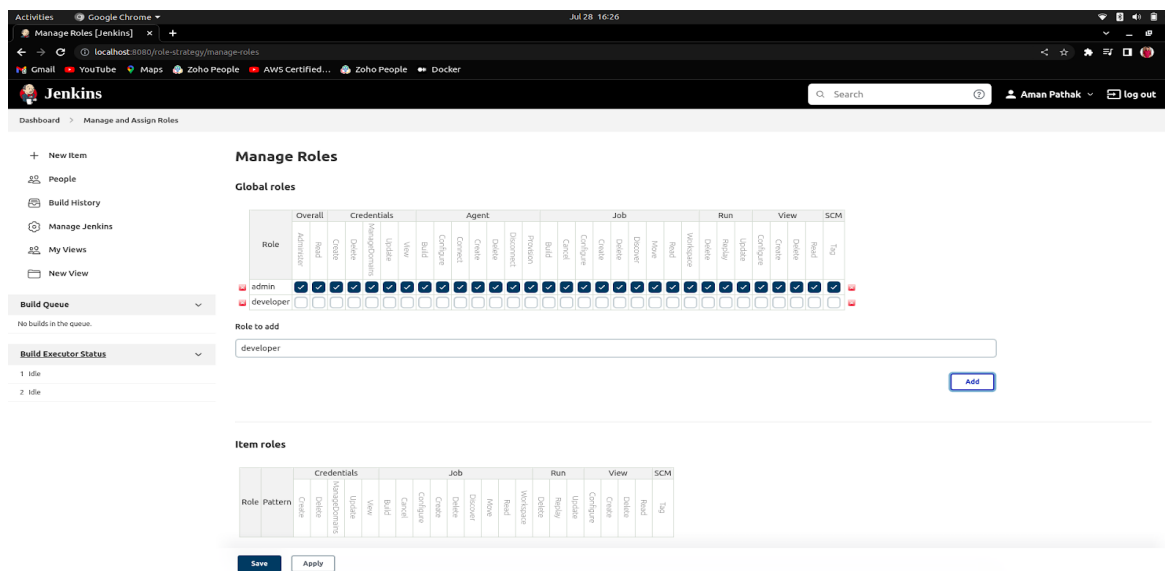
1. Install the plugin named Role-based Authorization Strategy by going into the Manage-Jenkins -> Manage Plugins - Available and then search the plugin name which is written above.
2. Now, you have to configure this plugin by going into Manage Plugin -> Configure Global Security selecting the Role-based Strategy, and applying and saving the configuration.



3. Now, the Manage and Assign roles option will appear under the security section in the Manage Jenkins.



4. Now, go into the section and click on Manage Roles. Then, create the role named <developer> which has no access then apply and save.



5. Now, when you log in with another user e.g. Jon Snow. You'll get this.

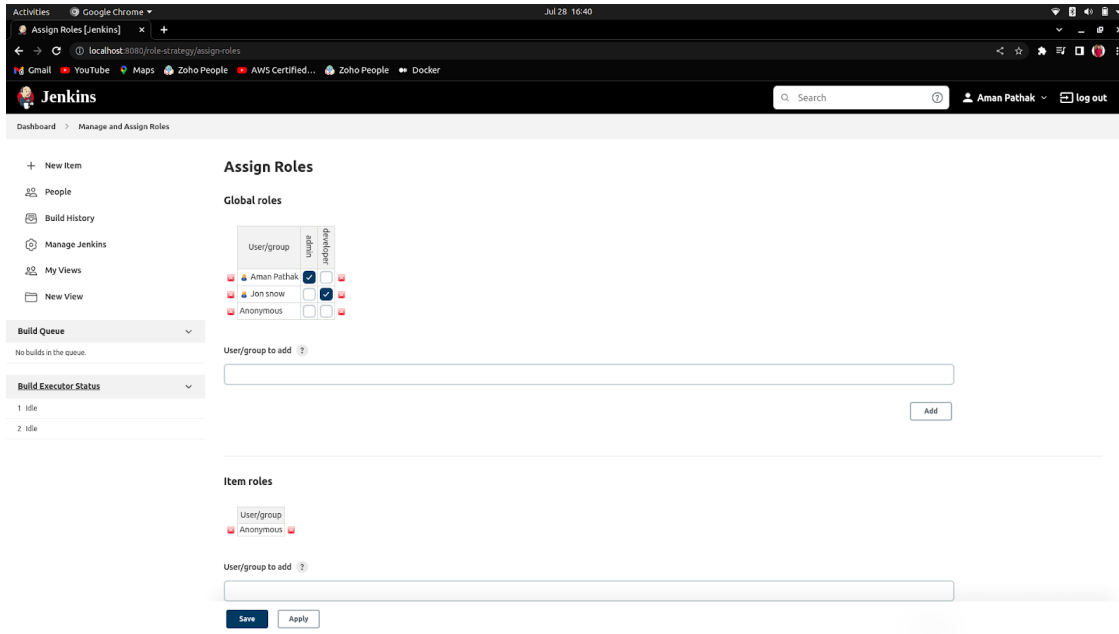


REST API Jenkins 2.346.2

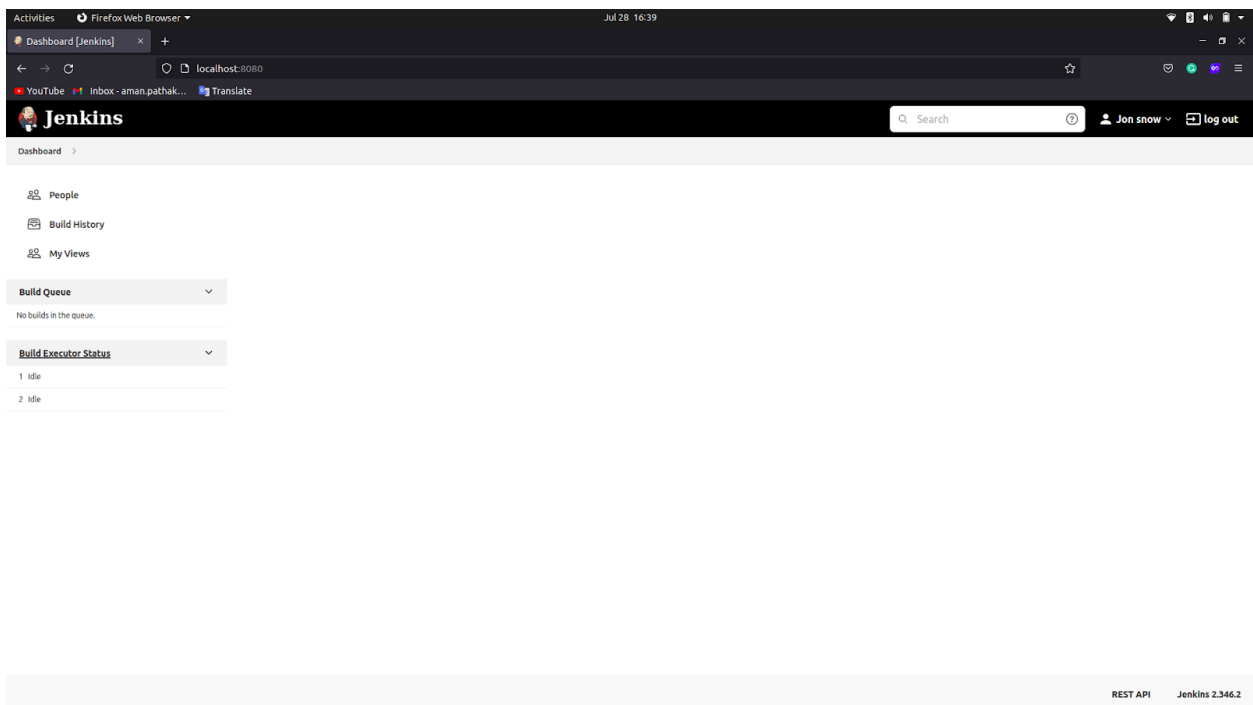
6. Now, go to the Assign Roles and add the user then, give the developer role to that user. Click on Apply and save.

A screenshot of the Jenkins 'Manage Roles' page. The left sidebar contains navigation links: 'New Item', 'People', 'Build History', 'Manage Jenkins', 'My Views', and 'New View'. The main content area is titled 'Manage Roles' and shows a table of 'Global roles'. The table has columns for 'Role', 'Overall', 'Credentials', 'Agent', 'Job', 'Run', 'View', and 'SCM'. The 'admin' role is checked for all permissions, while the 'developer' role is only checked for 'Overall/Read'. Below the table is a 'Role to add' input field and an 'Add' button. At the bottom, there is a 'Save' button and an 'Apply' button.

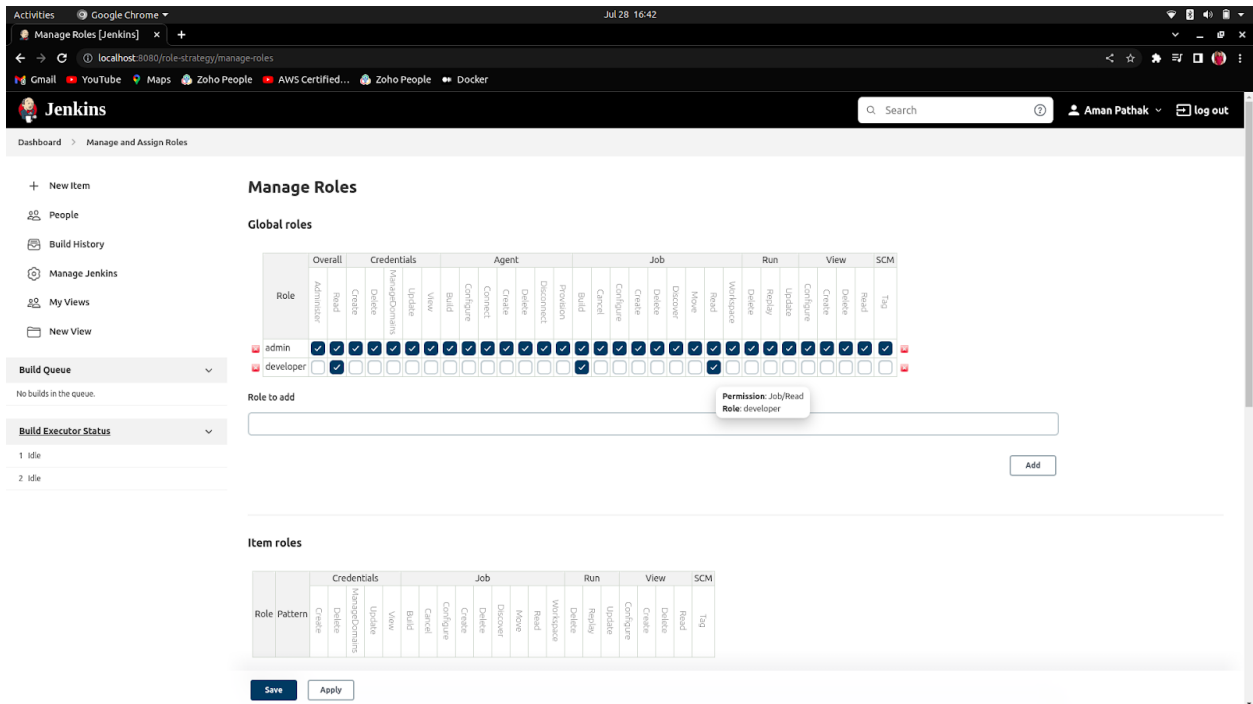
Role	Overall	Credentials	Agent	Job	Run	View	SCM
admin	☒	☒	☒	☒	☒	☒	☒
developer	☒	☐	☐	☐	☐	☐	☐



7. If you give access by changing the roles and assigning the roles to read and build jobs to the other user, It will look like this.

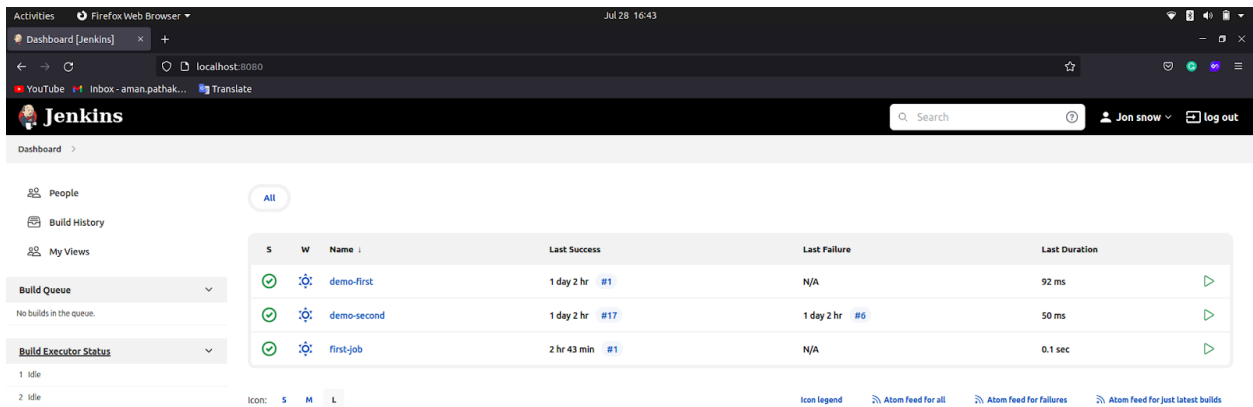


8. If you add one more permission which is job read then, the other can see the jobs.



The screenshot shows the Jenkins 'Manage Roles' page. The 'Global roles' section contains a table with permissions for 'admin' and 'developer' roles. The 'admin' role has all permissions checked, while the 'developer' role has 'Job/Read' checked. A tooltip for 'Job/Read' shows the role 'developer'. Below the table, there is a 'Role to add' section with a search bar and an 'Add' button. The 'Item roles' section is empty.

Role	Overall		Credentials		Agent										Job		Run		View		SCM				
	Administer	Read	Create	Update	View	Build	Configure	Correct	Delete	Disconnect	Provision	Build	Cancel	Configure	Delete	Discover	Move	Read	Workspaces	Create	Update	Replay	Read	Delete	Tag
admin	☒	☒	☒	☒	☒	☒	☒	☒	☒	☒	☒	☒	☒	☒	☒	☒	☒	☒	☒	☒	☒	☒	☒	☒	☒
developer	☐	☒	☐	☐	☐	☐	☐	☐	☐	☐	☐	☒	☐	☐	☐	☐	☐	☒	☐	☐	☐	☐	☐	☐	☐



The screenshot shows the Jenkins Dashboard. The 'Build Queue' section shows 'No builds in the queue.' The 'Build Executor Status' section shows 1 idle and 2 idle executors. The 'All' tab is selected, showing a table of builds.

S	W	Name	Last Success	Last Failure	Last Duration
✓	⚙️	demo-first	1 day 2 hr #1	N/A	92 ms
✓	⚙️	demo-second	1 day 2 hr #17	1 day 2 hr #6	50 ms
✓	⚙️	first-job	2 hr 43 min #1	N/A	0.1 sec

GitHub Job without GitHub Plugin

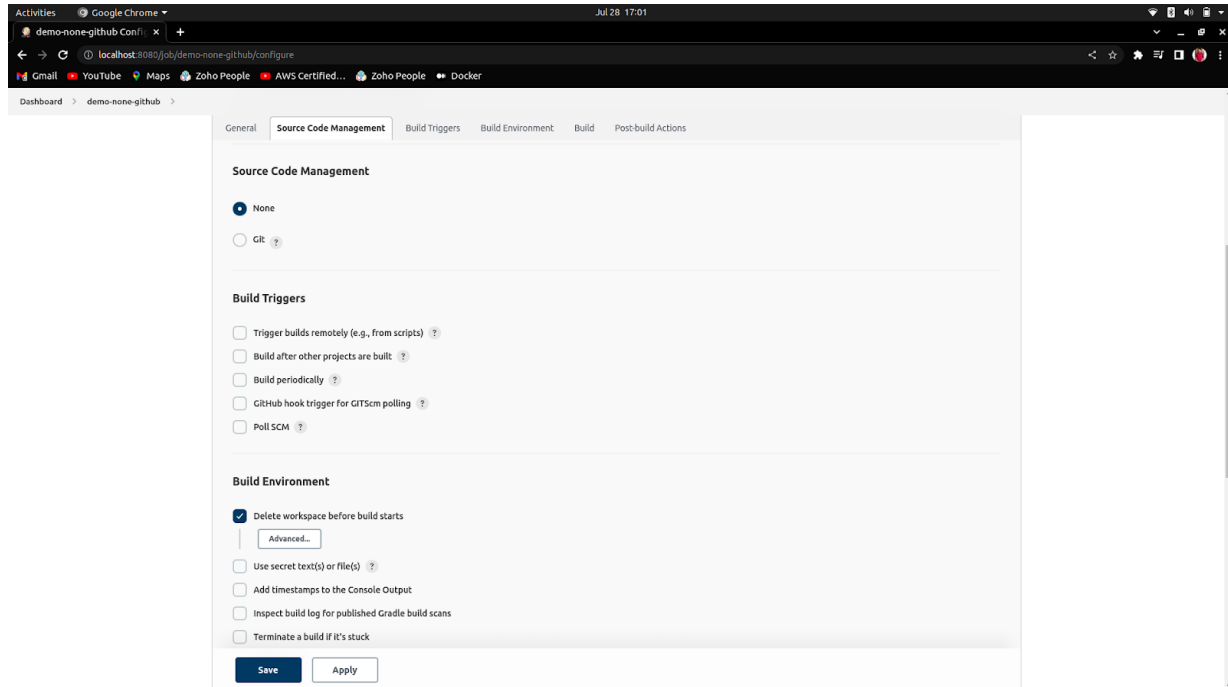
1. Create a Job with freestyle projects.

The screenshot shows the Jenkins 'New Item' configuration page in a Google Chrome browser. The address bar indicates the URL is `localhost:8080/view/all/newJob`. The page title is 'New Item [Jenkins]'. The job name field is filled with 'demo-none-github' and is marked as a required field. Below the name field, there are several project type options, each with an icon and a description:

- Freestyle project**: This is the central feature of Jenkins. Jenkins will build your project, combining any SCM with any build system, and this can be even used for something other than software build.
- Pipeline**: Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.
- Multi-configuration project**: Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
- Folder**: Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.
- Multibranch Pipeline**: Creates a set of Pipeline projects according to detected branches in one SCM repository.
- Organization Folder**: Creates a set of multibranch project subfolders by scanning for repositories.

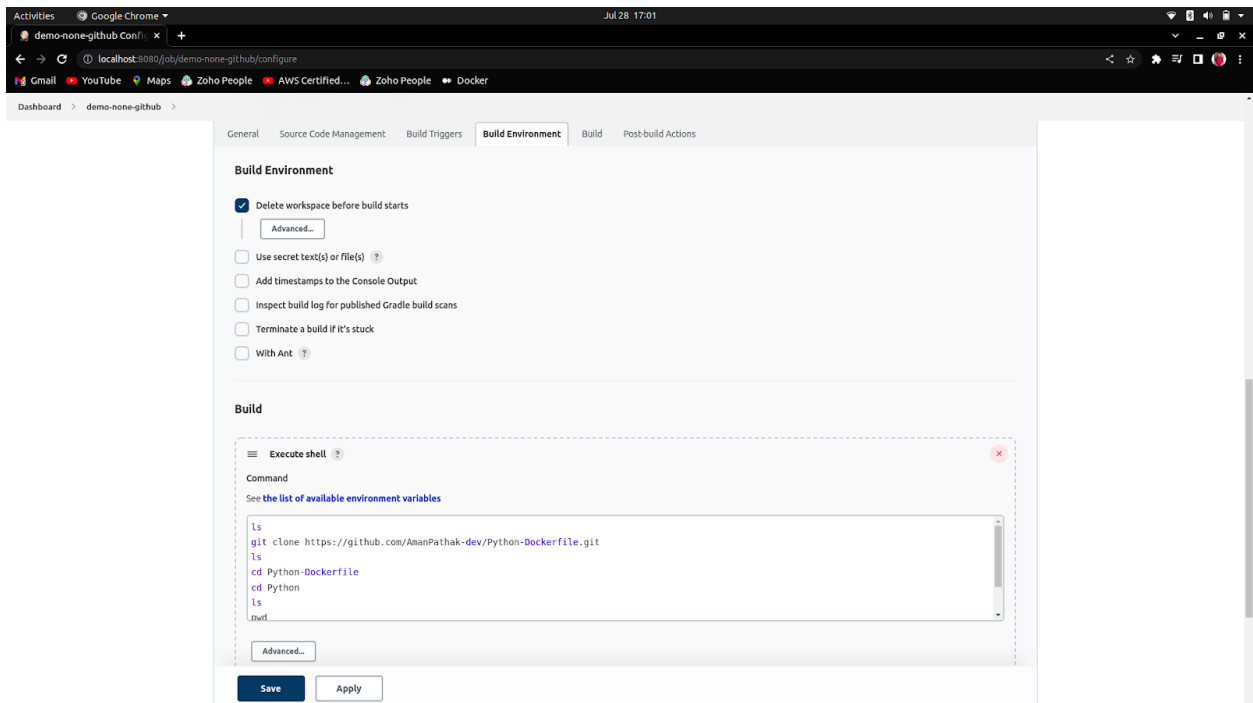
Below these options, there is a section titled 'If you want to create a new item from other existing, you can use this option:' with a 'Copy from' dropdown menu and a text input field labeled 'Type to autocomplete'. At the bottom left, there is an 'OK' button.

2. Take Source code Management as None(for not github).

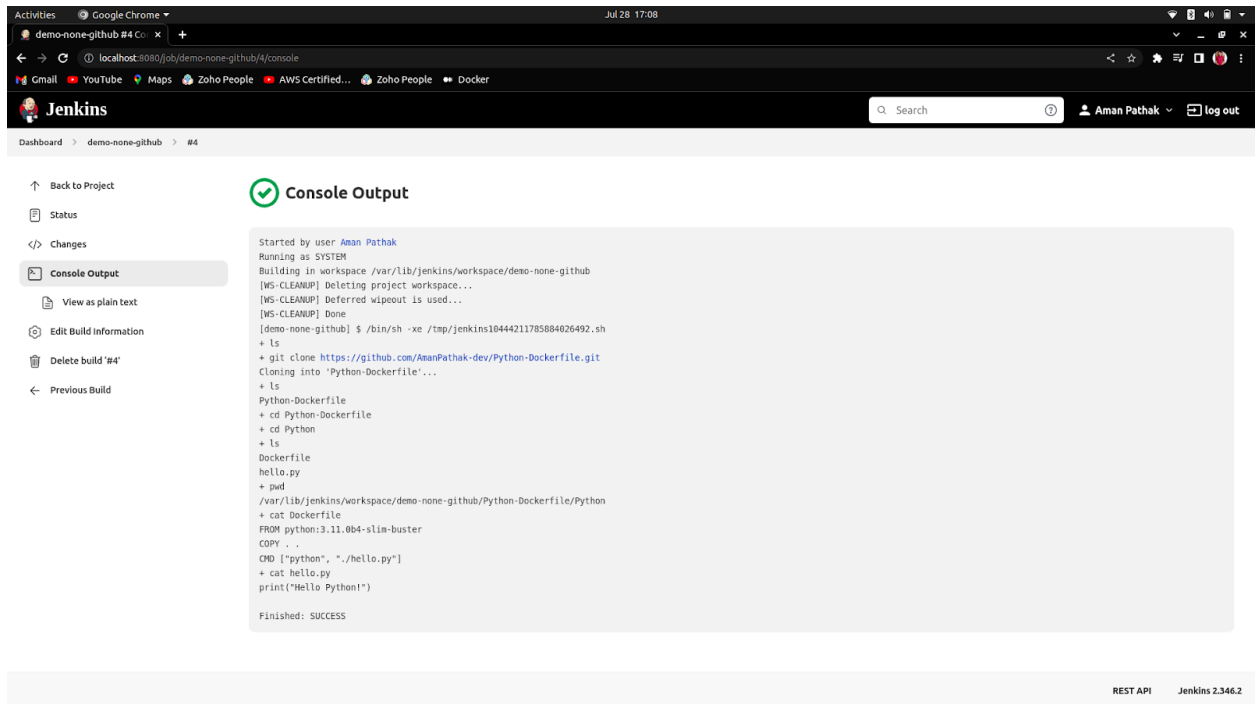


3. Build Environment -> Check the Delete workspace before build starts. **It is checked because when you are going to build the job again it couldn't succeed with the <git clone> because the file will already exist. That's why we checked that.**

Then, write your bash/shell script.



4. The Output



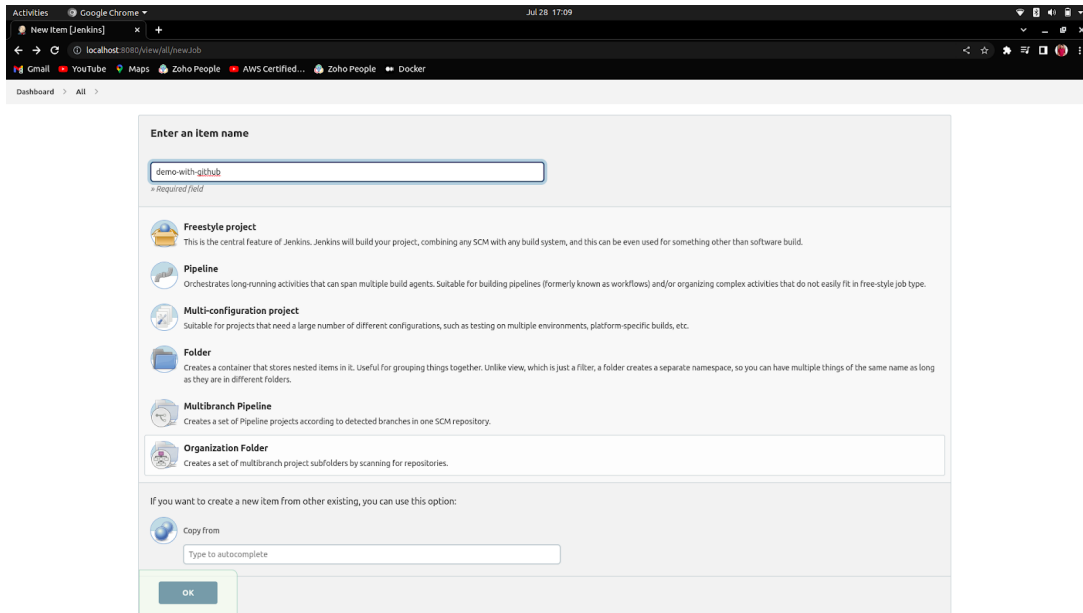
The screenshot shows the Jenkins web interface in a Google Chrome browser. The address bar indicates the URL is `localhost:8080/job/demo-none-github/4/console`. The Jenkins dashboard is visible, with the left sidebar showing navigation options: Back to Project, Status, Changes, Console Output (selected), View as plain text, Edit Build information, Delete build '#4', and Previous Build. The main content area displays the 'Console Output' for build #4, which is successful. The output text is as follows:

```
Started by user Aman Pathak
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/demo-none-github
[WS-CLEANUP] Deleting project workspace...
[WS-CLEANUP] Deferred wipeout is used...
[WS-CLEANUP] Done
[demo-none-github] $ /bin/sh -xe /tmp/jenkins10444211785884626492.sh
+ ls
+ git clone https://github.com/AmanPathak-dev/Python-Dockerfile.git
Cloning into 'Python-Dockerfile'...
+ ls
Python-Dockerfile
+ cd Python-Dockerfile
+ cd Python
+ ls
Dockerfile
hello.py
+ pwd
/var/lib/jenkins/workspace/demo-none-github/Python-Dockerfile/Python
+ cat Dockerfile
FROM python:3.11.0b4-slim-buster
COPY . .
CMD ["python", "./hello.py"]
+ cat hello.py
print("Hello Python!")
Finished: SUCCESS
```

At the bottom right of the interface, the text 'REST API' and 'Jenkins 2.346.2' is visible.

GitHub Job with GitHub Plugin

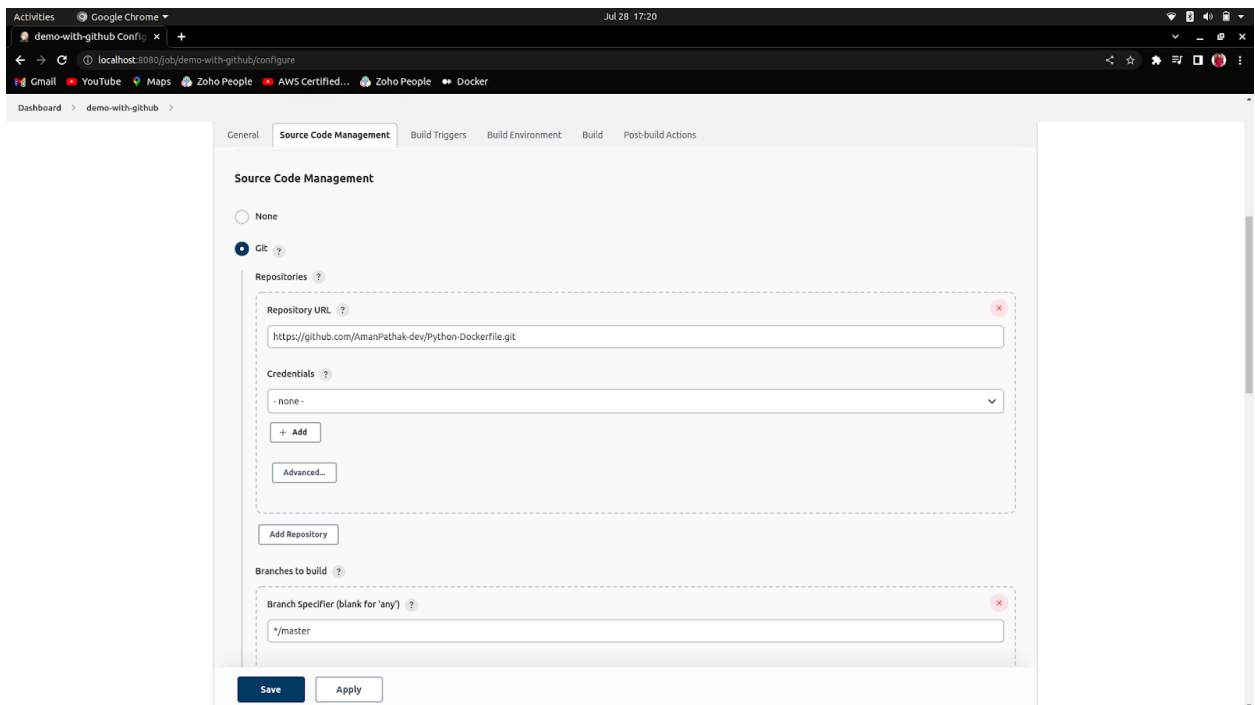
1. Create a freestyle project.



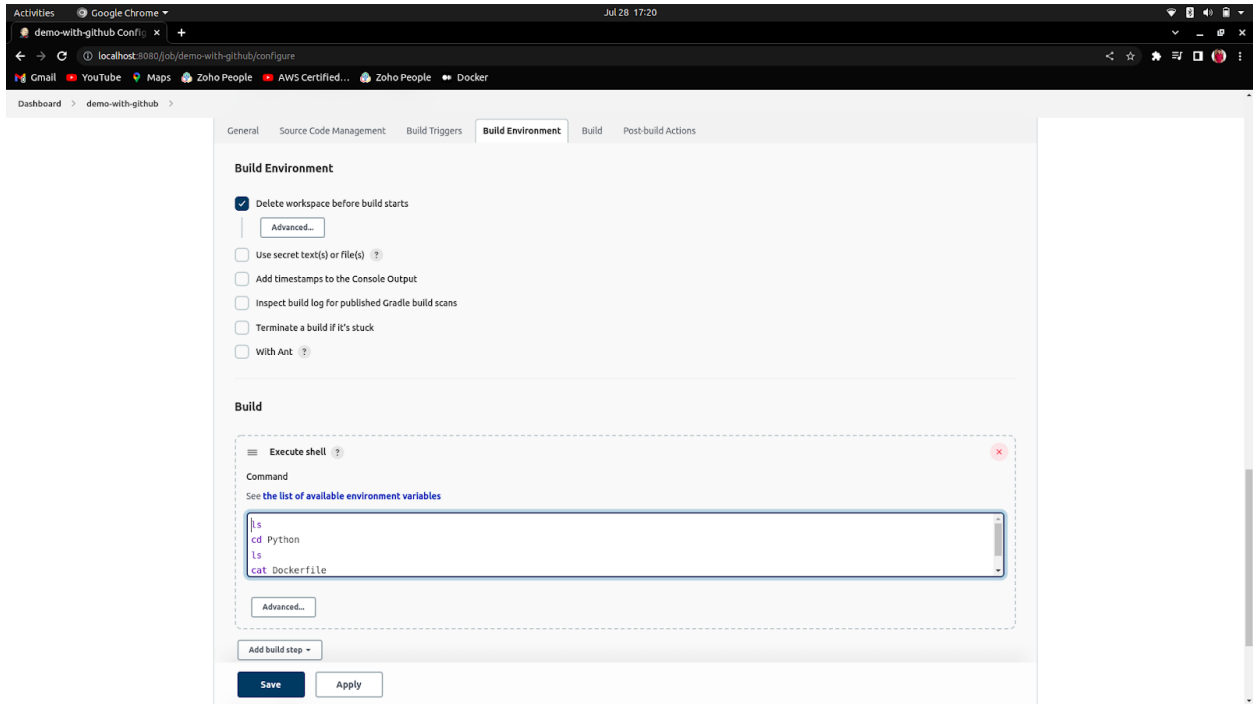
The screenshot shows the Jenkins 'New Item' dialog. The 'Enter an item name' field contains 'demo-with-github'. Below the input field, several project types are listed: Freestyle project, Pipeline, Multi-configuration project, Folder, and Multibranch Pipeline. The 'Freestyle project' is selected. Below these options, there is a section for 'If you want to create a new item from other existing, you can use this option:' with a 'Copy from' dropdown menu. An 'OK' button is at the bottom.

2. Source code Management must be Git.

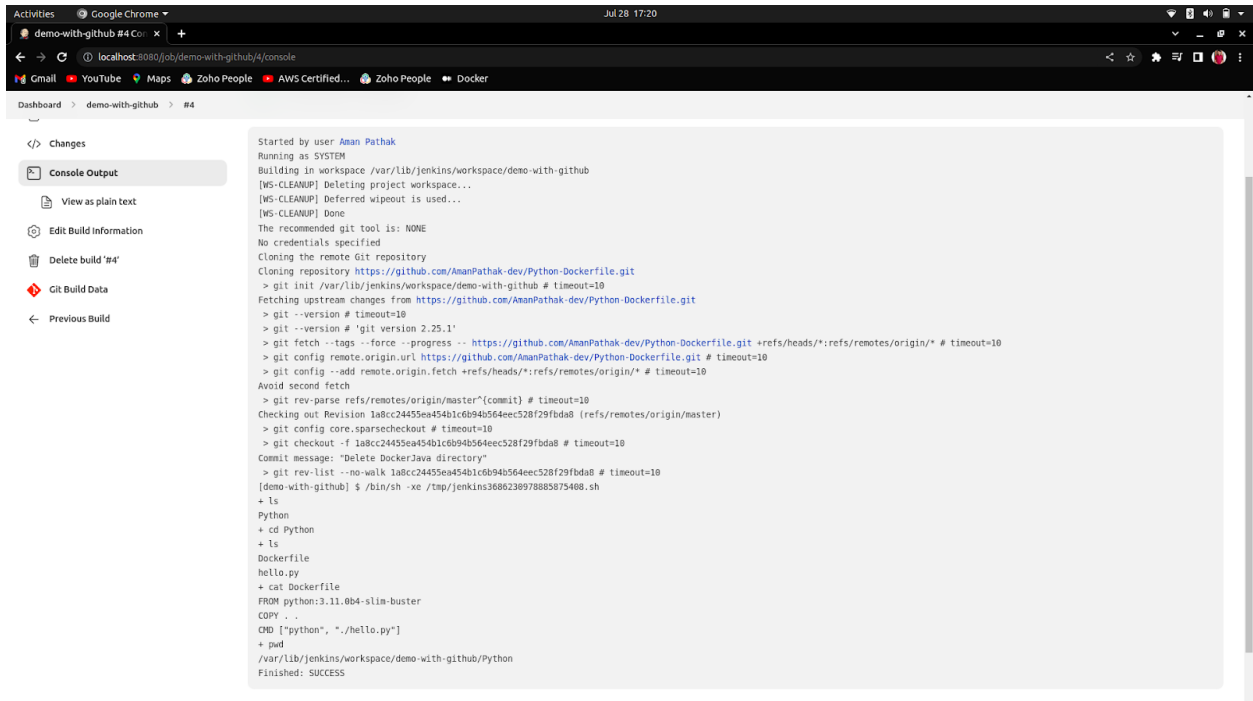
And add the further details as written below.



The screenshot shows the Jenkins 'Source Code Management' configuration page for the 'demo-with-github' project. The 'Source Code Management' tab is selected. The 'None' radio button is unselected, and the 'Git' radio button is selected. Below this, the 'Repositories' section is expanded, showing a 'Repository URL' field with the value 'https://github.com/AmanPathak-dev/Python-Dockerfile.git'. The 'Credentials' dropdown menu is set to 'none'. There is an 'Add' button and an 'Advanced...' link. Below the 'Repositories' section, there is an 'Add Repository' button. The 'Branches to build' section is expanded, showing a 'Branch Specifier (blank for 'any')' field with the value '*/master'. At the bottom, there are 'Save' and 'Apply' buttons.

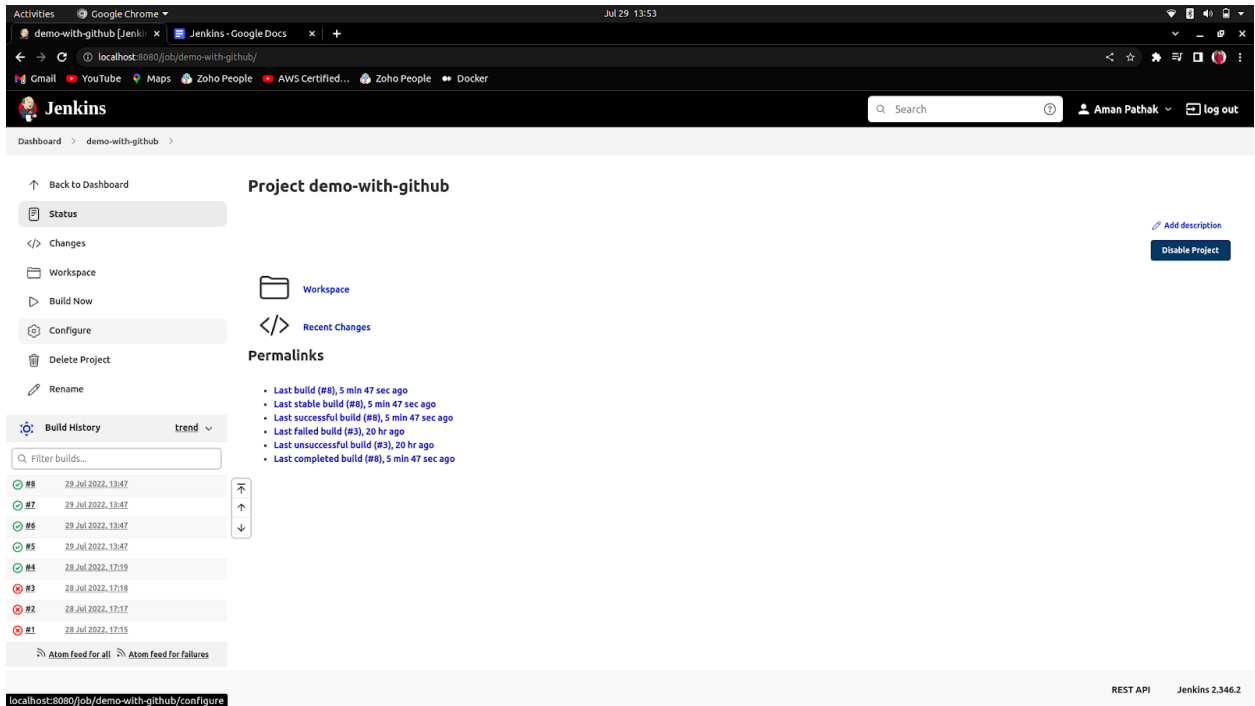


The Output



How to build a job using Trigger Builds remotely (Authentication token)

1. Go to any Job.



The screenshot shows the Jenkins web interface in a Google Chrome browser. The address bar indicates the URL is `localhost:8080/job/demo-with-github/`. The page title is "Project demo-with-github". On the left sidebar, the "Build History" tab is selected, showing a list of builds with their status (green for success, red for failure) and timestamps. The main content area displays "Workspace" and "Recent Changes" sections. Below these, there are "Permalinks" for various build types. At the bottom of the page, the URL `localhost:8080/job/demo-with-github/configure` is visible in the footer.

Dashboard > demo-with-github >

Project demo-with-github

Workspace

Recent Changes

Permalinks

- Last build (#8), 5 min 47 sec ago
- Last stable build (#8), 5 min 47 sec ago
- Last successful build (#8), 5 min 47 sec ago
- Last failed build (#3), 20 hr ago
- Last unsuccessful build (#3), 20 hr ago
- Last completed build (#8), 5 min 47 sec ago

Build History trend

Filter builds...

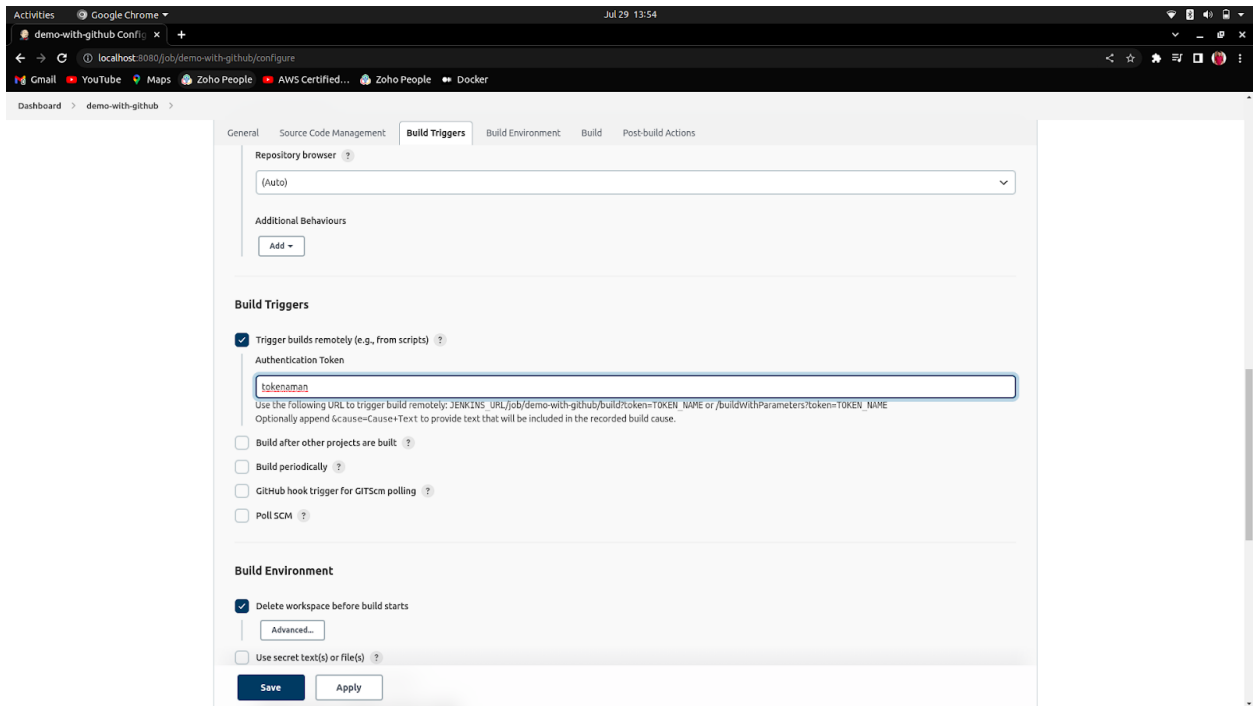
#	Build	Time
#8	29 Jul 2022, 13:47	Success
#7	29 Jul 2022, 13:47	Success
#6	29 Jul 2022, 13:47	Success
#5	29 Jul 2022, 13:47	Success
#4	28 Jul 2022, 17:19	Success
#3	28 Jul 2022, 17:18	Failure
#2	28 Jul 2022, 17:17	Failure
#1	28 Jul 2022, 17:15	Failure

Atom feed for all Atom feed for failures

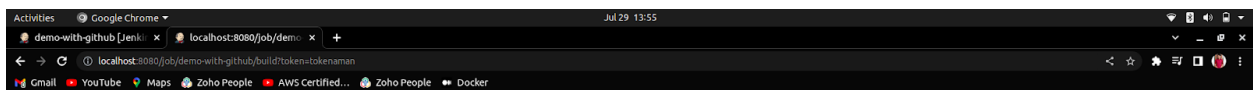
localhost:8080/job/demo-with-github/configure

REST API Jenkins 2.346.2

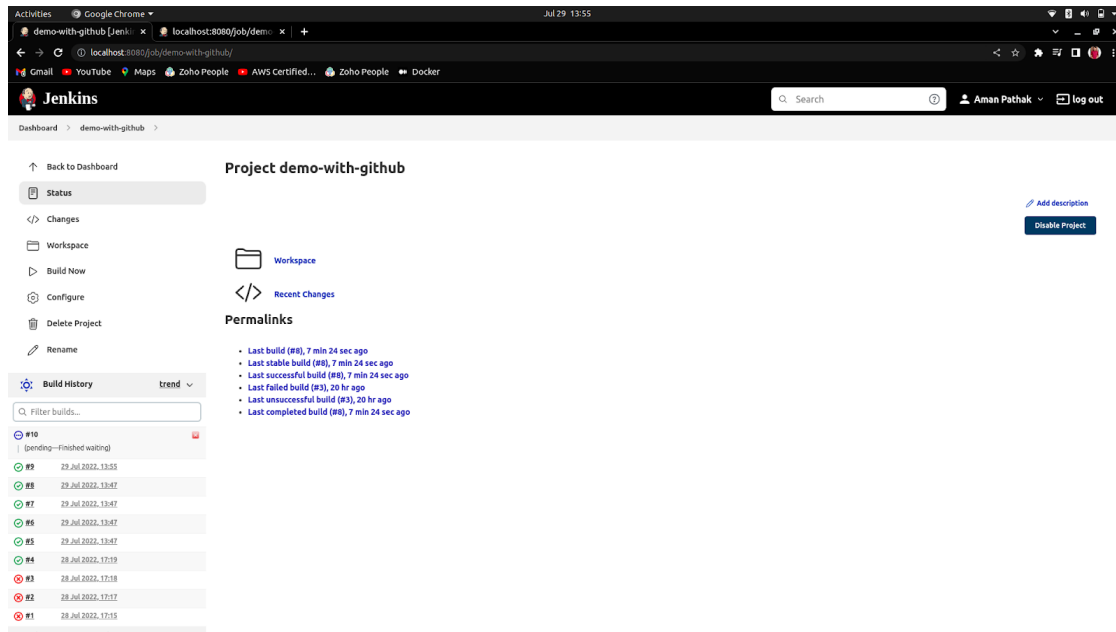
2. Enter the token name inside the Trigger build remotely section.



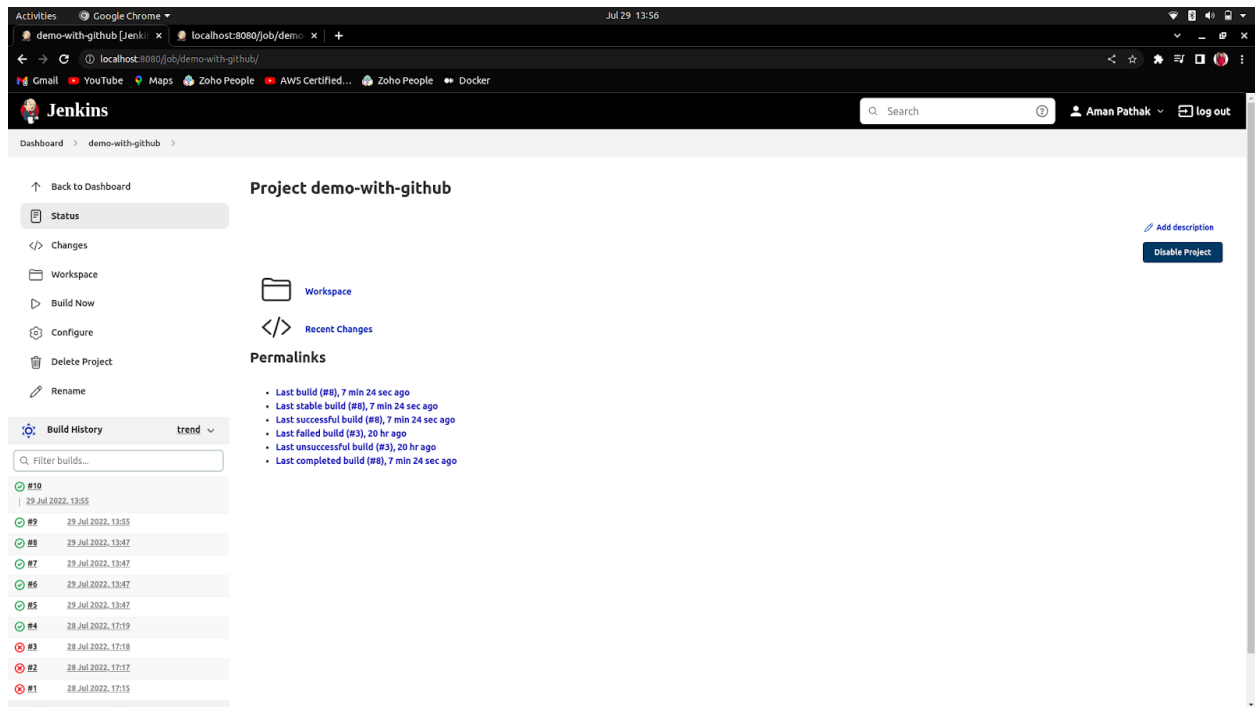
3. Now, cop the URL that was under the token name field and paste in the new tab



4. Now, come to the Jenkins dashboard.
You will see the build has been automatically started.



Build Successful



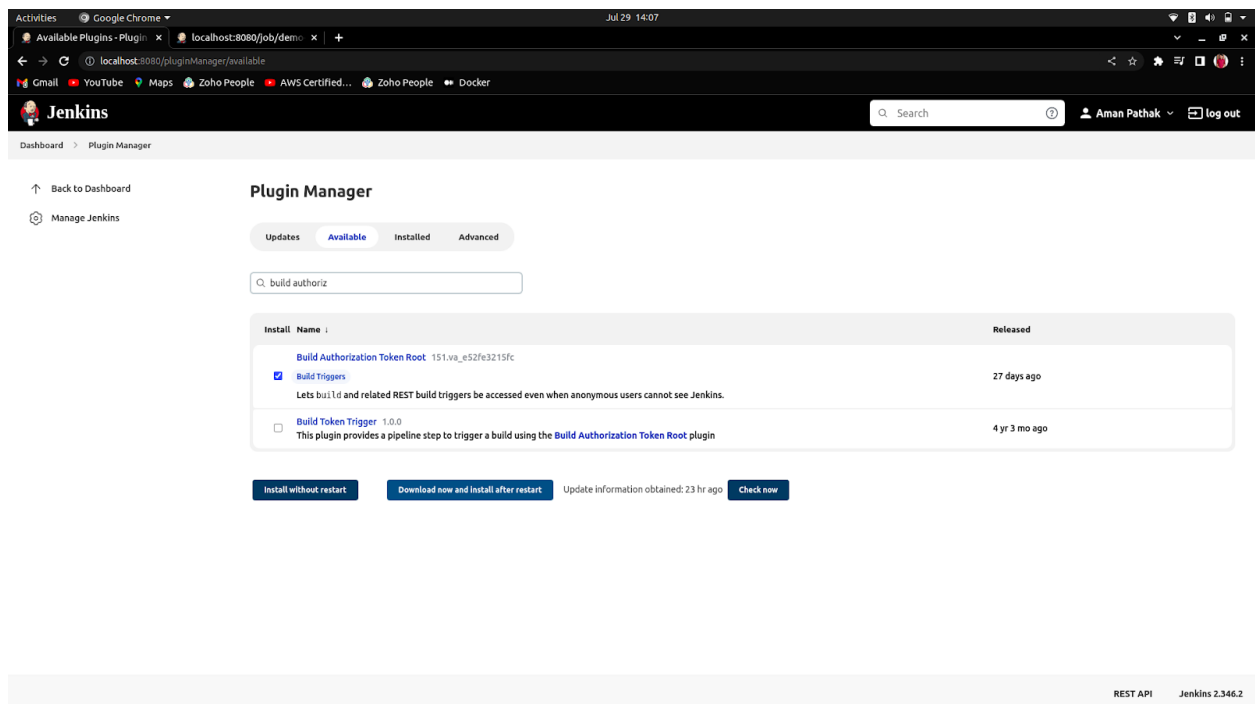
Build Triggers

1. Build Job through Incognito Mode and Terminal

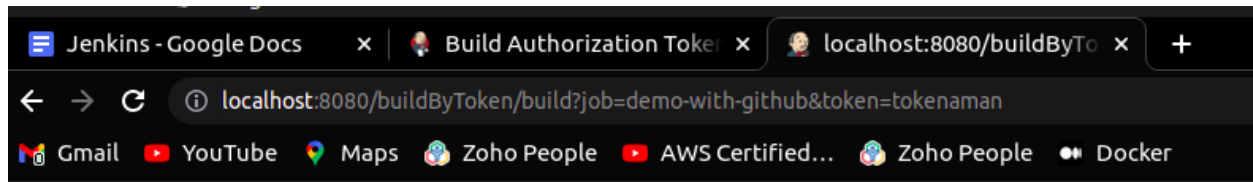
Why?

As I am building a job through just a URL. But when I put the same URL in incognito mode of any browser it asked me to log in again, to resolve that issue. Follow the steps below:

1. First of all, I need to install a plugin named *Build Authorization Token Root* as [Build Authorization Token Link Official Plugin](#)



2. Enter the URL which is written in the tab.



3. The result you can see.

Dashboard

Build Queue

Build Executor Status

S	W	Name	Last Success	Last Failure	Last Duration
✓	✖	demo-with-github	12 min #10	20 hr #3	1.9 sec
✓	✖	demo-none-github	20 min #8	21 hr #3	1 sec
✓	✖	first-job	1 day 0 hr #1	N/A	0.1 sec
✓	✖	demo-first	1 day 23 hr #1	N/A	92 ms
✓	✖	demo-for-other-user	21 hr #2	N/A	62 ms
✓	✖	demo-second	1 day 23 hr #17	1 day 23 hr #6	50 ms

Icon: S M L

Atom feed for all Atom feed for failures Atom feed for just latest builds

REST API Jenkins 2.346.2

The build has been successful.

The screenshot shows the Jenkins web interface in a Google Chrome browser. The address bar shows the URL `localhost:8080/job/demo-with-github/`. The Jenkins logo and name are at the top left, and a search bar and user profile (Aman Pethak) are at the top right. The main content area is titled "Project demo-with-github". On the left sidebar, the "Build History" tab is selected, showing a list of builds. The most recent build, #11, is highlighted in green, indicating it was successful. The build list includes build numbers, status icons (green for success, red for failure), and timestamps. The main content area on the right shows options for "Workspace" and "Recent Changes", a "Permalinks" section with a list of links to specific build details, and buttons for "Add description" and "Disable Project".

Build History

Build Number	Status	Timestamp
#11	Success	29 Jul 2022, 14:11
#10	Success	29 Jul 2022, 13:55
#9	Success	29 Jul 2022, 13:55
#8	Success	29 Jul 2022, 13:47
#7	Success	29 Jul 2022, 13:47
#6	Success	29 Jul 2022, 13:47
#5	Success	29 Jul 2022, 13:47
#4	Success	28 Jul 2022, 17:19
#3	Failure	28 Jul 2022, 17:18
#2	Failure	28 Jul 2022, 17:17

Project demo-with-github

Workspace

Recent Changes

Permalinks

- Last build (#11), 44 sec ago
- Last stable build (#11), 44 sec ago
- Last successful build (#11), 44 sec ago
- Last failed build (#3), 20 hr ago
- Last unsuccessful build (#3), 20 hr ago
- Last completed build (#11), 44 sec ago

Buttons: Add description, Disable Project

Through Terminal

1. Enter the same URL and make one change before the & symbol. Use backslash (\).

```
amanpathak@aman-pathak:~$ curl http://localhost:8080/buildByToken/build?job=demo-with-github\&token=tokenaman
amanpathak@aman-pathak:~$
```

2. The build has been successful.

The screenshot shows the Jenkins web interface in a Google Chrome browser. The address bar displays 'localhost:8080/job/demo-with-github/'. The page title is 'Project demo-with-github'. On the left sidebar, the 'Build History' tab is selected, showing a list of builds. Build #12 is the most recent, with a status of 'Success' (green circle) and a timestamp of '29 Jul 2022, 14:20'. The main content area shows the 'Workspace' and 'Recent Changes' sections. The 'Permalinks' section lists several links for the latest build (#11), including 'Last build', 'Last stable build', 'Last successful build', 'Last failed build', 'Last unsuccessful build', and 'Last completed build'. The 'Build History' table lists builds from #4 to #12, all with 'Success' status.

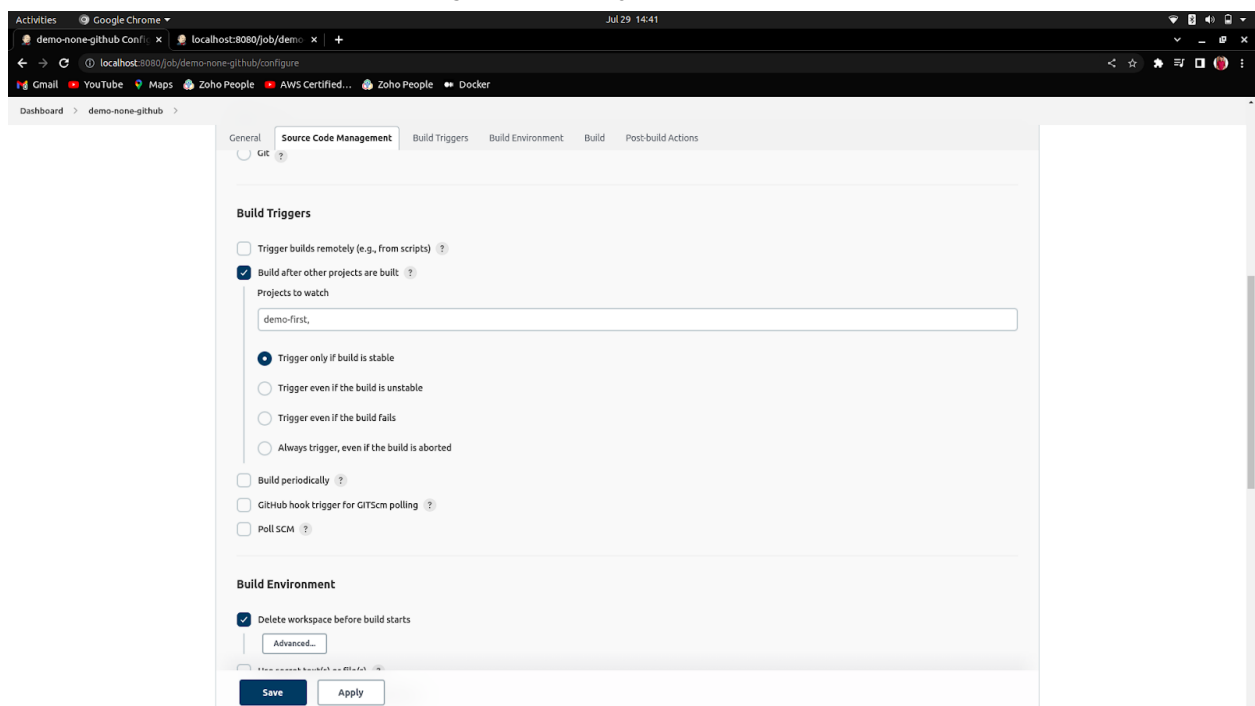
Build Number	Status	Timestamp
#12	Success	29 Jul 2022, 14:20
#11	Success	29 Jul 2022, 14:11
#10	Success	29 Jul 2022, 13:55
#9	Success	29 Jul 2022, 13:55
#8	Success	29 Jul 2022, 13:47
#7	Success	29 Jul 2022, 13:47
#6	Success	29 Jul 2022, 13:47
#5	Success	29 Jul 2022, 13:47
#4	Success	28 Jul 2022, 17:19

2. Build after other projects are built

Why?

When you have to build your specific project after some desired projects. You can use this Build Trigger.

1. Enter the other project that you want to build first.



The screenshot shows the Jenkins web interface in a Google Chrome browser. The address bar indicates the URL is `localhost:8080/job/demo-none-github/configure`. The page title is "demo-none-github". The navigation bar includes "Dashboard" and "demo-none-github". The main configuration area has tabs for "General", "Source Code Management", "Build Triggers", "Build Environment", "Build", and "Post-build Actions". The "Build Triggers" tab is selected.

Build Triggers

- ☐ Trigger builds remotely (e.g., from scripts) ?
- ☒ Build after other projects are built ?

Projects to watch

demo-first,

- ☒ Trigger only if build is stable
- ☐ Trigger even if the build is unstable
- ☐ Trigger even if the build fails
- ☐ Always trigger, even if the build is aborted

- ☐ Build periodically ?
- ☐ GitHub hook trigger for GITScm polling ?
- ☐ Poll SCM ?

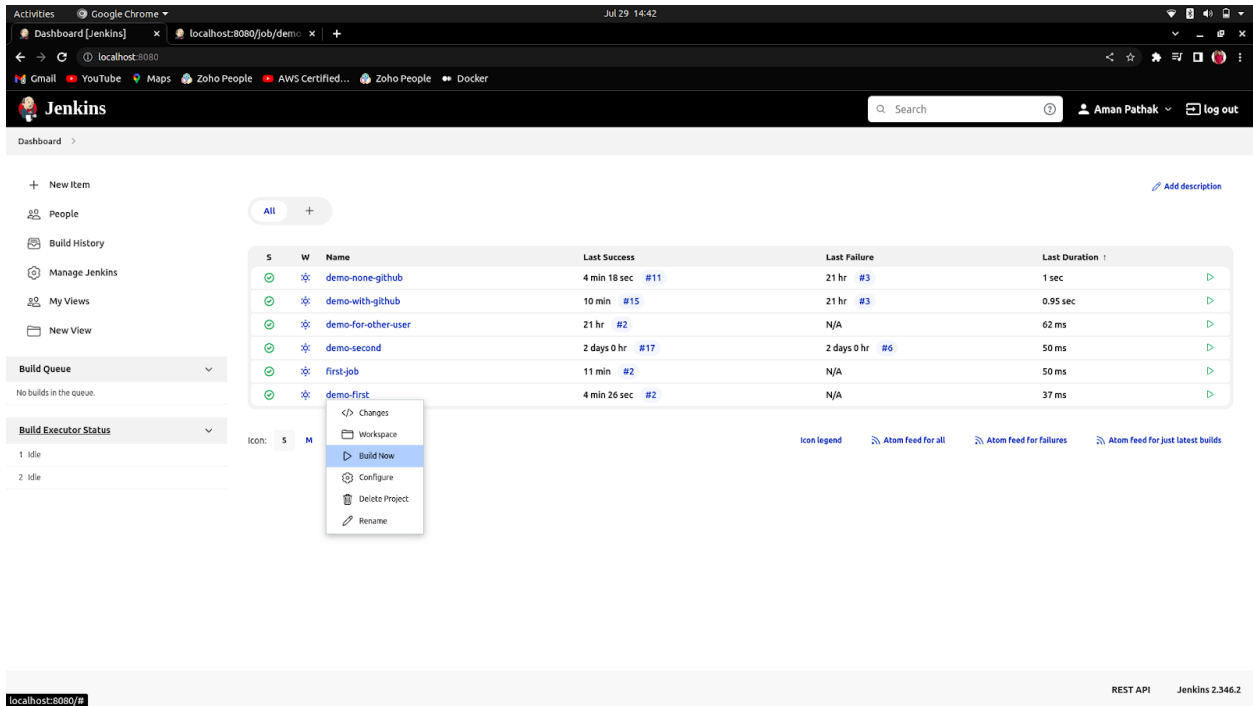
Build Environment

- ☒ Delete workspace before build starts

Advanced...

Save Apply

2. Now, run the project that you entered under the Projects to Watch section.



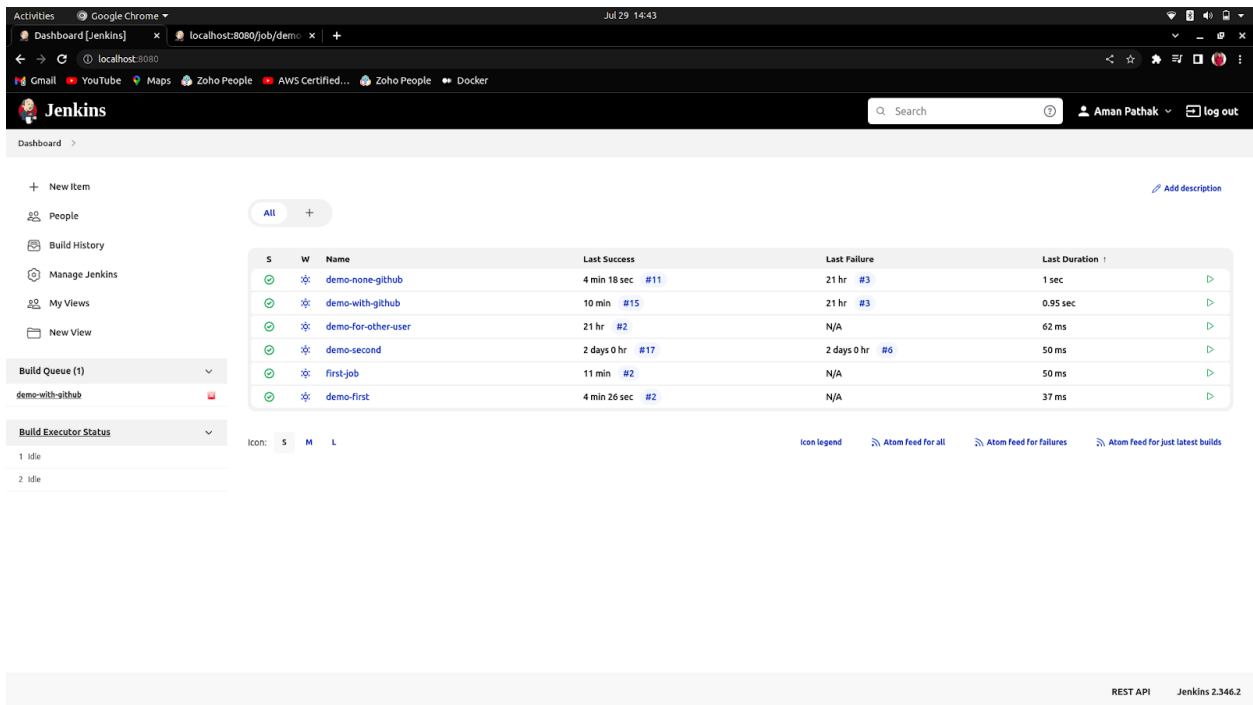
The screenshot shows the Jenkins Dashboard interface. On the left, there's a sidebar with navigation options: New Item, People, Build History, Manage Jenkins, My Views, and New View. Below these are sections for 'Build Queue' (showing 'No builds in the queue') and 'Build Executor Status' (showing 1 idle and 2 busy executors). The main area displays a table of jobs under the 'All' filter. A context menu is open over the 'demo-first' job, showing options: Changes, Workspace, Build Now (highlighted), Configure, Delete Project, and Rename. The table has columns for Status (S), Watch (W), Name, Last Success, Last Failure, and Last Duration. The 'demo-first' job is currently in a successful state.

S	W	Name	Last Success	Last Failure	Last Duration
✓	✱	demo-none-github	4 min 18 sec #11	21 hr #3	1 sec
✓	✱	demo-with-github	10 min #15	21 hr #3	0.95 sec
✓	✱	demo-for-other-user	21 hr #2	N/A	62 ms
✓	✱	demo-second	2 days 0 hr #17	2 days 0 hr #6	50 ms
✓	✱	first-job	11 min #2	N/A	50 ms
✓	✱	demo-first	4 min 26 sec #2	N/A	37 ms

Icon: S M

REST API Jenkins 2.346.2

3. You will see on the left, the project is built.



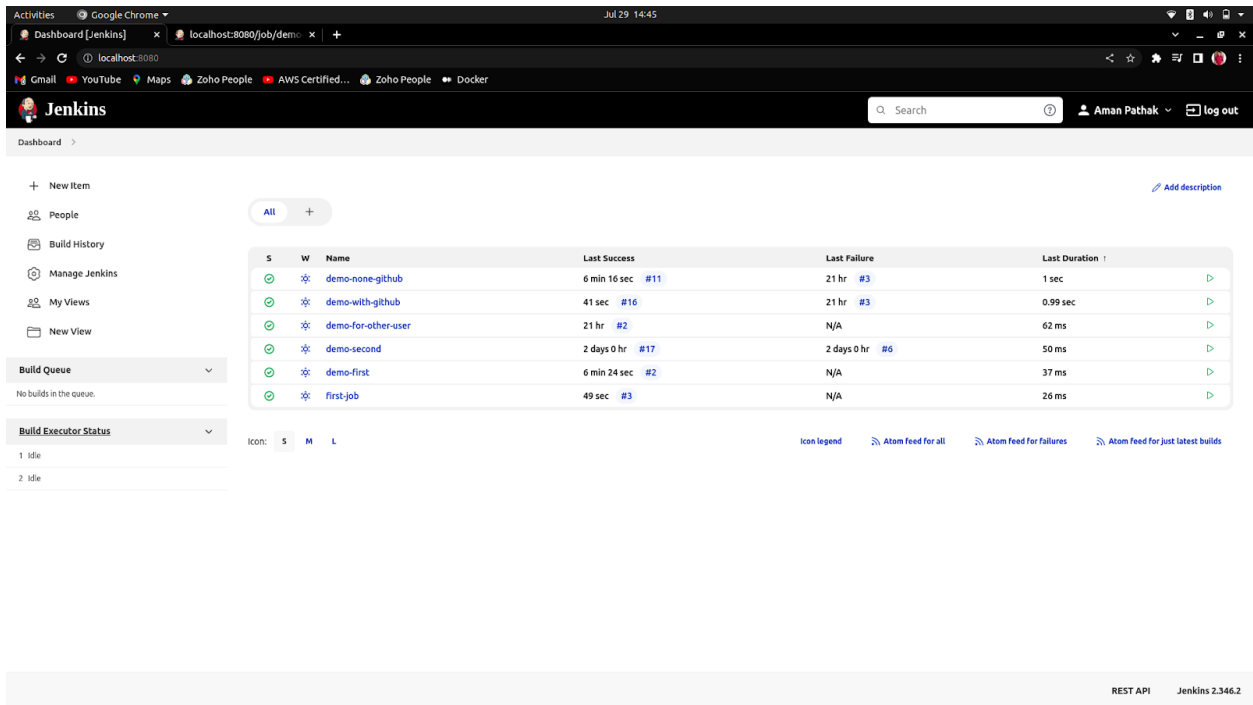
The screenshot shows the Jenkins Dashboard interface after a build. The 'Build Queue' section now shows one build: 'demo-with-github'. The 'Build Executor Status' section still shows 1 idle and 2 busy executors. The main table of jobs remains the same, but the 'demo-first' job is now in a failed state (indicated by a red 'X' icon). The 'demo-with-github' job is now in a successful state (indicated by a green checkmark icon).

S	W	Name	Last Success	Last Failure	Last Duration
✓	✱	demo-none-github	4 min 18 sec #11	21 hr #3	1 sec
✓	✱	demo-with-github	10 min #15	21 hr #3	0.95 sec
✓	✱	demo-for-other-user	21 hr #2	N/A	62 ms
✓	✱	demo-second	2 days 0 hr #17	2 days 0 hr #6	50 ms
✓	✱	first-job	11 min #2	N/A	50 ms
✗	✱	demo-first	4 min 26 sec #2	N/A	37 ms

Icon: S M L

REST API Jenkins 2.346.2

4. If you observe, Both build numbers are incremented by 1.



The screenshot shows the Jenkins Dashboard in a web browser. The dashboard displays a list of builds with columns for status (S), workspace (W), name, last success, last failure, and last duration. The builds are sorted by last success time. The builds listed are:

S	W	Name	Last Success	Last Failure	Last Duration
✓	*	demo-none-github	6 min 16 sec #11	21 hr #3	1 sec
✓	*	demo-with-github	41 sec #16	21 hr #3	0.99 sec
✓	*	demo-for-other-user	21 hr #2	N/A	62 ms
✓	*	demo-second	2 days 0 hr #17	2 days 0 hr #6	50 ms
✓	*	demo-first	6 min 24 sec #2	N/A	37 ms
✓	*	first-job	49 sec #3	N/A	26 ms

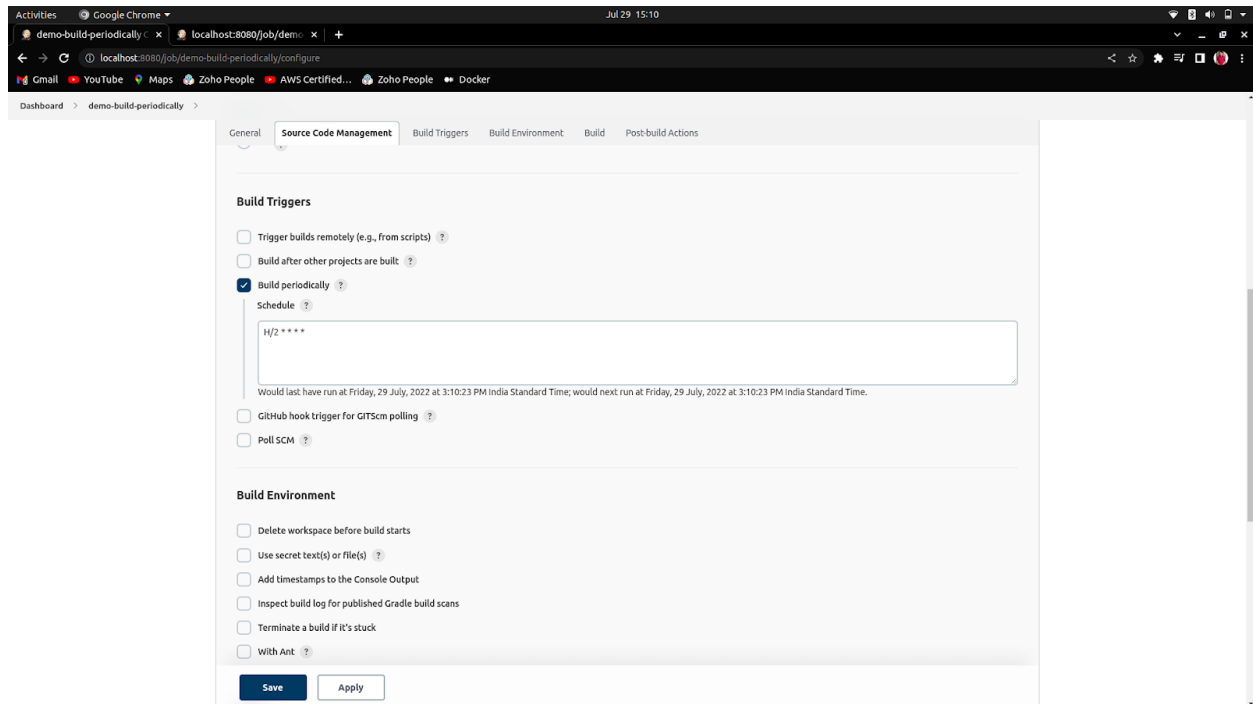
The dashboard also includes a sidebar with links to New Item, People, Build History, Manage Jenkins, My Views, and New View. The Build Queue section shows no builds in the queue. The Build Executor Status section shows 1 idle and 2 idle executors. The bottom right corner of the dashboard displays the REST API and Jenkins version 2.346.2.

3. Build Job Periodically

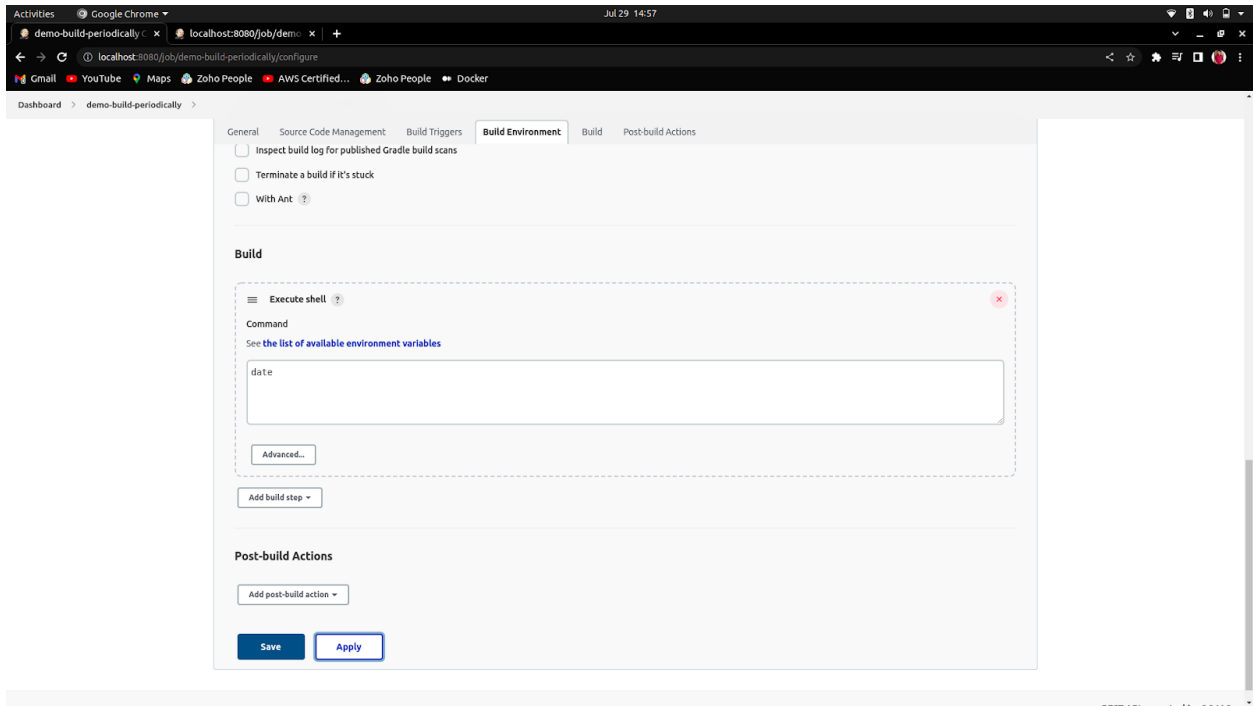
Why?

When you have to build your job at a specific time or same interval e.g. every two minutes. You can use Build periodically under the Build Triggers section.

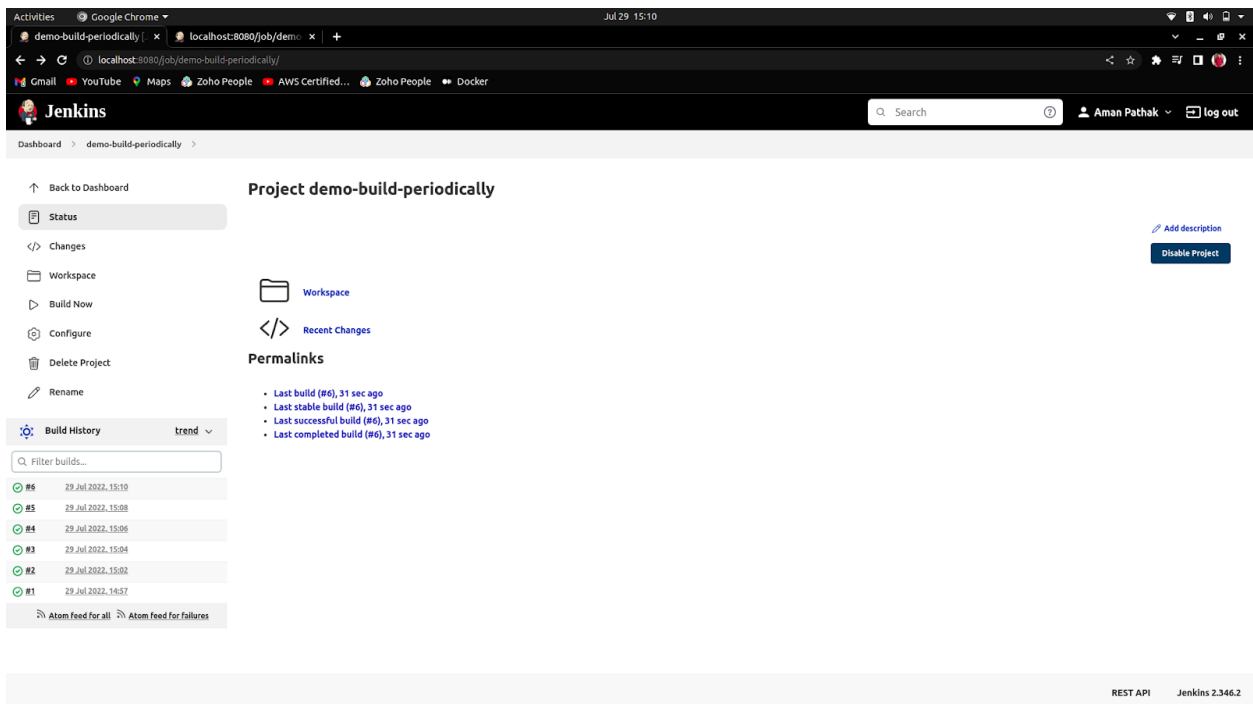
1. Write the time interval the same as written in the cron job.



2. Write the command and apply then save it.



3. Here, you can check it out. The build is automatically done every two minutes.



4. Poll SCM(Source Code Management)

Why?

This Build trigger is the same as Build Periodically. But there is only one difference which is, that the project will fetch from the GitHub repository(mandatory) and if there is no commit in the repository then, the job will not build. But, if there is any commit change in the GitHub repository, then the job will build as a per-time scheduler which is also known as Cronjob/Crontab.

1. Configuration for the Project.

The screenshot shows the Jenkins web interface for configuring a project named 'demo-poll-scm'. The 'Source Code Management' tab is selected. The configuration is set to 'Git'. Under 'Repositories', the 'Repository URL' is 'https://github.com/AmanPathak-dev/Python-Dockerfile.git' and 'Credentials' is set to 'none'. There is an 'Add Repository' button. Under 'Branches to build', the 'Branch Specifier (blank for 'any')' is set to '*/master'. At the bottom, there are 'Save' and 'Apply' buttons.

General Source Code Management Build Triggers Build Environment Build Post-build Actions

Source Code Management

☐ None

☒ Git ?

Repositories ?

Repository URL ?

https://github.com/AmanPathak-dev/Python-Dockerfile.git

Credentials ?

- none -

+ Add

Advanced...

Add Repository

Branches to build ?

Branch Specifier (blank for 'any') ?

*/master

Save Apply

Activities Google Chrome Jul 29 15:35

demo-poll-scm Config Python-Dockerfile/hello... Current Local Time in New...
localhost:8080/job/demo-poll-scm/configure

Dashboard > demo-poll-scm >

General Source Code Management **Build Triggers** Build Environment Build Post-build Actions

Repository browser ?
(Auto)

Additional Behaviours
Add

Build Triggers

☐ Trigger builds remotely (e.g., from scripts) ?
☐ Build after other projects are built ?
☐ Build periodically ?
☐ GitHub hook trigger for GITScm polling ?
☒ Poll SCM ?
 Schedule ?
 H/2 * * * * *
 Would last have run at Friday, 29 July, 2022 at 3:34:30 PM India Standard Time; would next run at Friday, 29 July, 2022 at 3:36:30 PM India Standard Time.
☐ Ignore post-commit hooks ?

Build Environment

Save Apply

Activities Google Chrome Jul 29 15:35

demo-poll-scm Config Python-Dockerfile/hello... Current Local Time in New...
localhost:8080/job/demo-poll-scm/configure

Dashboard > demo-poll-scm >

General Source Code Management Build Triggers **Build Environment** Build Post-build Actions

☐ Ignore post-commit hooks ?

Build Environment

☐ Delete workspace before build starts
☐ Use secret text(s) or file(s) ?
☐ Add timestamps to the Console Output
☐ Inspect build log for published Gradle build scans
☐ Terminate a build if it's stuck
☐ With Ant ?

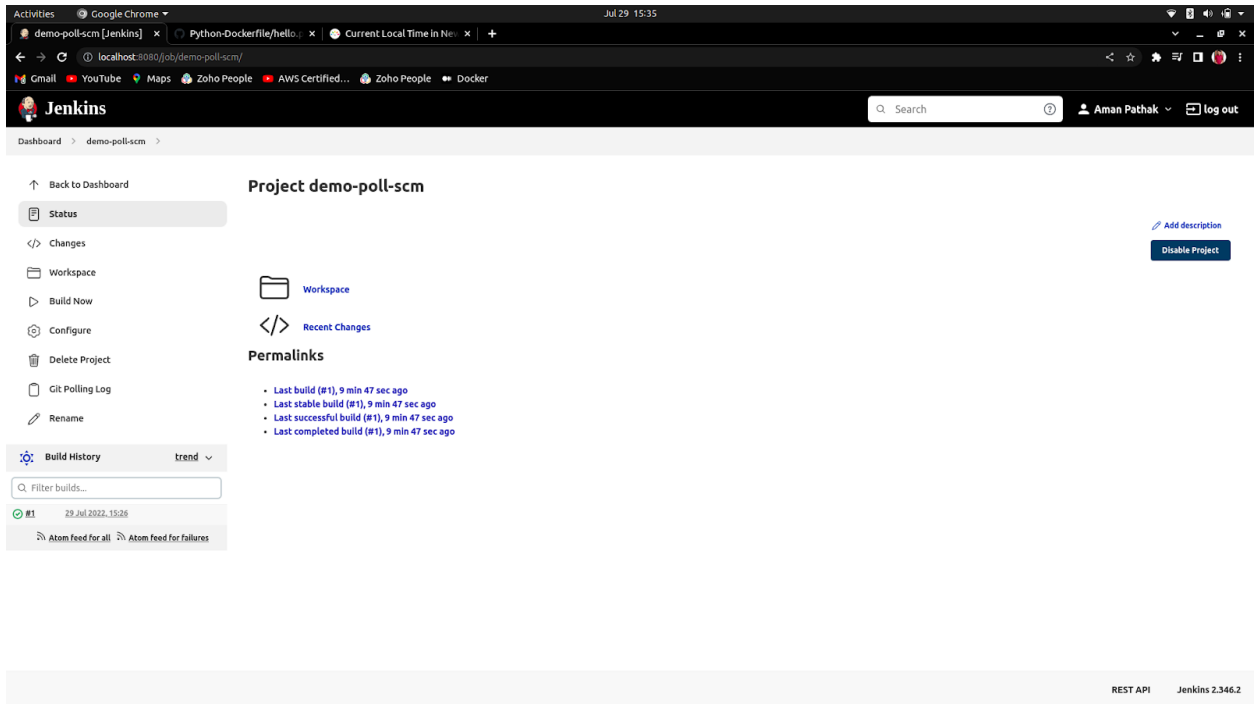
Build

Execute shell ?
 Command
 See [the list of available environment variables](#)
 cd Python
 ls
 cat hello.py
 pwd
 Advanced...

Add build step

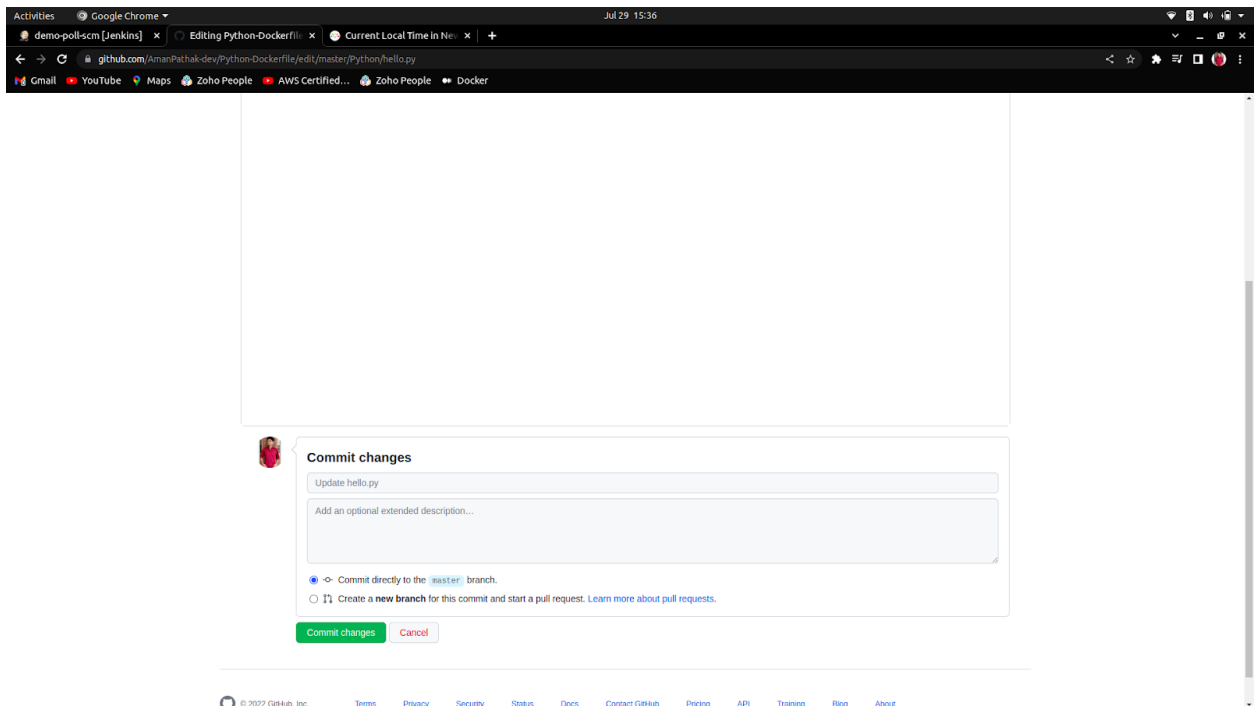
Save Apply

2. Job is built automatically. But then no job build



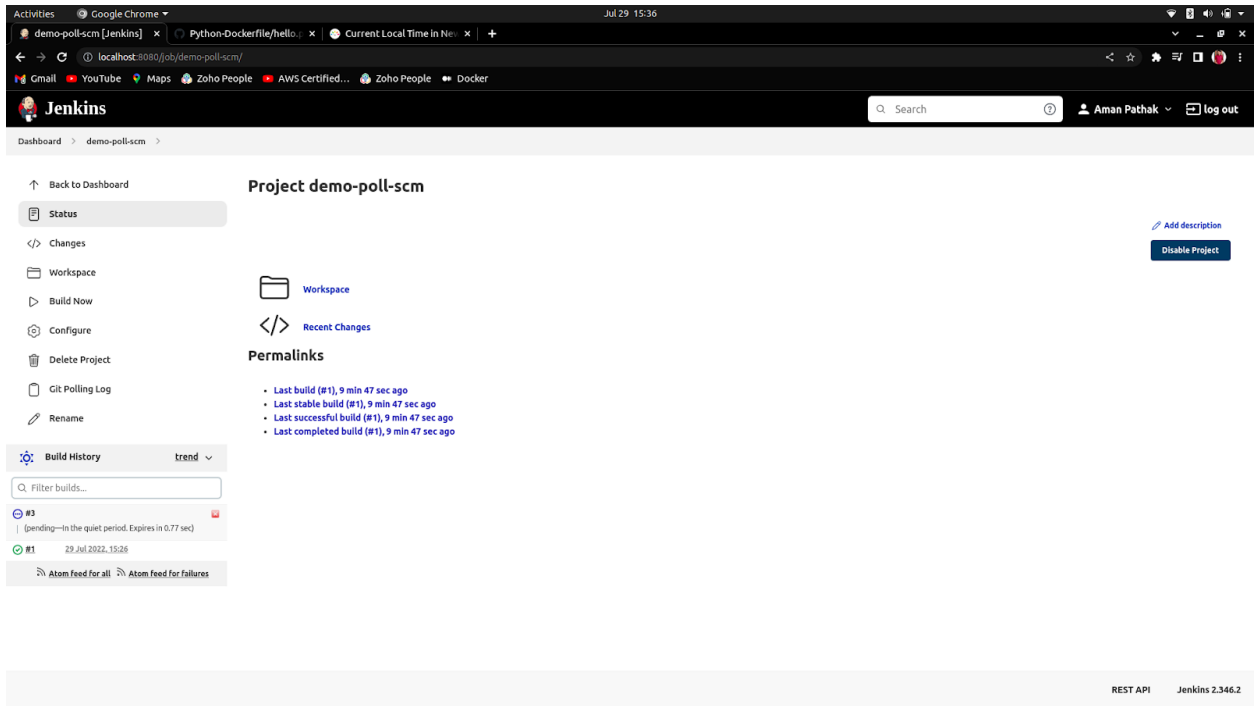
The screenshot shows the Jenkins web interface in a Google Chrome browser. The address bar indicates the URL is `localhost:8080/job/demo-poll-scm/`. The Jenkins logo is in the top left, and a search bar and user profile (Aman Pathak) are in the top right. The main content area is titled "Project demo-poll-scm" and includes a sidebar with navigation links: Back to Dashboard, Status, Changes, Workspace, Build Now, Configure, Delete Project, Git Polling Log, and Rename. The "Build History" section shows a single build (#1) from 29 Jul 2022, 15:28, with a status of "Success". The "Permalinks" section lists four links for the build, all indicating it was completed 9 minutes and 47 seconds ago. A "Disable Project" button is visible in the top right of the main area. The footer shows "REST API" and "Jenkins 2.346.2".

3. Changing some files data of the GitHub repository to build a job.



The screenshot shows the GitHub web interface in a Google Chrome browser. The address bar indicates the URL is `github.com/AmanPathak-dev/Python-Dockerfile/edit/master/Python/hello.py`. The main content area is a large, empty white box. Below it, a "Commit changes" dialog is open, showing a commit message "Update hello.py" and a description field. The dialog also includes options to commit directly to the "master" branch or create a new branch. At the bottom of the dialog are "Commit changes" and "Cancel" buttons. The footer shows the GitHub logo and copyright information: "© 2022 GitHub, Inc." followed by links for Terms, Privacy, Security, Status, Docs, Contact GitHub, Pricing, API, Training, Blog, and About.

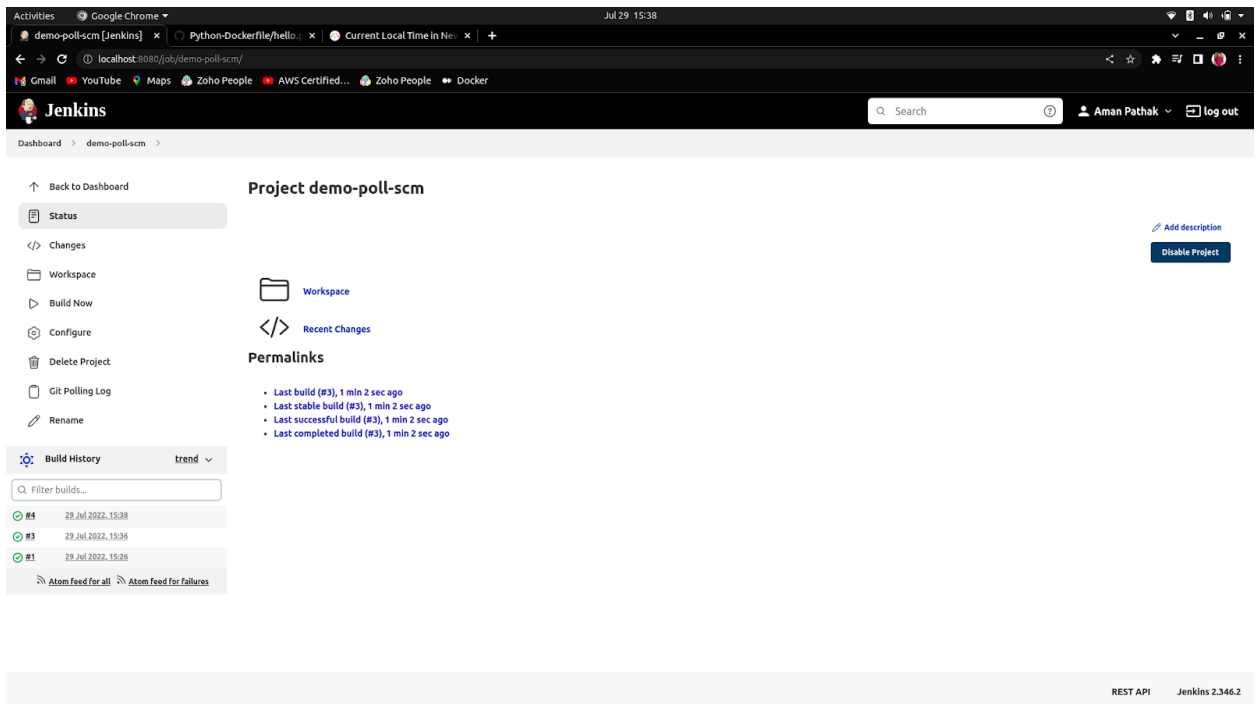
4. As GitHub repository has a new commit. Now, it is building a job.



The screenshot shows the Jenkins interface for the project 'demo-poll-scm'. The left sidebar contains navigation links: Back to Dashboard, Status, Changes, Workspace, Build Now, Configure, Delete Project, Git Polling Log, and Rename. The main content area displays the project name, a 'Workspace' link, 'Recent Changes', and 'Permalinks' for the last build (#1). The 'Build History' section shows a table with two builds: #3 (pending) and #1 (successful). The bottom status bar indicates 'REST API' and 'Jenkins 2.346.2'.

Build Number	Status	Timestamp
#3	Pending	29-Jul-2022, 15:28
#1	Successful	29-Jul-2022, 15:28

5. The Build is successful.



The screenshot shows the Jenkins interface for the project 'demo-poll-scm'. The left sidebar contains navigation links: Back to Dashboard, Status, Changes, Workspace, Build Now, Configure, Delete Project, Git Polling Log, and Rename. The main content area displays the project name, a 'Workspace' link, 'Recent Changes', and 'Permalinks' for the last build (#3). The 'Build History' section shows a table with three builds: #4, #3, and #1, all of which are successful. The bottom status bar indicates 'REST API' and 'Jenkins 2.346.2'.

Build Number	Status	Timestamp
#4	Successful	29-Jul-2022, 15:38
#3	Successful	29-Jul-2022, 15:36
#1	Successful	29-Jul-2022, 15:28

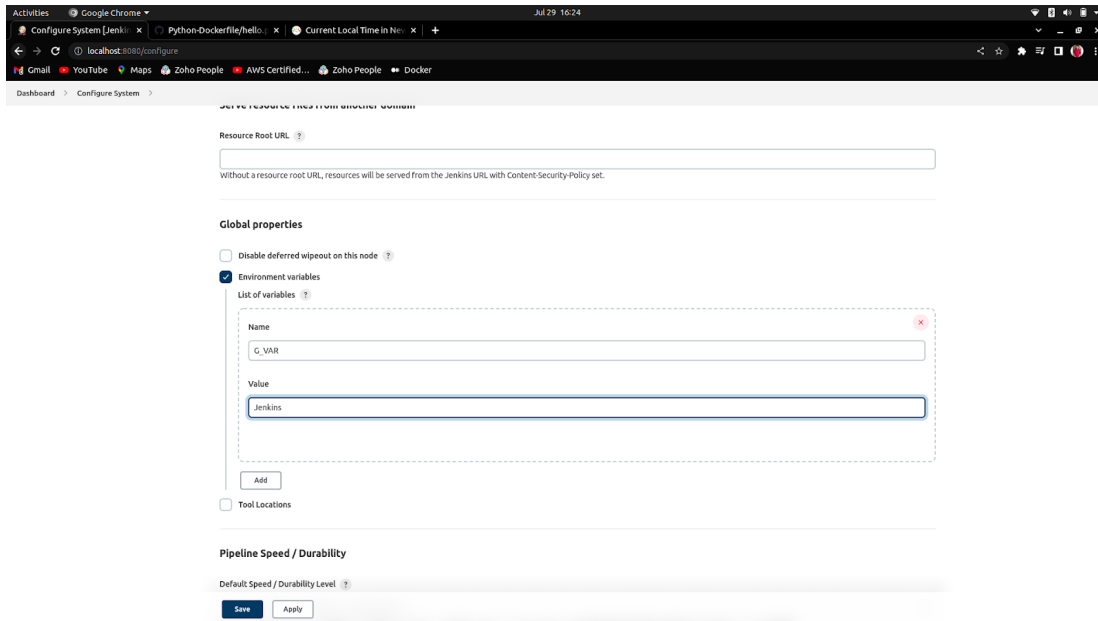
The Output

The screenshot shows a web browser window displaying the Jenkins console output for a build named 'demo-poll-scm #4'. The browser's address bar shows the URL 'localhost:8080/job/demo-poll-scm/4/console'. The Jenkins interface includes a sidebar with navigation links: 'Back to Project', 'Status', 'Changes', 'Console Output' (selected), 'View as plain text', 'Edit Build Information', 'Delete build\'s4', 'Polling Log', 'Git Build Data', and 'Previous Build'. The main area displays the console output, which begins with a green checkmark icon and the title 'Console Output'. The output text describes the build process, including the workspace path, git repository configuration, and the execution of a Python script named 'hello.py'. The build concludes with the message 'Finished: SUCCESS'.

```
Started by an SCM change
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/demo-poll-scm
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/demo-poll-scm/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/AmnPathak-dev/Python-Dockerfile.git # timeout=10
Fetching upstream changes from https://github.com/AmnPathak-dev/Python-Dockerfile.git
> git --version # timeout=10
> git --version # 'git version 2.25.1'
> git fetch --tags --force --progress -- https://github.com/AmnPathak-dev/Python-Dockerfile.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/master^{commit} # timeout=10
Checking out Revision 3004127269ff7e2a1aba2f60714445bb175dd7ac (refs/remotes/origin/master)
> git config core.sparsecheckout # timeout=10
> git checkout -f 3004127269ff7e2a1aba2f60714445bb175dd7ac # timeout=10
Commit message: "Update hello.py"
> git rev-list --no-walk 54734abf0071e753c3eddb50be0cc1ac123bde17 # timeout=10
[demo-poll-scm] $ /bin/sh -xe /tmp/jenkins9595245220184034307.sh
+ ls
Python
+ cd Python
+ ls
Dockerfile
hello.py
+ cat hello.py
print("Hello Jenkins!")
print("Is this a Docker?")
print("Cool! There are Two tools is working")
+ pwd
/var/lib/jenkins/workspace/demo-poll-scm/Python
Finished: SUCCESS
```

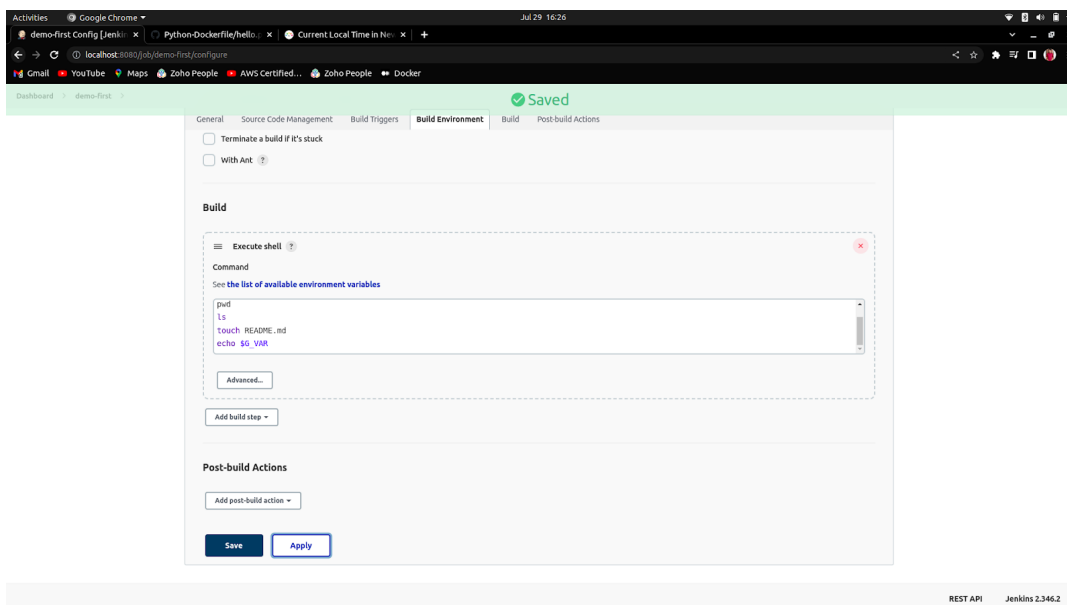
How to define variables globally

1. Go to the Manage Jenkins -> Configure System and scroll down then, look for Global properties.



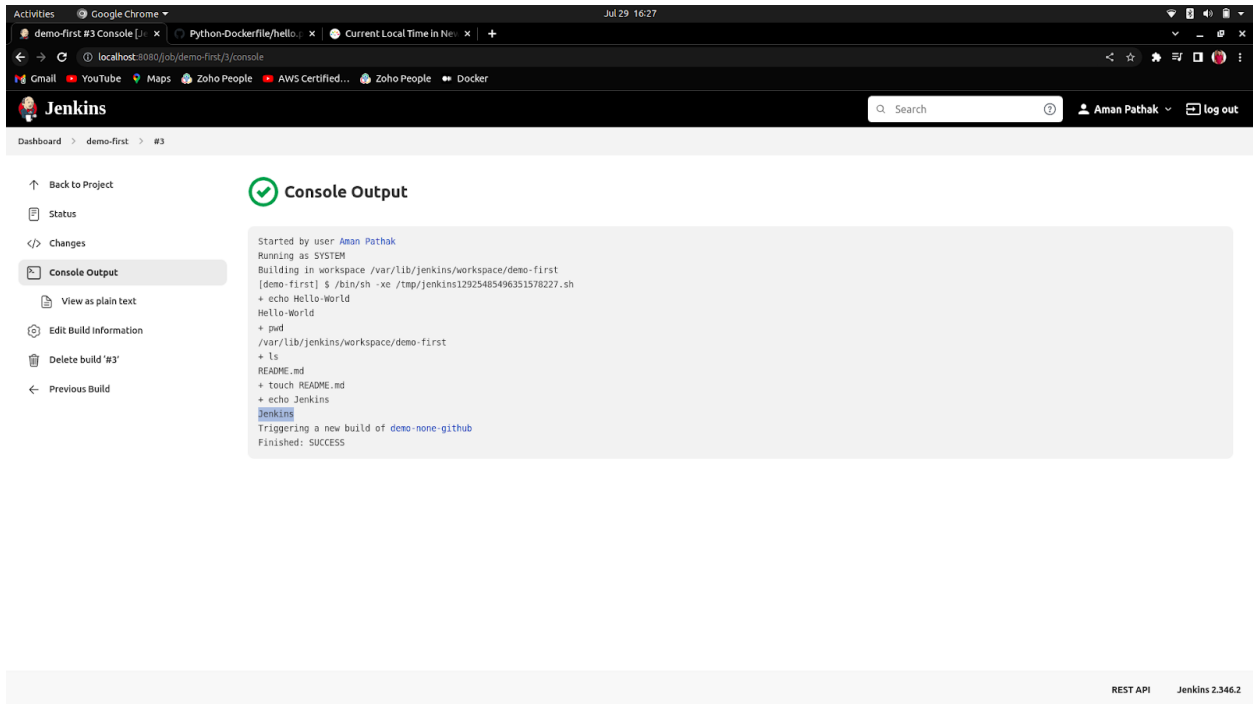
The screenshot shows the Jenkins 'Configure System' page. The 'Global properties' section is active, with the 'Environment variables' checkbox checked. A new variable is being added with the name 'C_VAR' and the value 'Jenkins'. The 'Add' button is visible below the input fields. The 'Resource Root URL' field is empty, and the 'Pipeline Speed / Durability' section is also visible.

2. Now, Create a job and print that value of variable by passing the environment variable.



The screenshot shows the Jenkins 'demo-first' job configuration page. The 'Build' section is active, and a new build step 'Execute shell' has been added. The command field contains the following text: `pwd`, `ls`, `touch README.md`, and `echo $C_VAR`. The 'Save' button is visible at the bottom of the configuration page.

3. Here, you can see the value of variable is appeared.



The screenshot shows the Jenkins web interface in a Google Chrome browser. The address bar indicates the URL is `localhost:8080/job/demo-first/console`. The Jenkins header shows the user `Aman Pathak` and a `log out` button. The left sidebar contains navigation links: `Back to Project`, `Status`, `Changes`, `Console Output` (selected), `View as plain text`, `Edit Build Information`, `Delete build '#3'`, and `Previous Build`. The main content area is titled `Console Output` with a green checkmark icon. The output text is as follows:

```
Started by user Aman Pathak
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/demo-first
[demo-first] $ /bin/sh -xe /tmp/jenkins12925485496351578227.sh
+ echo Hello-World
Hello-World
+ pwd
/var/lib/jenkins/workspace/demo-first
+ ls
README.md
+ touch README.md
+ echo Jenkins
Jenkins
Triggering a new build of demo-none github
Finished: SUCCESS
```

At the bottom right of the page, there is a footer with the text `REST API` and `Jenkins 2.346.2`.

Parameterised in Job

1. Define the variable and pass the default value.

The screenshot shows the Jenkins configuration page for a job named "demo-parameterized-job". The "General" tab is selected, and the checkbox "This project is parameterised" is checked. Below this, there are two parameter types defined:

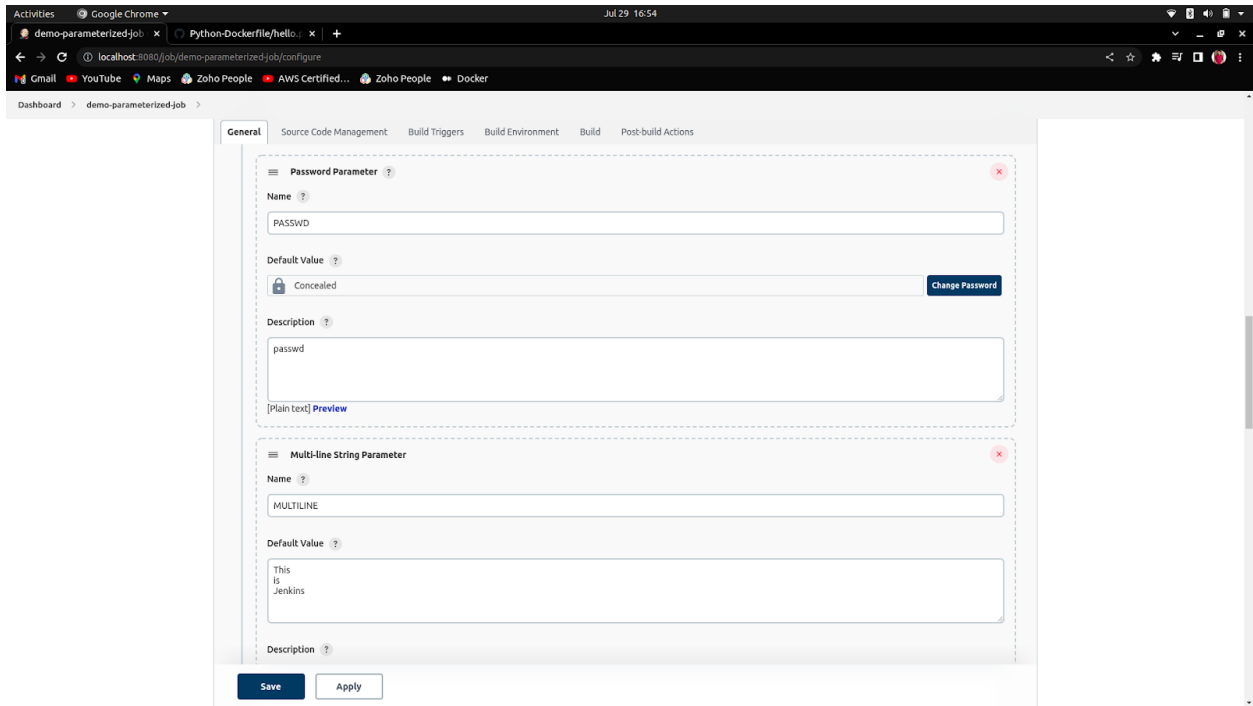
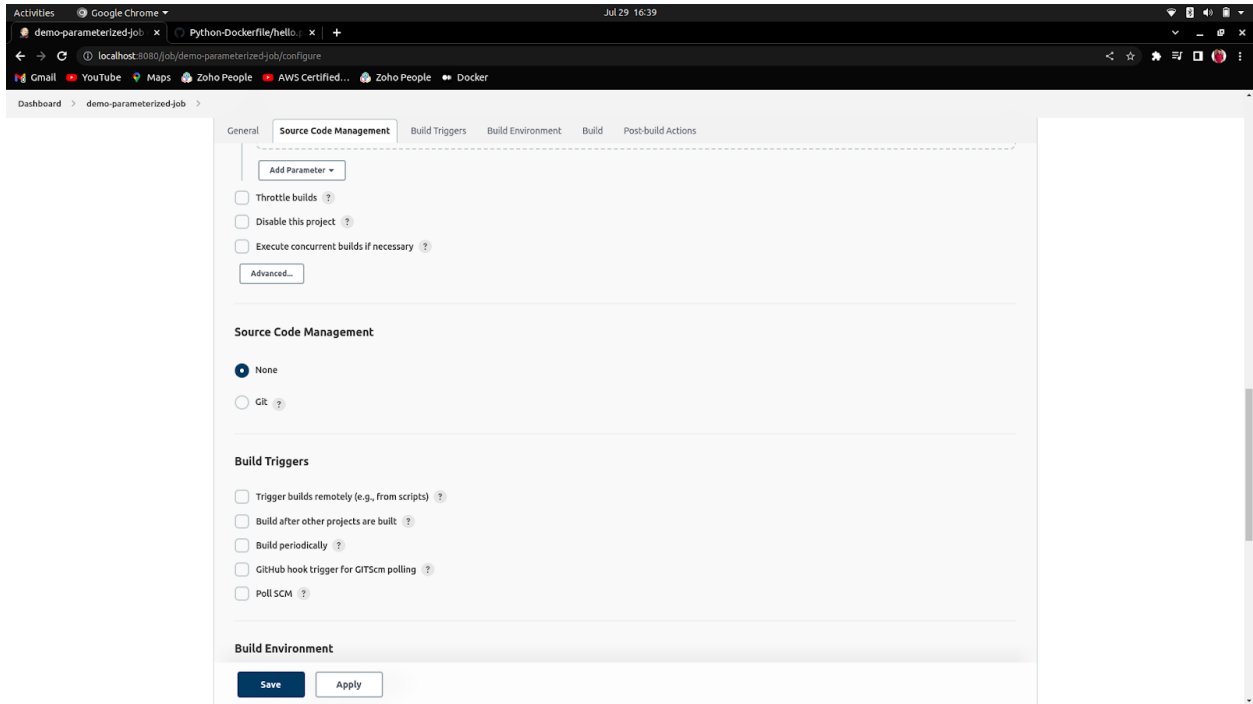
- String Parameter:**
 - Name: NAME
 - Default Value: DEFAULT
 - Description: (empty)
 - Trim the string: (unchecked)
- Choice Parameter:**
 - Name: CHOICES
 - Choices: Cricket, Football, Baseball, Volleyball

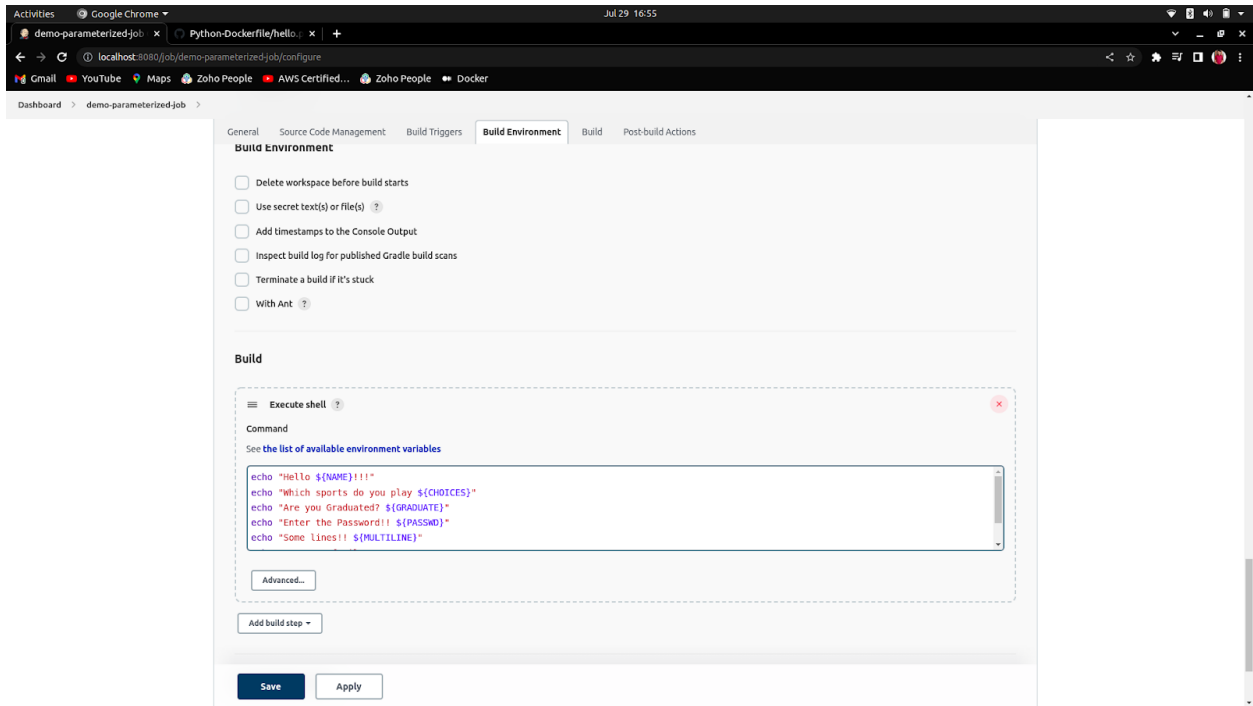
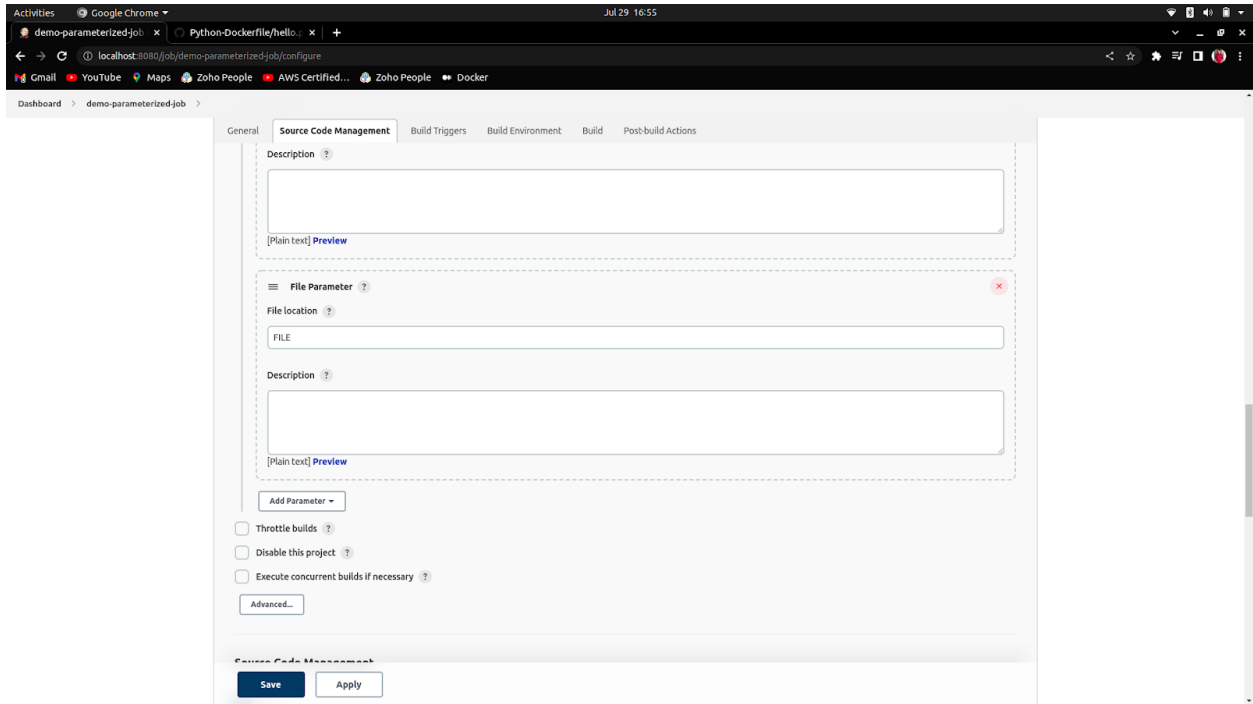
At the bottom of the configuration, there are "Save" and "Apply" buttons.

The screenshot shows the Jenkins configuration page for the same job, "demo-parameterized-job". The "General" tab is still selected. The "Choice Parameter" is now at the top, and a new "Boolean Parameter" has been added below it:

- Choice Parameter:**
 - Name: CHOICES
 - Choices: Cricket, Football, Baseball, Volleyball
 - Description: (empty)
- Boolean Parameter:**
 - Name: GRADUATED
 - Set by Default: (unchecked)
 - Description: (empty)

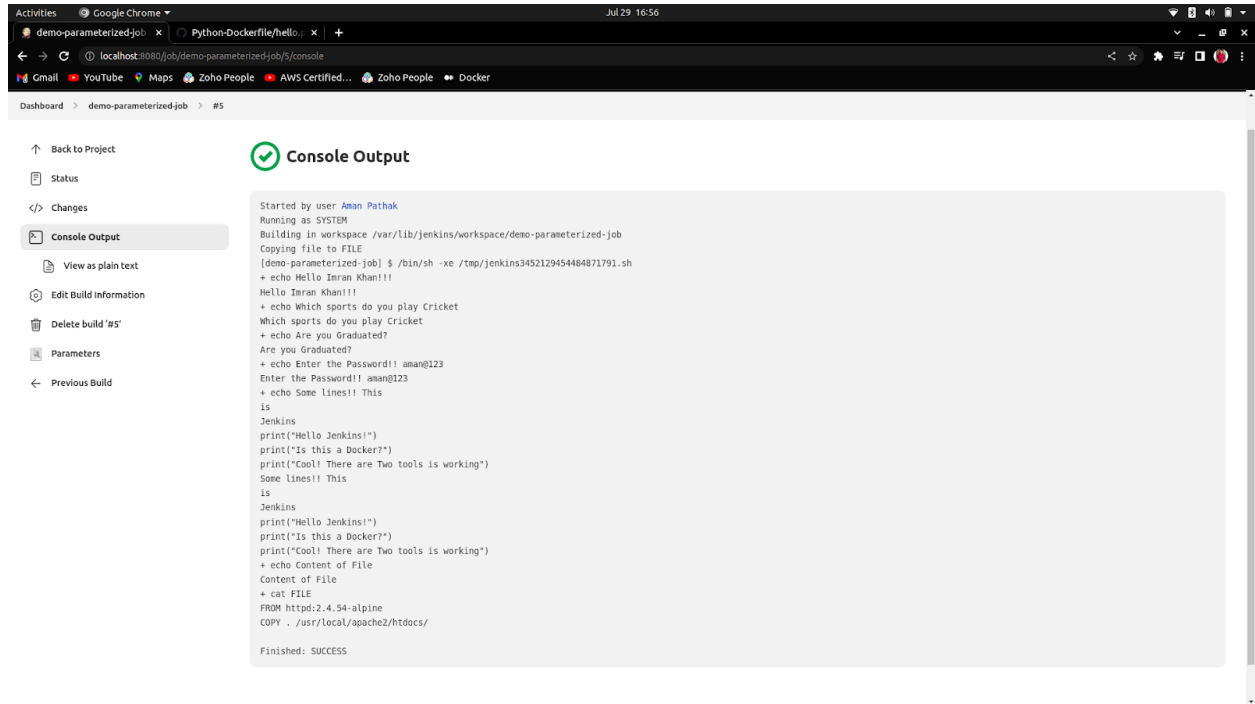
At the bottom of the configuration, there are "Save" and "Apply" buttons.





```
echo "Are you Graduated? ${GRADUATE}"
echo "Enter the Password!! ${PASSWD}"
echo "Some lines!! ${MULTILINE}"
echo "Content of File"
cat FILE
```

Console Output



The screenshot shows a web browser window displaying the Jenkins console output for a build named 'demo-parameterized-job'. The browser's address bar shows the URL 'localhost:8080/job/demo-parameterized-job/5/console'. The Jenkins interface includes a left sidebar with navigation links: 'Back to Project', 'Status', 'Changes', 'Console Output' (selected), 'View as plain text', 'Edit Build Information', 'Delete build #5', 'Parameters', and 'Previous Build'. The main content area, titled 'Console Output' with a green checkmark icon, displays the following text:

```
Started by user Aman Pothak
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/demo-parameterized-job
Copying file to FILE
[demo-parameterized-job] $ /bin/sh -xe /tmp/jenkins3452129454484871791.sh
+ echo Hello Imran Khan!!!
Hello Imran Khan!!!
+ echo Which sports do you play Cricket
Which sports do you play Cricket
+ echo Are you Graduated?
Are you Graduated?
+ echo Enter the Password!! aman@123
Enter the Password!! aman@123
+ echo Some lines!! This
is
Jenkins
print("Hello Jenkins!")
print("Is this a Docker?")
print("Cool! There are Two tools is working")
Some lines!! This
is
Jenkins
print("Hello Jenkins!")
print("Is this a Docker?")
print("Cool! There are Two tools is working")
+ echo Content of File
Content of File
+ cat FILE
FROM httpd:2.4.54-alpine
COPY . /usr/local/apache2/htdocs/

Finished: SUCCESS
```

Custom Workspace

1. Create a new job and do the configuration as given below:

Dashboard > demo-custom-workspace >

General Source Code Management Build Triggers Build Environment Build Post-build Actions

Description

[Plain text] [Preview](#)

☐ Discard old builds ?

☐ GitHub project

☐ This project is parameterised ?

☐ Throttle builds ?

☐ Disable this project ?

☐ Execute concurrent builds if necessary ?

☐ Quiet period ?

☐ Retry Count ?

☐ Block build when upstream project is building ?

☐ Block build when downstream project is building ?

☒ Use custom workspace ?

Directory

/var/lib/jenkins/myws/demospace

Build

≡ **Execute shell** ?

Command

See [the list of available environment variables](#)

pwd
touch init.txt

Advanced...

Add build step ▾

Post-build Actions

Add post-build action ▾

Save Apply

2. Now, the directory has been created and file too.
To check it, go to your terminal.

```
amanpathak@aman-pathak:/var/lib/jenkins$ cd myws/  
amanpathak@aman-pathak:/var/lib/jenkins/myws$ ls  
demospace  
amanpathak@aman-pathak:/var/lib/jenkins/myws$ cd demospace/  
amanpathak@aman-pathak:/var/lib/jenkins/myws/demospace$ ls  
init.txt
```

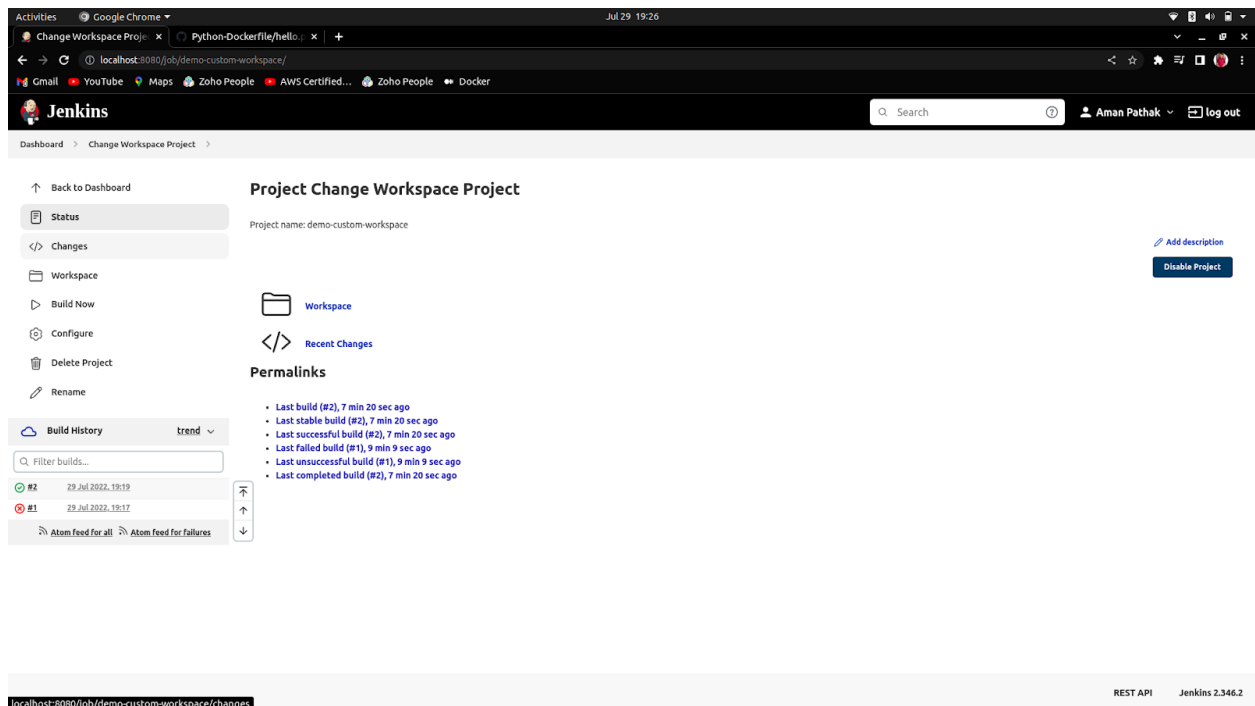
Change display name and project name

1. Go to in the Configure of the Specific Project-> Enter the desired name that you want in the text field.



A screenshot of the Jenkins configuration page for a project. The 'Display Name' field is highlighted with a blue border and contains the text 'Change Workspace Project'. Above the field is a small question mark icon. The background is a light gray.

The Output



A screenshot of the Jenkins web interface. The browser address bar shows 'localhost:8080/job/demo-custom-workspace/'. The Jenkins header shows the user 'Aman Pathak' and a 'log out' button. The main content area is titled 'Project Change Workspace Project' and shows the project name 'demo-custom-workspace'. On the left, there is a sidebar with navigation links: 'Back to Dashboard', 'Status', 'Changes', 'Workspace', 'Build Now', 'Configure', 'Delete Project', and 'Rename'. The 'Build History' section shows a table with two builds: #2 (successful, 29 Jul 2022, 19:19) and #1 (failed, 29 Jul 2022, 19:17). The 'Permalinks' section lists various build links. At the bottom, there is a footer with the URL 'localhost:8080/job/demo-custom-workspace/changes' and the text 'REST API Jenkins 2.346.2'.

But the Project hasn't changed. For that you have to click on the Rename button which is left on the screen and enter the desired name.

Now, The project name has changed too.

The screenshot shows the Jenkins web interface in a Google Chrome browser. The address bar indicates the URL is `localhost:8080/job/demo-custom-my-workspace/`. The Jenkins logo and navigation menu are visible at the top. The main content area is titled "Project Change Workspace Project" and shows the project name "demo-custom-my-workspace". Below this, there are links for "Workspace" and "Recent Changes". A "Permalinks" section lists several build links. On the left sidebar, the "Build History" tab is selected, showing a table with two build entries: #2 (successful) and #1 (failed). The footer of the page displays "REST API" and "Jenkins 2.346.2".

Dashboard > Change Workspace Project >

Back to Dashboard

Status

Changes

Workspace

Build Now

Configure

Delete Project

Rename

Build History trend

Filter builds...

#2 29-Jul-2022, 19:19

#1 29-Jul-2022, 19:17

Atom feed for all Atom feed for failures

Project Change Workspace Project

Project name: demo-custom-my-workspace

[Add description](#)

[Disable Project](#)

[Workspace](#)

[Recent Changes](#)

Permalinks

- [Last build \(#2\), 8 min 36 sec ago](#)
- [Last stable build \(#2\), 8 min 36 sec ago](#)
- [Last successful build \(#2\), 8 min 36 sec ago](#)
- [Last failed build \(#1\), 10 min ago](#)
- [Last unsuccessful build \(#1\), 10 min ago](#)
- [Last completed build \(#2\), 8 min 36 sec ago](#)

REST API Jenkins 2.346.2

Building Upstream and Downstream Project

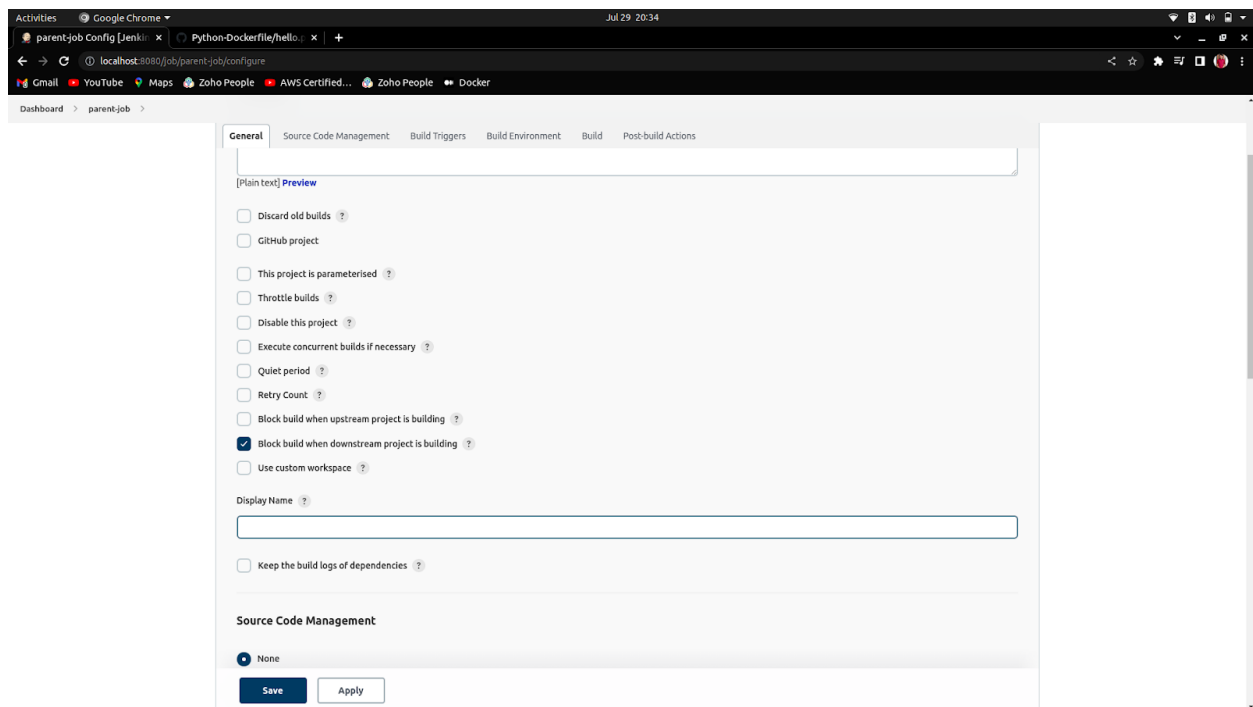
Upstream- Parent job

Downstream- Child job

That job that is triggering another job is known as the Parent job and that job that is triggered by another job/parent job is known as the Child job.

Block build when the upstream project is building

Here, the scenario is if you are building a parent job then, you couldn't build a child job at the same time. To build the child job, you have to complete the build of the parent job first.



Activities Google Chrome Jul 29 20:40

parent-job Config [Jenkins] Python-Dockerfile/hello

localhost:8080/job/parent-job/configure

Gmail YouTube Maps Zoho People AWS Certified... Zoho People Docker

Dashboard > parent-job >

General Source Code Management **Build Triggers** Build Environment Build Post-build Actions

Build Triggers

☐ Trigger builds remotely (e.g., from scripts) ?

☒ Build after other projects are built ?

Projects to watch

child-job,

☒ Trigger only if build is stable

☐ Trigger even if the build is unstable

☐ Trigger even if the build fails

☐ Always trigger, even if the build is aborted

☐ Build periodically ?

☐ GitHub hook trigger for GITSCM polling ?

☐ Poll SCM ?

Build Environment

☐ Delete workspace before build starts

☐ Use secret text(s) or file(s) ?

☐ Add timestamps to the Console Output

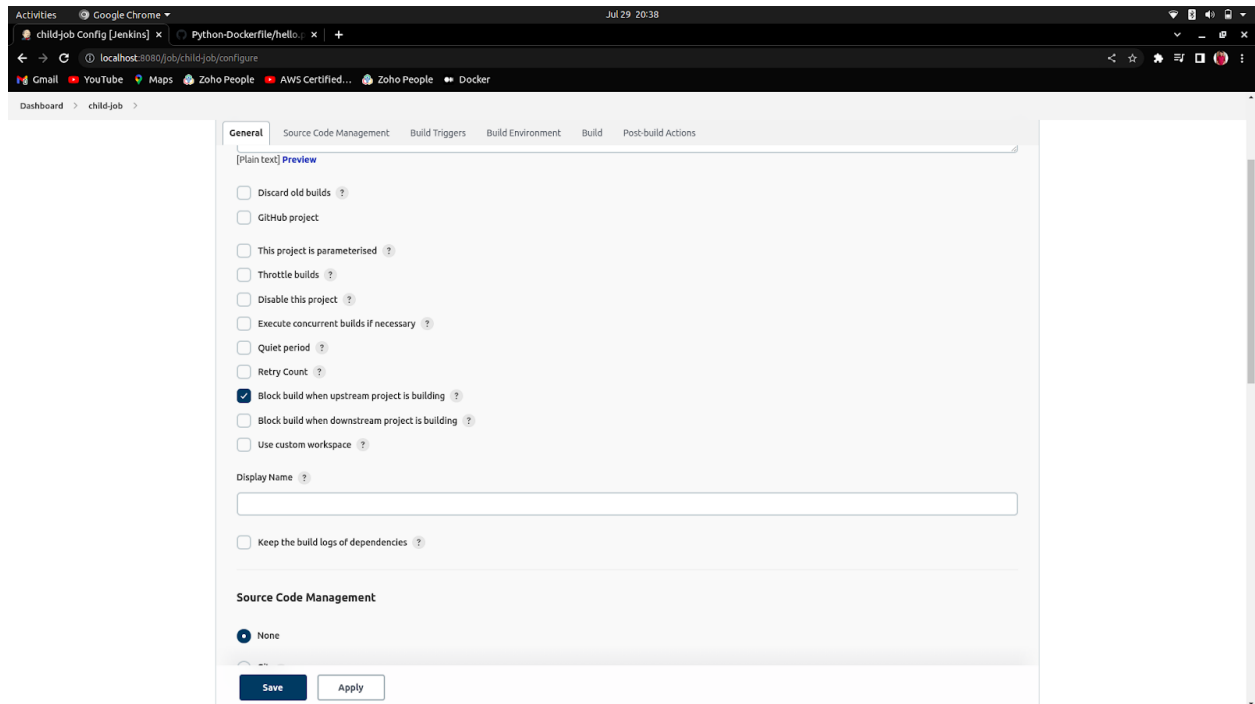
☐ Inspect build logs for published Gradle build scans

Save **Apply**

Block build when downstream project is building

Here, the scenario is if you are building a child job then, you couldn't build a parent job at the same time. To build the parent job, you have to complete the build of the child job first.

Complete Reverse of the previous one.



Activities Google Chrome Jul 29 20:36

childjob Config [Jenkins] x Python-Dockerfile/hello... x +

localhost:8080/job/child-job/configure

Gmail YouTube Maps Zoho People AWS Certified... Zoho People Docker

Dashboard > child-job >

General Source Code Management Build Triggers Build Environment Build Post-build Actions

Build Triggers

☐ Trigger builds remotely (e.g., from scripts) ?

☒ Build after other projects are built ?

Projects to watch

parent:job,

☒ Trigger only if build is stable

☐ Trigger even if the build is unstable

☐ Trigger even if the build fails

☐ Always trigger, even if the build is aborted

☐ Build periodically ?

☐ GitHub hook trigger for GITSCM polling ?

☐ Poll SCM ?

Build Environment

☐ Delete workspace before build starts

☐ Use secret text(s) or file(s) ?

☐ Add timestamps to the Console Output

Save Apply

Activities

Google Chrome

Jul 29 21:00

Available Plugins - PluginPython-Dockerfile/hello...localhost:8080/pluginManager/availableGmailYouTubeMapsZoho PeopleAWS Certified...Zoho PeopleDocker

Jenkins

Search

Aman Pathak

log out

DashboardPlugin Manager

Back to Dashboard

Manage Jenkins

Plugin Manager

UpdatesAvailableInstalledAdvanced

build pipe

Install	Name	Released
☒	Build Pipeline1.5.8 User InterfaceBuild ToolsOther Post-Build Actions This plugin renders upstream and downstream connected jobs that typically form a build pipeline. In addition, it offers the ability to define manual triggers for jobs that require intervention prior to execution, e.g. an approval process outside of Jenkins. Warning: This plugin version may not be safe to use. Please review the following security notices: Stored XSS vulnerability	4 yr 7 mo ago
☐	Webhook Step173.vfa_b_93560b_977 Allows build pipelines to wait for notification from an external system before continuing.	2 mo 12 days ago
☐	Pipeline timeline1.0.3 An interactive build timeline to help you visualize your build pipeline and identify bottlenecks.	3 yr 5 mo ago

Install without restart

Download now and install after restart

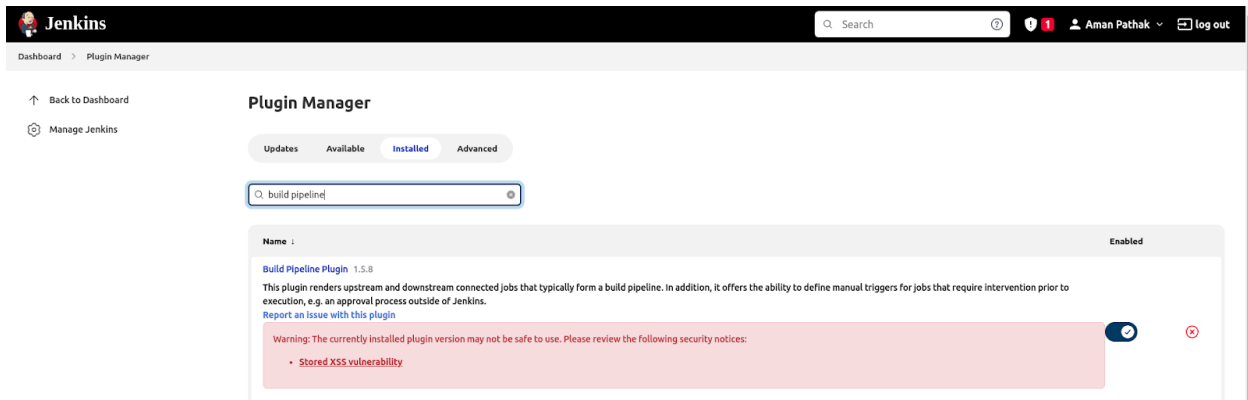
Update information obtained: 1 day 6 hr ago

Check now

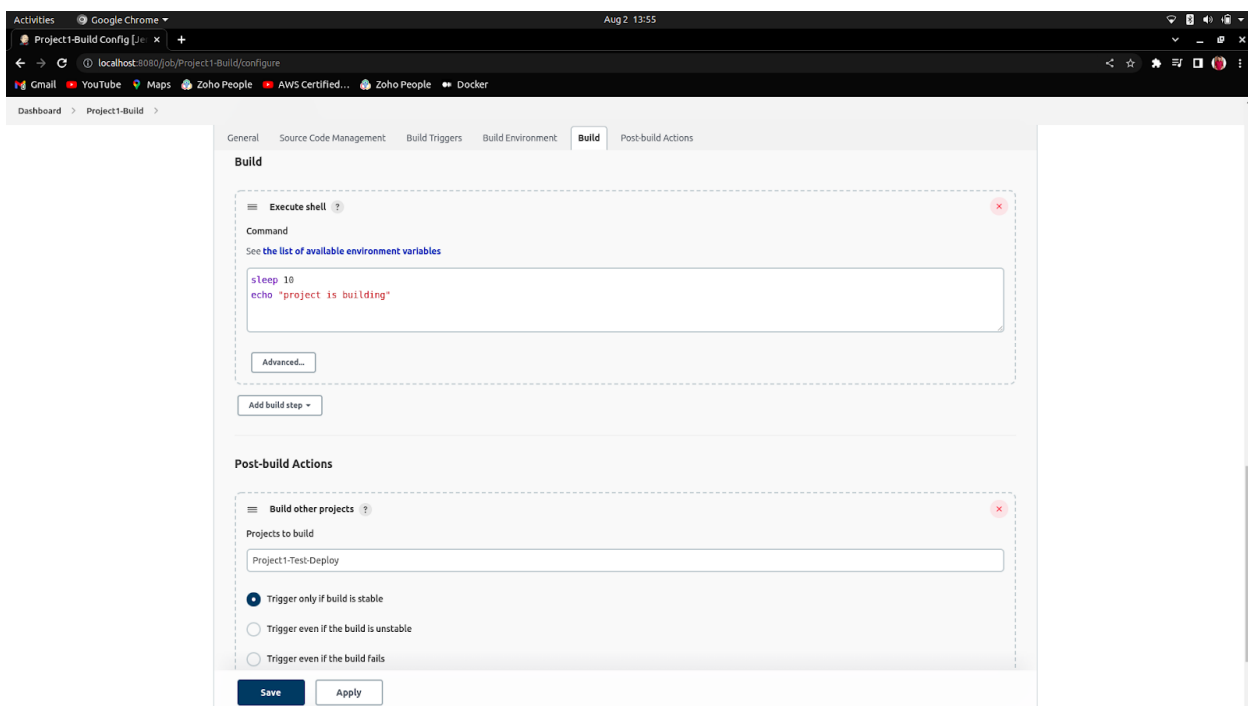
REST APIJenkins 2.346.2

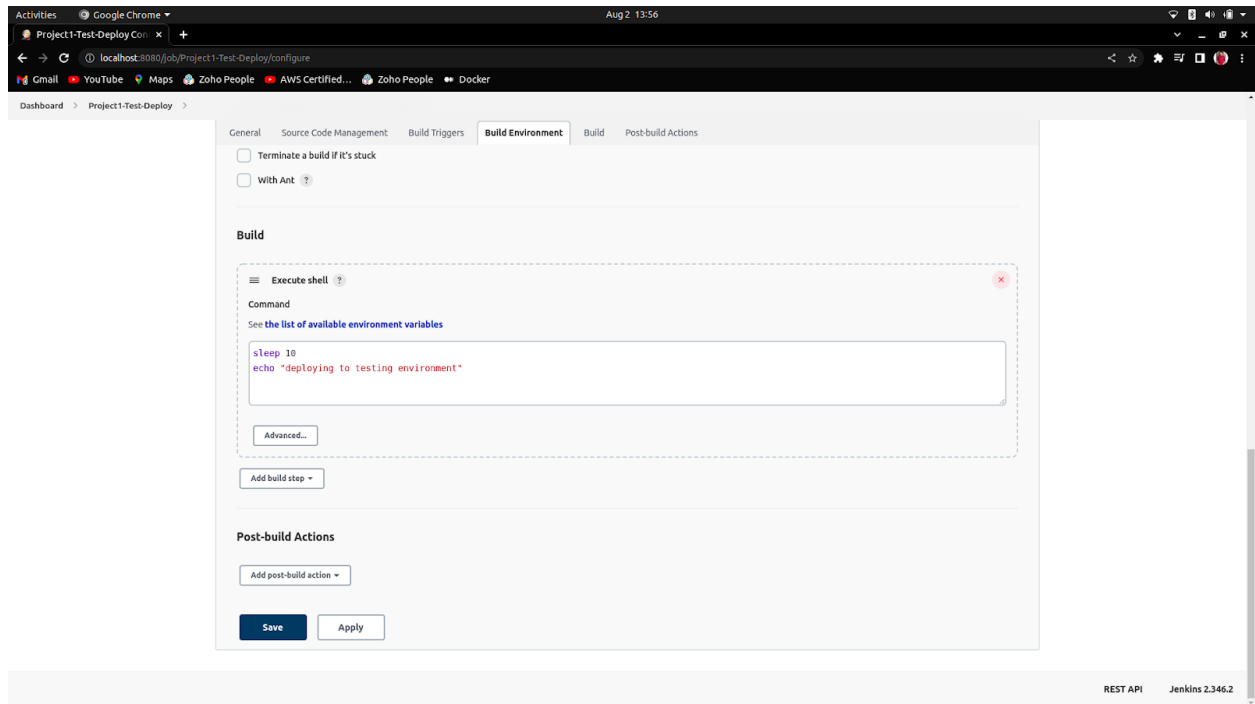
Jenkins Pipeline using Build Pipeline

1. First of all, Install the plugin named Build Pipeline.



2. Create two Jobs, first will be Parent and the second will be Child job and assign the Post-build actions in Parent Job for Child job as per below:





3. Click on “+” to create the Build Pipeline.

[Add description](#)

All		+				
S	W	Name	Last Success	Last Failure	Last Duration	
✓	🔗	Change Workspace Project	3 days 18 hr #2	3 days 18 hr #1	26 ms	▶
✓	⚙️	child-job	3 days 17 hr #13	N/A	23 ms	▶
✓	⚙️	demo-build-periodically	3 days 21 hr #49	N/A	25 ms	▶
✓	⚙️	demo-first	3 days 21 hr #3	N/A	33 ms	▶

4. Choose Type as “Build Pipeline view” and assign the name of your Pipeline.

New view

Name

Type

☒ Build Pipeline View

Shows the jobs in a build pipeline view. The complete pipeline of jobs that a version propagates through are shown as a row in the view.

☐ List View

Shows items in a simple list format. You can choose which jobs are to be displayed in which view.

☐ My View

This view automatically displays all the jobs that the current user has an access to.

Create

5. Now, all you have to do is “Select the Initial Job” which will be your Parent's job.

Activities Google Chrome Aug 2, 13:59

Edit View [Jenkins] localhost:8080/view/Pipeline-Jenkins/configure

Dashboard > Pipeline-Jenkins

Build Queue
No builds in the queue.

Build Executor Status
1 Idle
2 Idle

Enter pipeline executors

Build Pipeline View Title

Pipeline Flow

Layout
Based on upstream/downstream relationship

This layout mode derives the pipeline structure based on the upstream/downstream trigger relationship between jobs. This is the only out-of-the-box supported layout mode, but is open for extension.

Upstream / downstream config

Select Initial Job ?
Project1-Build

Trigger Options

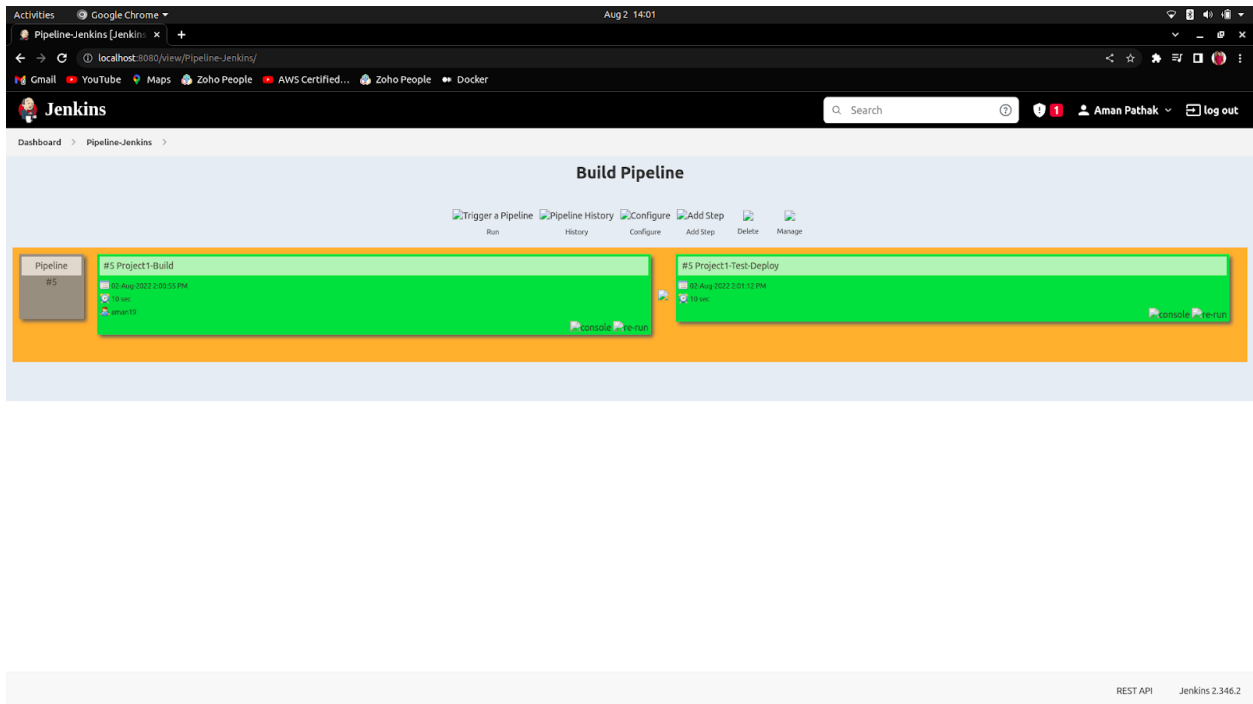
Build Cards
Standard build card

Use the default build cards

Restrict triggers to most recent successful builds ?
☒ Yes

OK Apply

6. In the last, Click on the run button and the result will be in front of you like this:

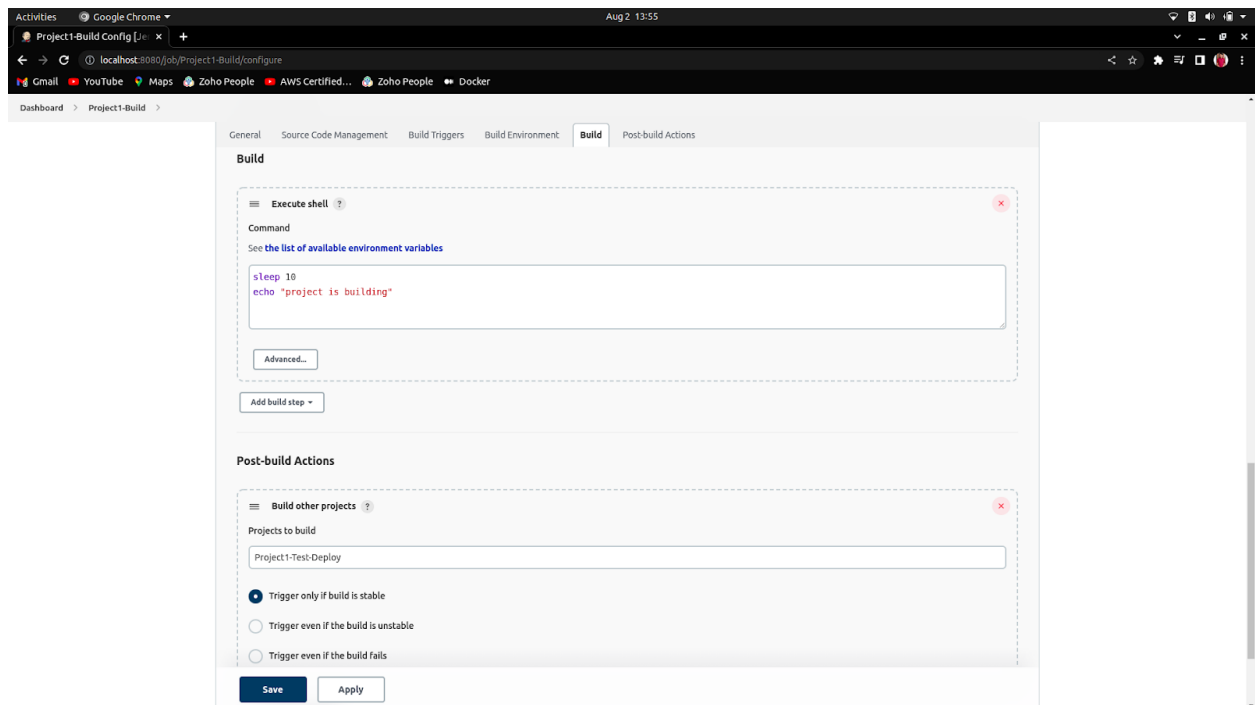


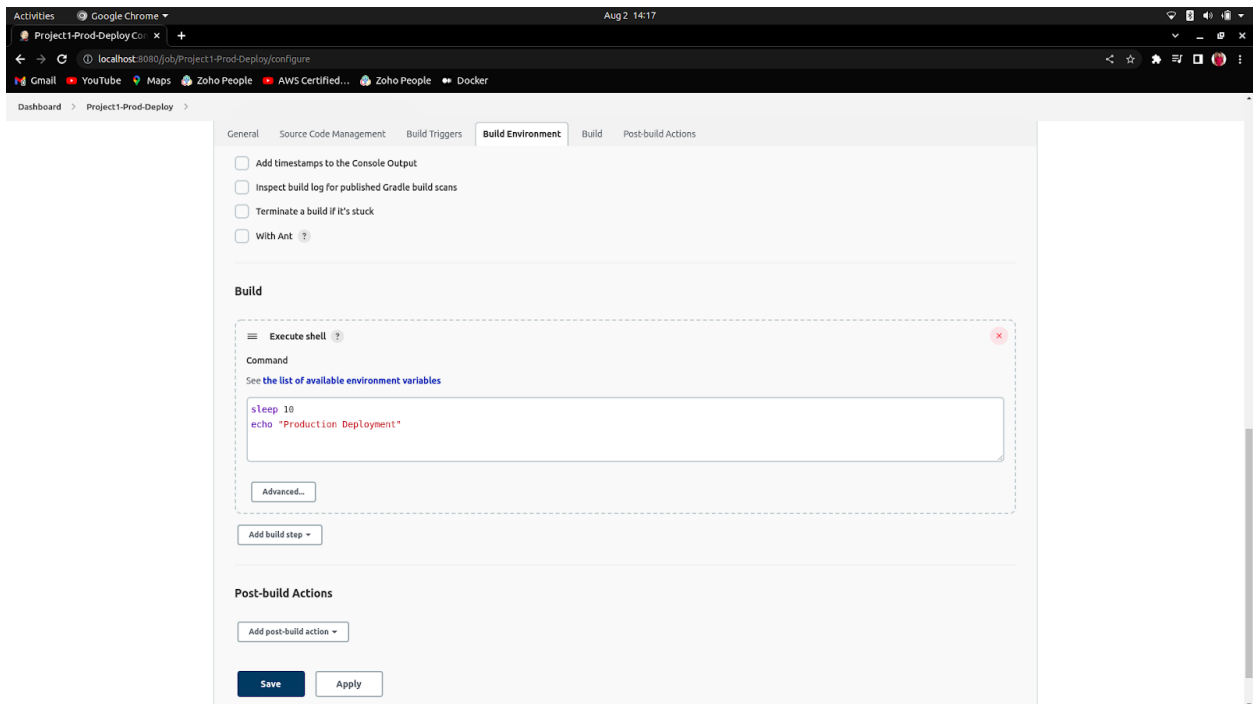
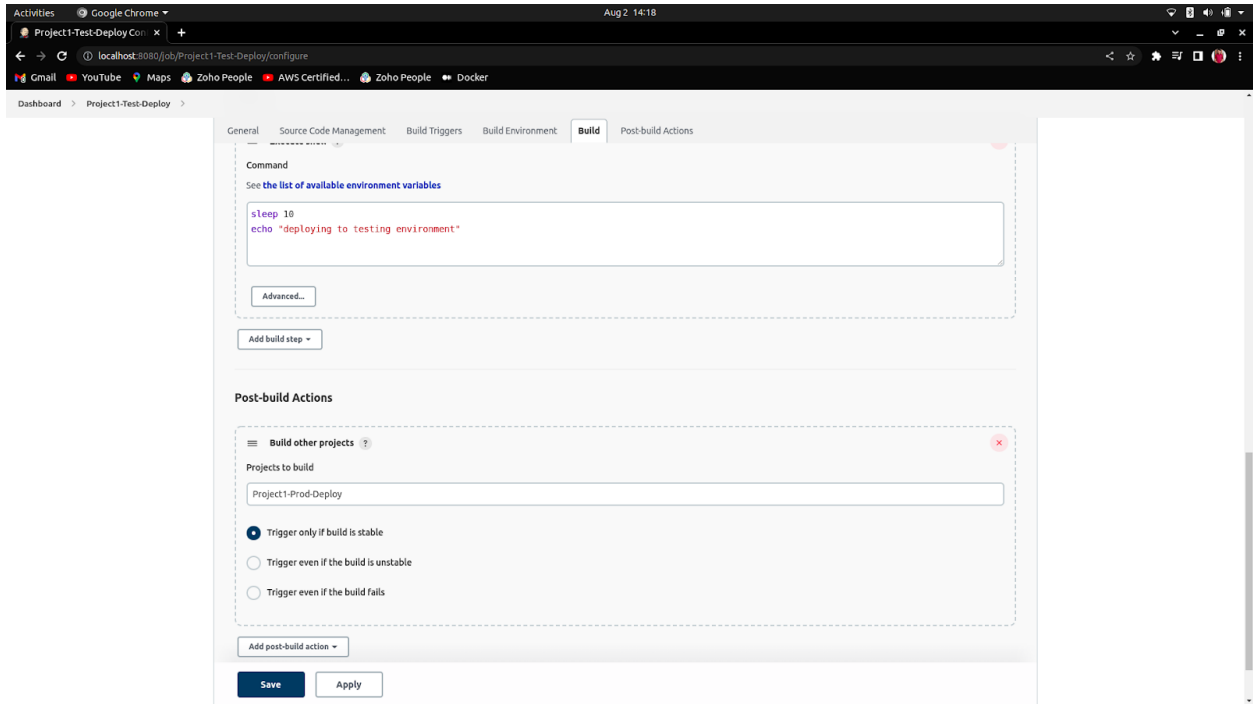
Continuous Deployment vs Continuous Delivery

In Continuous Deployment, whenever the code changes to an application is released automatically into the production environment. There is no inclusion of human intervention or manual click.

E.g

1. Create three Jobs, first will be GrandParent, the second will be Parent job and the third will be Child job and assign the Post-build actions in Grand Parent Job for Parent job and in Parent Job for Child Job as per below:





2. Click on “+” to create the Build Pipeline.

[Add description](#)

S	W	Name	Last Success	Last Failure	Last Duration
		Change Workspace Project	3 days 18 hr #2	3 days 18 hr #1	26 ms
		child-job	3 days 17 hr #13	N/A	23 ms
		demo-build-periodically	3 days 21 hr #49	N/A	25 ms
		demo-First	3 days 21 hr #3	N/A	33 ms

3. Choose “Build Pipeline view” and assign the name of your Pipeline.

New view

Name

Type

☒ Build Pipeline View

Shows the jobs in a build pipeline view. The complete pipeline of jobs that a version propagates through are shown as a row in the view.

☐ List View

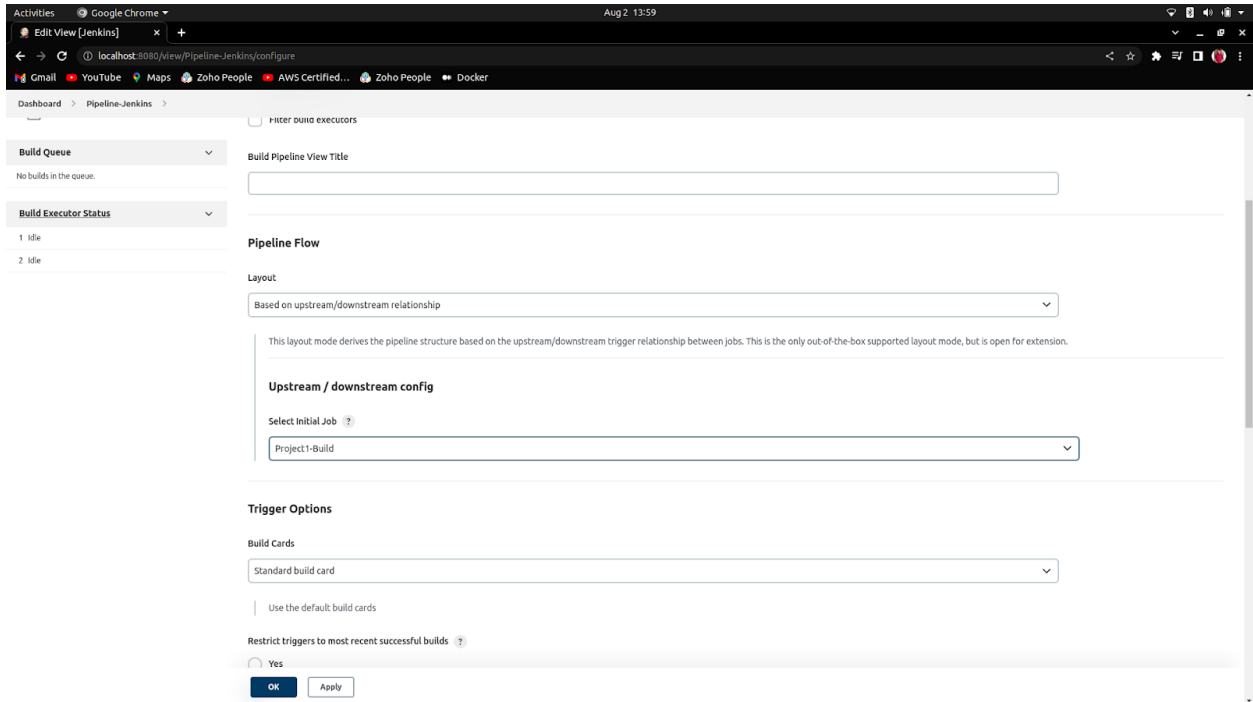
Shows items in a simple list format. You can choose which jobs are to be displayed in which view.

☐ My View

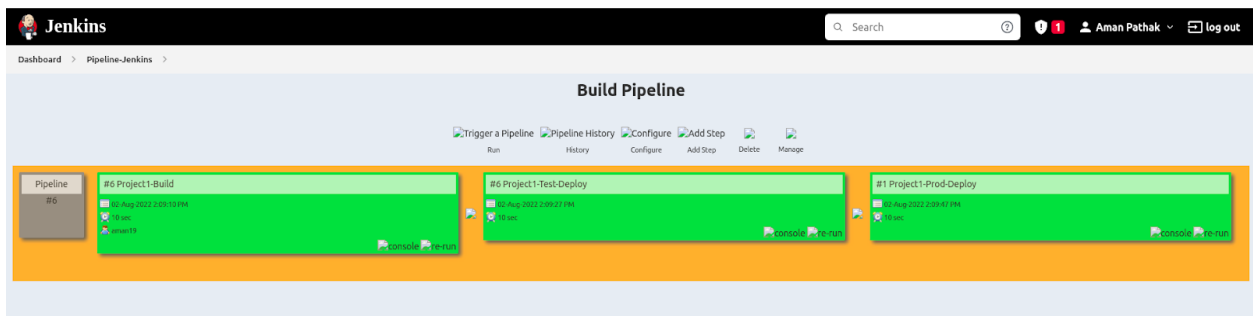
This view automatically displays all the jobs that the current user has an access to.

Create

4. Now, all you have to do is “Select the Initial Job” which will be your Parent's job.



5. The Final Output:

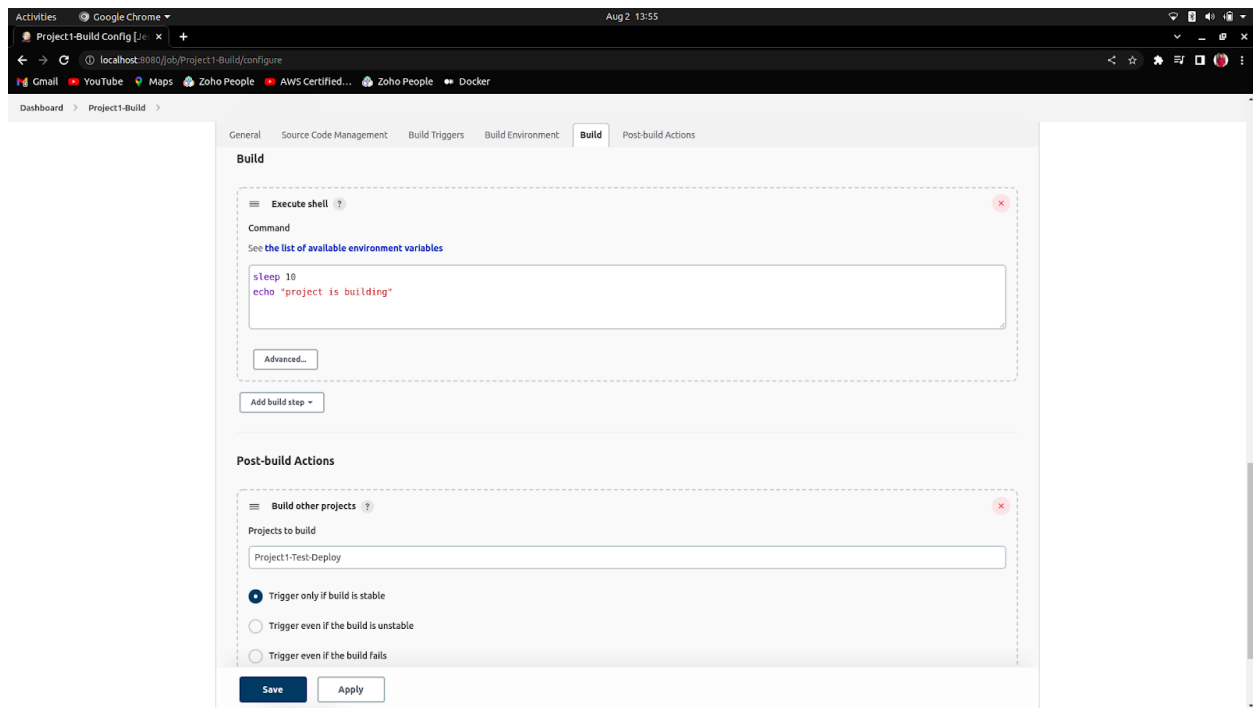


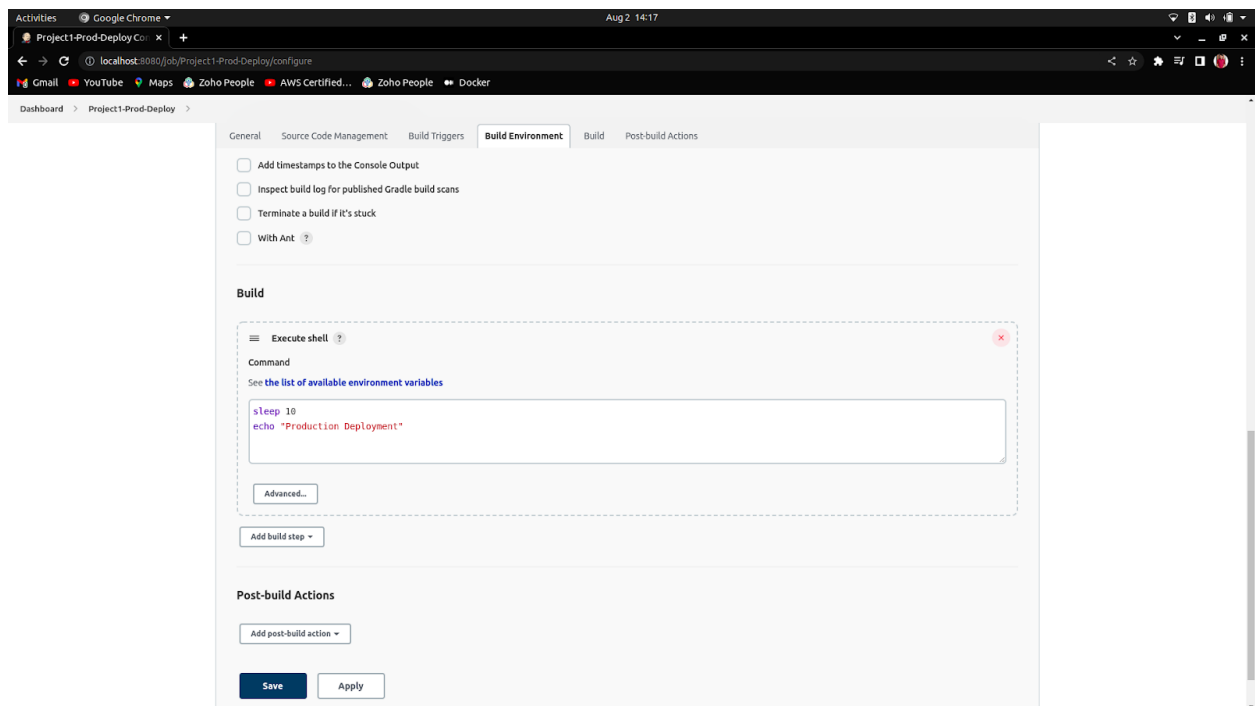
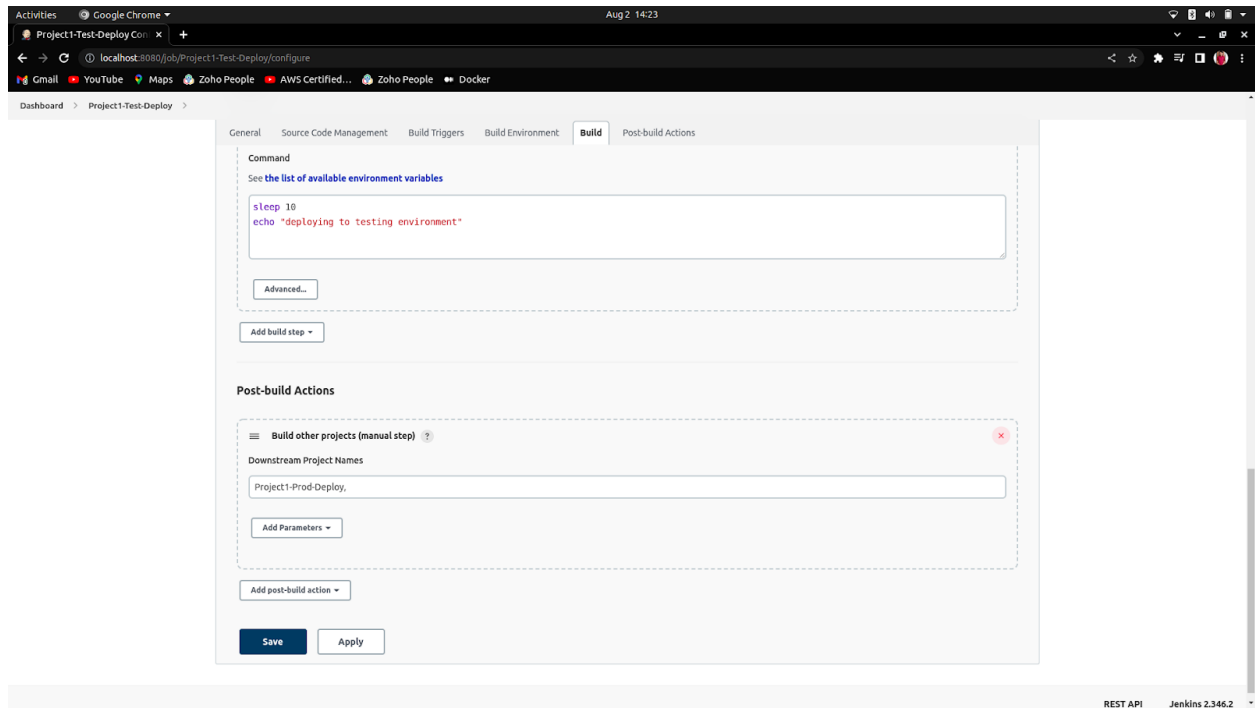
Continuous Delivery

In Continuous Delivery, whenever the code changes to an application are released only by human intervention or through manual clicks into the production environment. There is no any Automation in the Continuous Delivery.

E.g,

1. Create three Job, first will be GrandParent, the second will be Parent job and third will be Child job and assign the Post-build actions in Grand Parent Job for Parent job and rest the last will be **Manually** as per below:





2. Click on “+” to create the Build Pipeline.

[Add description](#)

S	W	Name	Last Success	Last Failure	Last Duration
		Change Workspace Project	3 days 18 hr #2	3 days 18 hr #1	26 ms
		child-job	3 days 17 hr #13	N/A	23 ms
		demo-build-periodically	3 days 21 hr #49	N/A	25 ms
		demo-First	3 days 21 hr #3	N/A	33 ms

3. Choose Type “Build Pipeline view” and assign the name of your Pipeline.

New view

Name

Type

☒ Build Pipeline View

Shows the jobs in a build pipeline view. The complete pipeline of jobs that a version propagates through are shown as a row in the view.

☐ List View

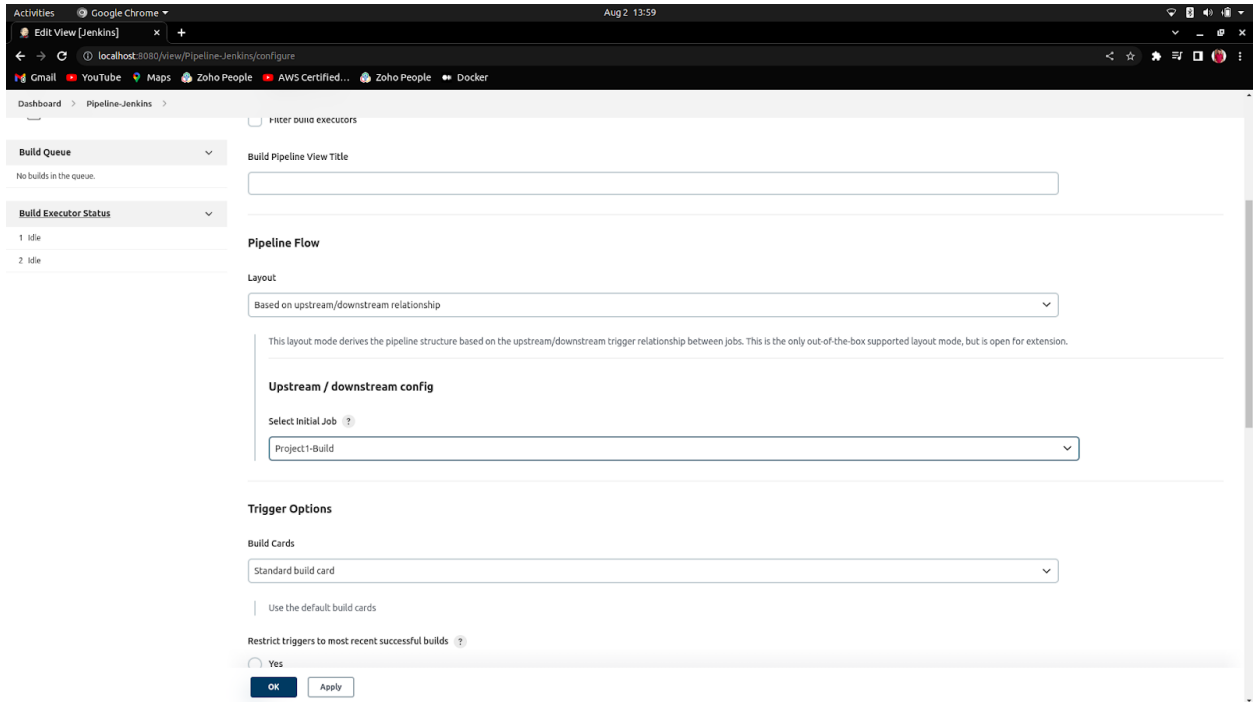
Shows items in a simple list format. You can choose which jobs are to be displayed in which view.

☐ My View

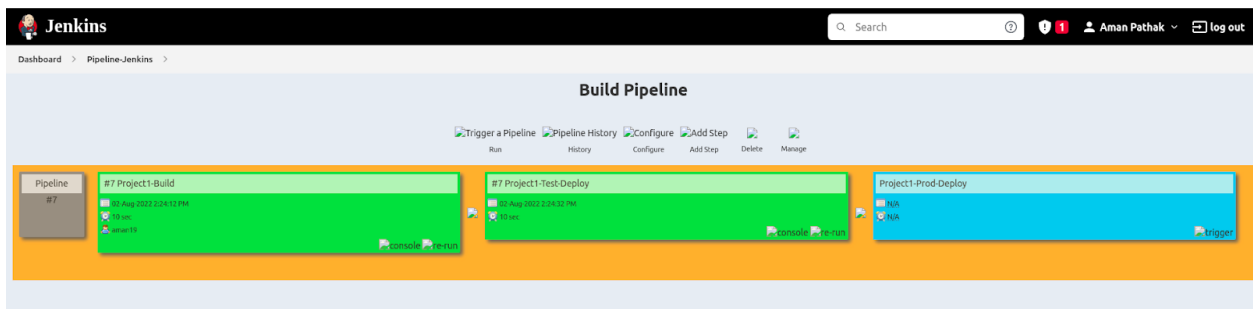
This view automatically displays all the jobs that the current user has an access to.

Create

4. Now, all you have to do is “Select the Initial Job” and which will be your Parent job.

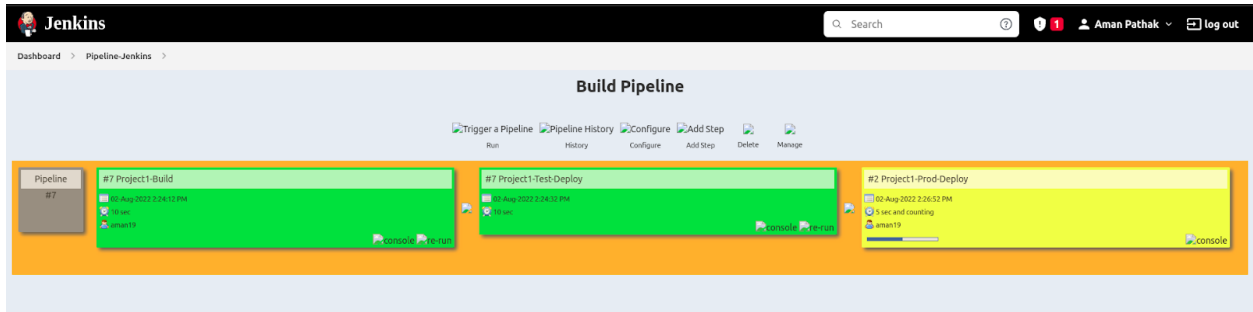


5. Now, when you run the Pipeline then you will face like this.

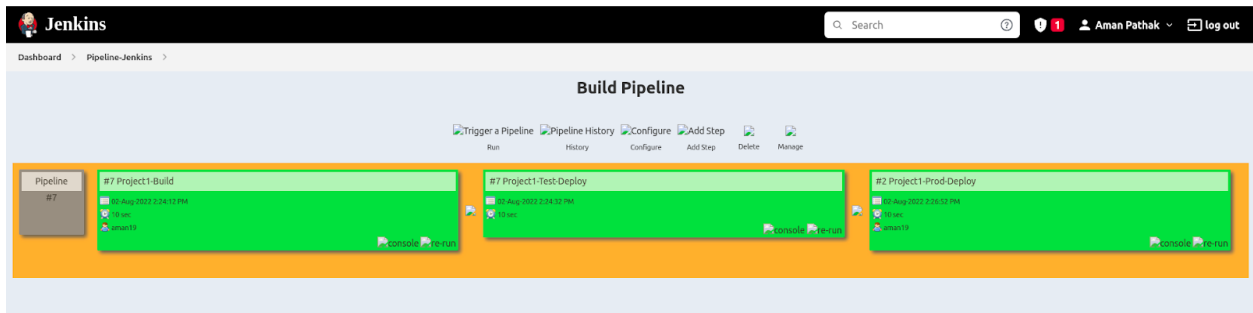


The last job will not run automatically as it is built with manual intervention.

6. So, to run that build too. You have to click on the trigger or run that build(on right bottom).

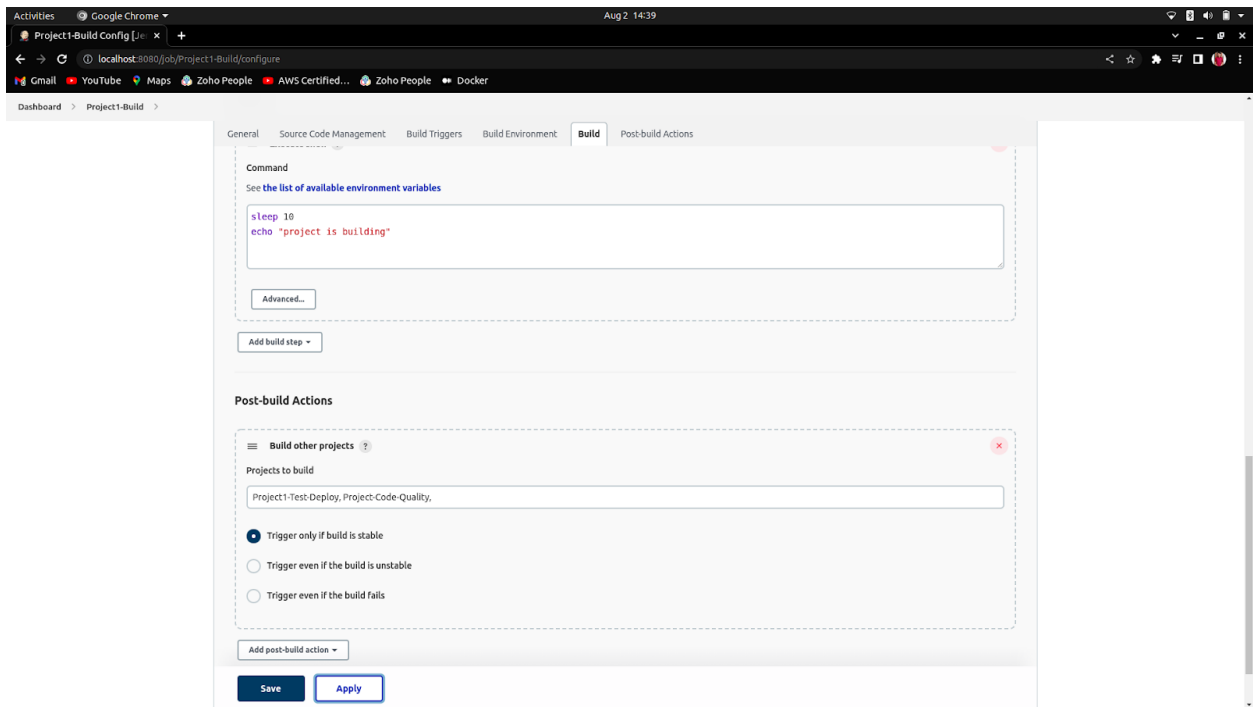


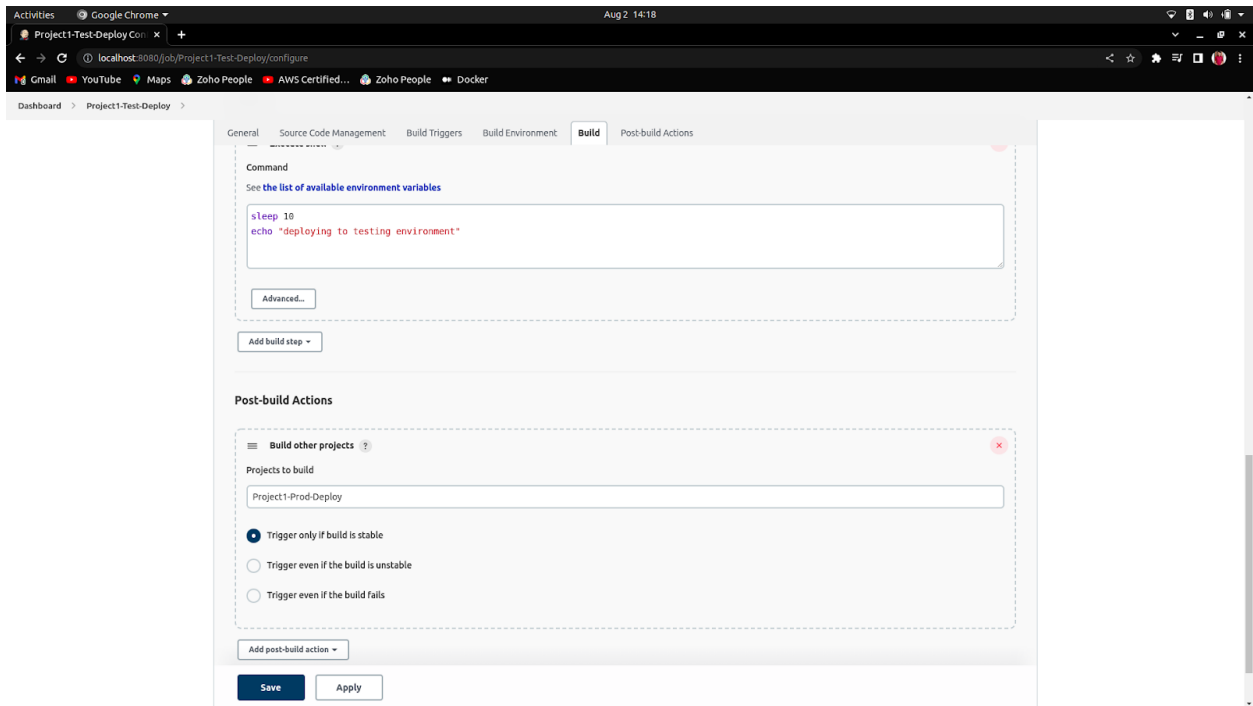
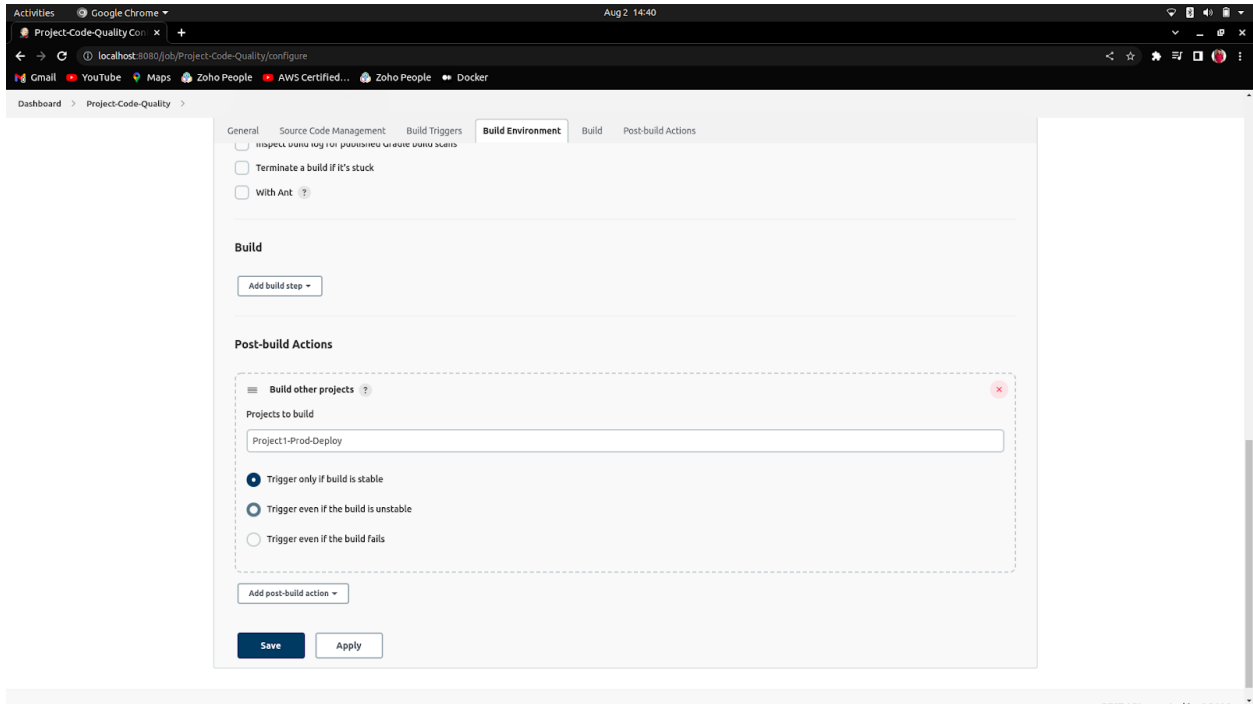
7. The Final Output:

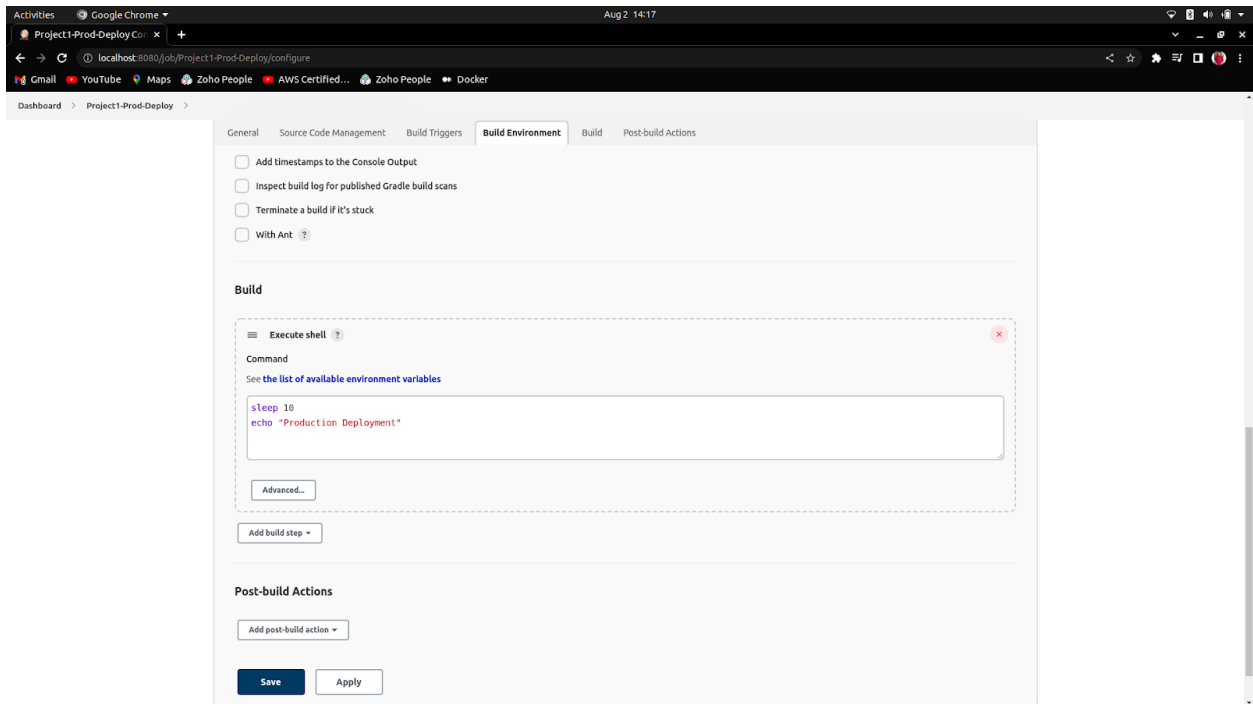


Run Two Job Parallel in Jenkins Pipeline

1. Create three Job, first will be GrandParent, the second will be Parent job and third will be Child job and assign the Post-build actions in Grand Parent Job for Parent job and in Parent Job for Child Job as per below:







6. Click on “+” to create the Build Pipeline.

[Add description](#)

All					
S	W	Name	Last Success	Last Failure	Last Duration
✓	🔗	Change Workspace Project	3 days 18 hr #2	3 days 18 hr #1	26 ms
✓	⚙️	child-job	3 days 17 hr #13	N/A	23 ms
✓	⚙️	demo-build-periodically	3 days 21 hr #49	N/A	25 ms
✓	⚙️	demo-first	3 days 21 hr #3	N/A	33 ms

7. Choose Type “Build Pipeline view” and assign the name of your Pipeline.

New view

Name

Type

☒ Build Pipeline View

Shows the jobs in a build pipeline view. The complete pipeline of jobs that a version propagates through are shown as a row in the view.

☐ List View

Shows items in a simple list format. You can choose which jobs are to be displayed in which view.

☐ My View

This view automatically displays all the jobs that the current user has an access to.

Create

8. Now, all you have to do is “Select the Initial Job” and which will be your Parent job.

Activities Google Chrome Aug 2, 13:59

Edit View [Jenkins]

localhost:8080/view/Pipeline-Jenkins/configure

Gmail YouTube Maps Zoho People AWS Certified... Zoho People Docker

Dashboard > Pipeline-Jenkins

Enter build executors

Build Queue

No builds in the queue.

Build Executor Status

1 Idle

2 Idle

Build Pipeline View Title

Pipeline Flow

Layout

Based on upstream/downstream relationship

This layout mode derives the pipeline structure based on the upstream/downstream trigger relationship between jobs. This is the only out-of-the-box supported layout mode, but is open for extension.

Upstream / downstream config

Select Initial Job ?

Project1-Build

Trigger Options

Build Cards

Standard build card

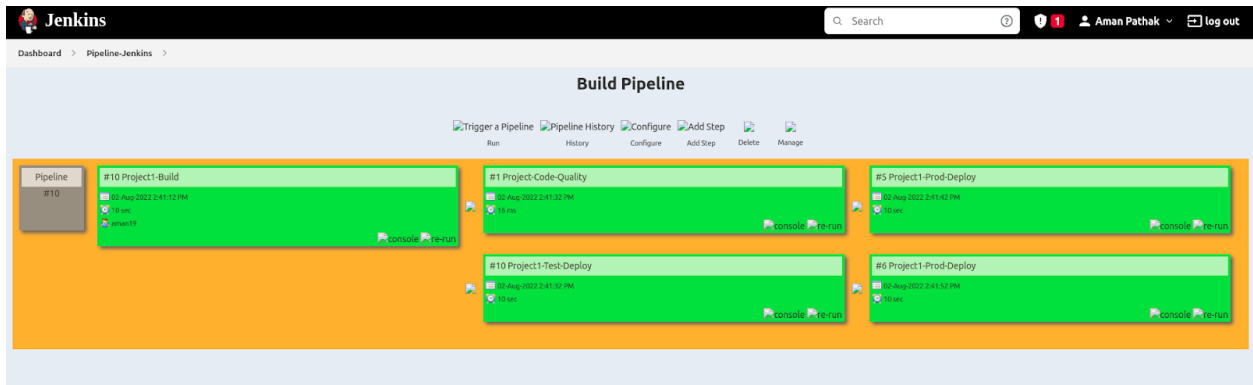
Use the default build cards

Restrict triggers to most recent successful builds ?

Yes

OK Apply

9. The Final Output:



The screenshot displays the Jenkins web interface. At the top, the Jenkins logo is on the left, and a search bar, user profile (Aman Pathak), and log out button are on the right. Below the header, the breadcrumb navigation shows 'Dashboard > Pipeline-Jenkins >'. The main section is titled 'Build Pipeline' and contains a toolbar with icons for 'Trigger a Pipeline', 'Pipeline History', 'Configure', 'Add Step', 'Delete', and 'Manage'. The pipeline is visualized as a sequence of five steps, each in a green box with a status icon (a checkmark) and a timestamp. The steps are: #10 Project1-Build (10 Aug 2022 2:41:12 PM), #1 Project-Code-Quality (10 Aug 2022 2:41:12 PM), #5 Project1-Prod-Deploy (10 Aug 2022 2:41:12 PM), #10 Project1-Test-Deploy (10 Aug 2022 2:41:12 PM), and #6 Project1-Prod-Deploy (10 Aug 2022 2:41:12 PM). Each step box has links for 'console' and 're-run' at the bottom right.

Step	Name	Status	Timestamp
#10	Project1-Build	Success	10 Aug 2022 2:41:12 PM
#1	Project-Code-Quality	Success	10 Aug 2022 2:41:12 PM
#5	Project1-Prod-Deploy	Success	10 Aug 2022 2:41:12 PM
#10	Project1-Test-Deploy	Success	10 Aug 2022 2:41:12 PM
#6	Project1-Prod-Deploy	Success	10 Aug 2022 2:41:12 PM

Deploy WAR to Tomcat Server through Jenkins (Automation)

Reference:

- [Web Link](#)
- [Video Link](#)

1. First of all, Configure the Tomcat Manager by following the below link:

[Tomcat Configuration with Manager](#)

[Apache Tomcat 9.0.65 Download Link](#)

```
<role rolename="admin-gui"/>
<role rolename="manager-script"/>
<role rolename="manager-gui"/>
<user username="aman" password="mypassword" roles="admin-gui,manager-gui,manager-script"/>
```

The Output should be like this:



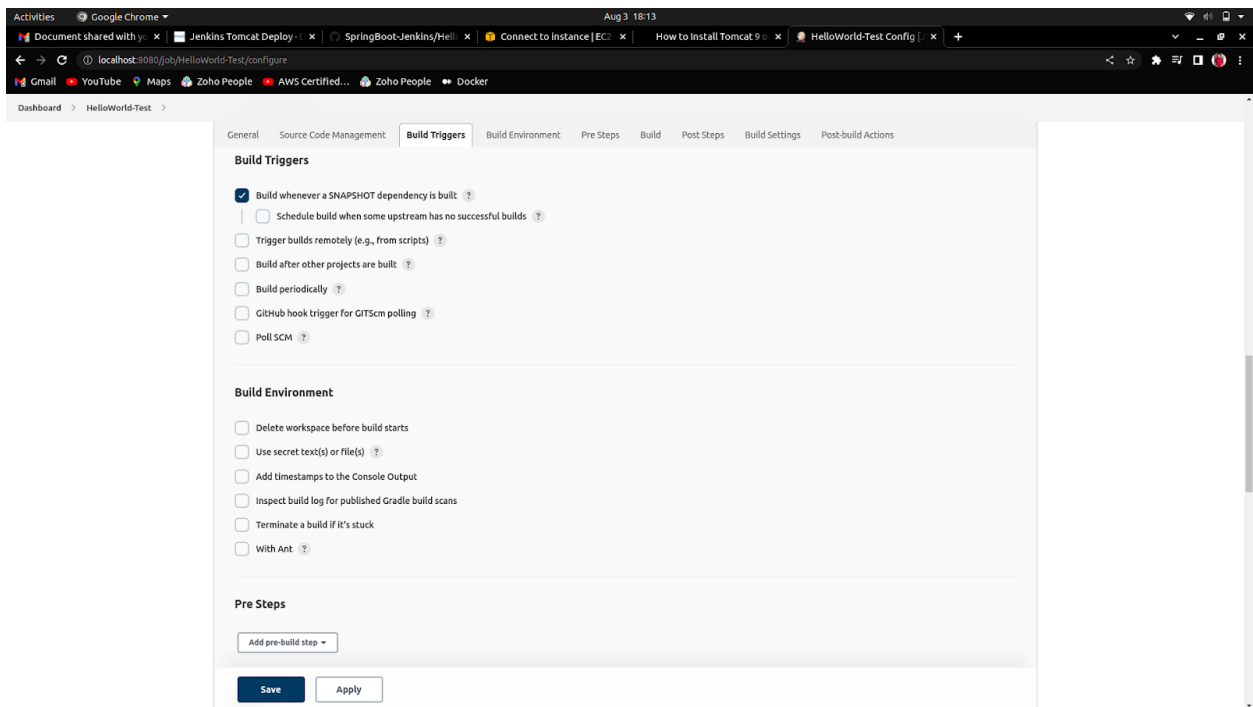
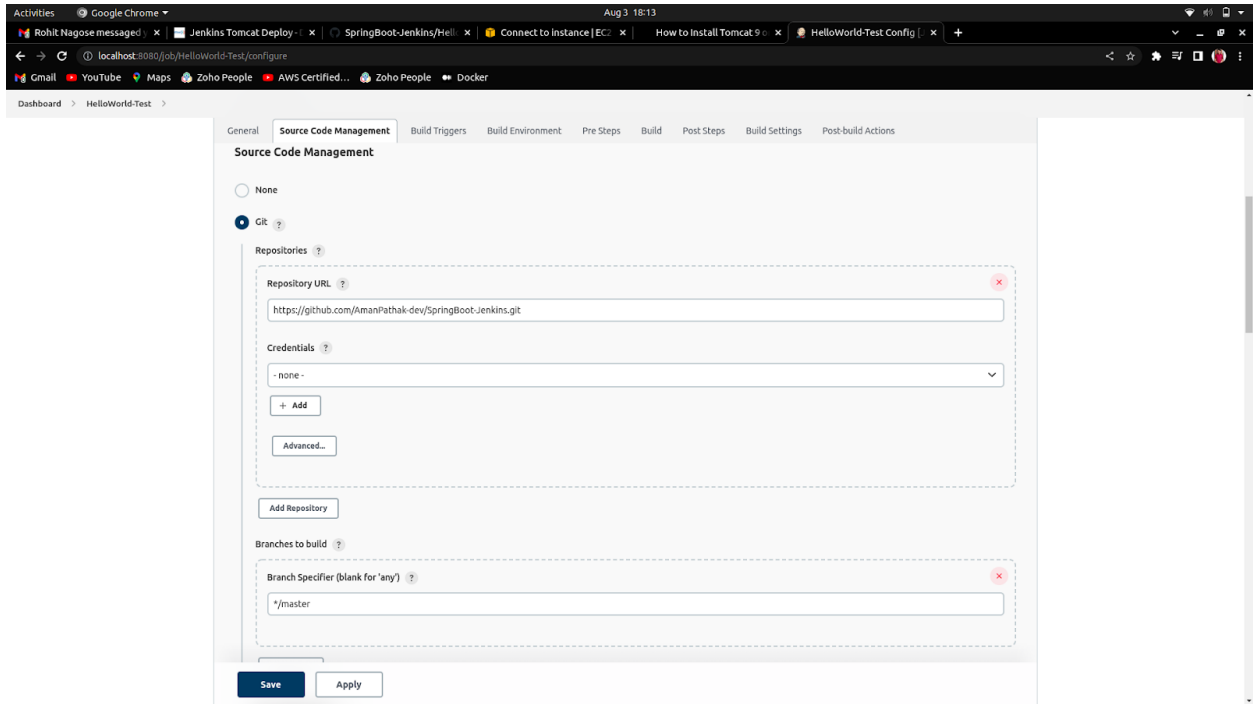
The screenshot shows the Apache Tomcat 9.0.11 Manager App interface. At the top, there is a navigation bar with links: Home, Documentation, Configuration, Examples, Wiki, Mailing Lists, and Find Help. Below the navigation bar, the title "Apache Tomcat/9.0.11" is displayed. A green banner with a cat icon (Tomcat logo) reads: "If you're seeing this, you've successfully installed Tomcat. Congratulations!". To the right of the banner, there are three buttons: "Server Status", "Manager App", and "Host Manager". Below the banner, there is a section titled "Recommended Reading:" with links to "Security Considerations HOW-TO", "Manager Application HOW-TO", and "Clustering/Session Replication HOW-TO". At the bottom, there is a "Developer Quick Start" section with links to "Tomcat Setup", "First Web Application", "Realms & AAA", "JDBC DataSources", "Examples", "Servlet Specifications", and "Tomcat Versions".

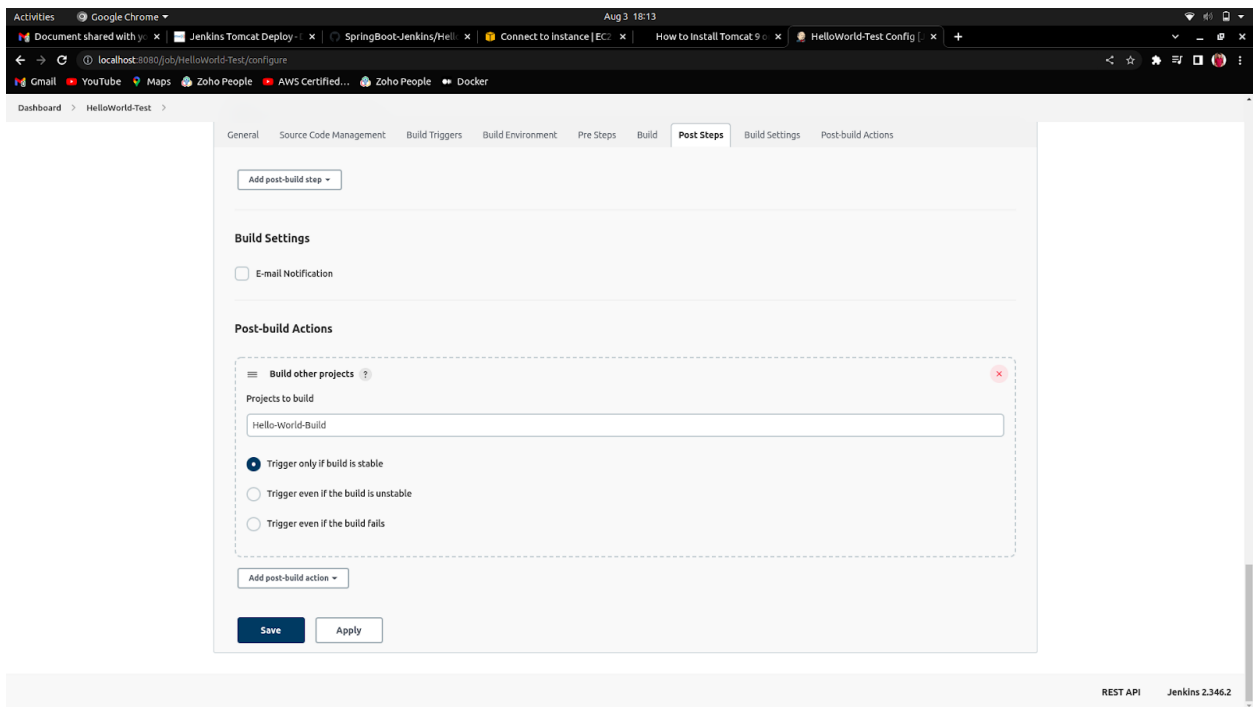
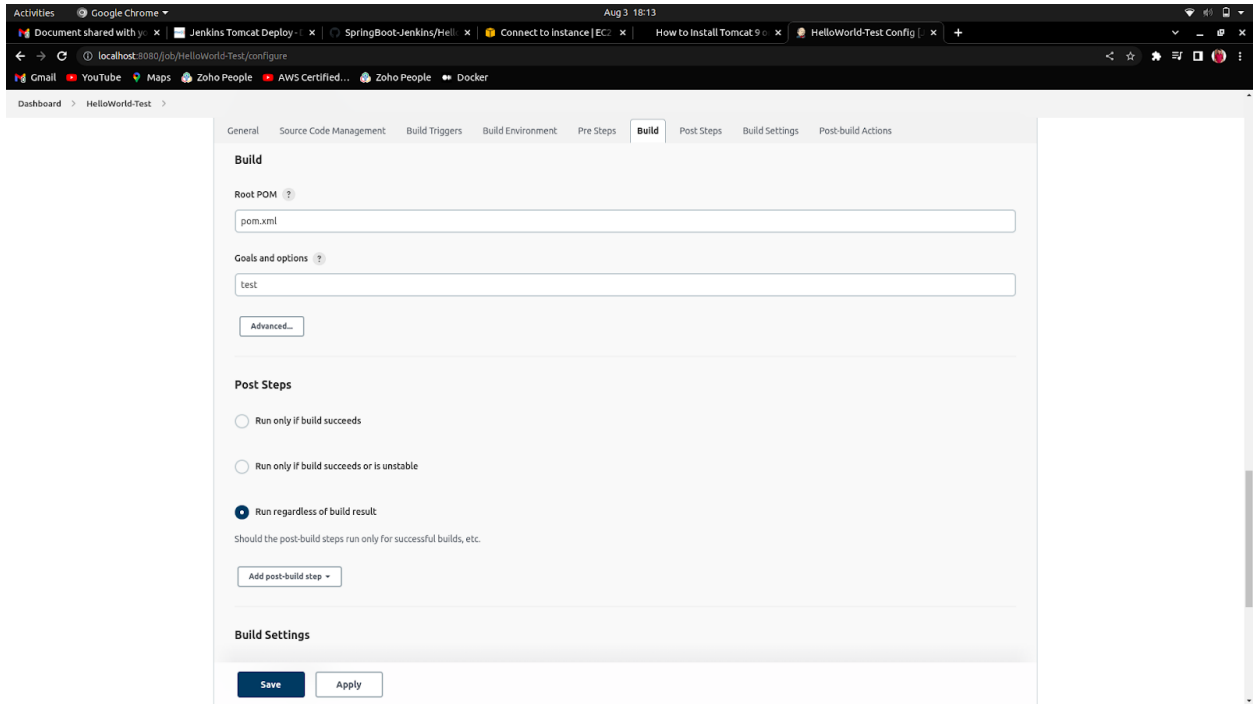
2. Now, You have to Create Four Jobs:

- a. **HelloWorld-Test:** This Job will perform the testing of the Project which is a Spring Boot-based Project.
- b. **Hello-World-Build:** This Job will perform the building of the Spring Boot Project which will be .war file as in the result.
- c. **HelloWorld-Deploy-Test:** In this Job, the deployment will be performed as a testing deployment to check whether the Job is successful or not.
- d. **HelloWorld-Deploy-Prod:** This job will perform the same task that was performed by the last job (HelloWorld-Deploy-Test) but it will be on the Production level where the inclusion is present of human intervention or manual clicks.

Now, All the jobs have their configuration, which is given below sequentially:

a. **HelloWorld-Test**





b. Hello-World-Build:

The screenshot shows the Jenkins configuration page for a job named 'Hello-World-Build'. The 'General' tab is selected, displaying various settings. The 'Permission to Copy Artifact' section is checked. The 'Projects to allow copy artifacts' field contains 'HelloWorld-*'. The 'Source Code Management' section is set to 'Git'. The 'Repository URL' is 'https://github.com/AmanPathak-dev/SpringBoot-Jenkins.git'. The 'Credentials' dropdown is set to 'none'. The 'Save' and 'Apply' buttons are at the bottom.

Dashboard > Hello-World-Build >

General Source Code Management Build Triggers Build Environment Pre Steps Build Post Steps Build Settings Post-build Actions

☒ Permission to Copy Artifact

Projects to allow copy artifacts ?

HelloWorld-*

☐ This project is parameterised ?

☐ Throttle builds ?

☐ Disable this project ?

☐ Execute concurrent builds if necessary ?

Advanced...

Source Code Management

☐ None

☒ Git ?

Repositories ?

Repository URL ?

https://github.com/AmanPathak-dev/SpringBoot-Jenkins.git

Credentials ?

- none -

+ Add

Save Apply

The screenshot shows the Jenkins configuration page for a job named 'Hello-World-Build'. The 'Source Code Management' tab is selected, displaying settings for repository management. The 'Add Repository' button is visible. The 'Branches to build' section is set to '*/master'. The 'Repository browser' dropdown is set to 'Auto'. The 'Additional Behaviours' section has an 'Add' button. The 'Build Triggers' section is checked. The 'Save' and 'Apply' buttons are at the bottom.

Dashboard > Hello-World-Build >

General Source Code Management Build Triggers Build Environment Pre Steps Build Post Steps Build Settings Post-build Actions

+ Add

Advanced...

Add Repository

Branches to build ?

Branch Specifier (blank for 'any') ?

*/master

Add Branch

Repository browser ?

(Auto)

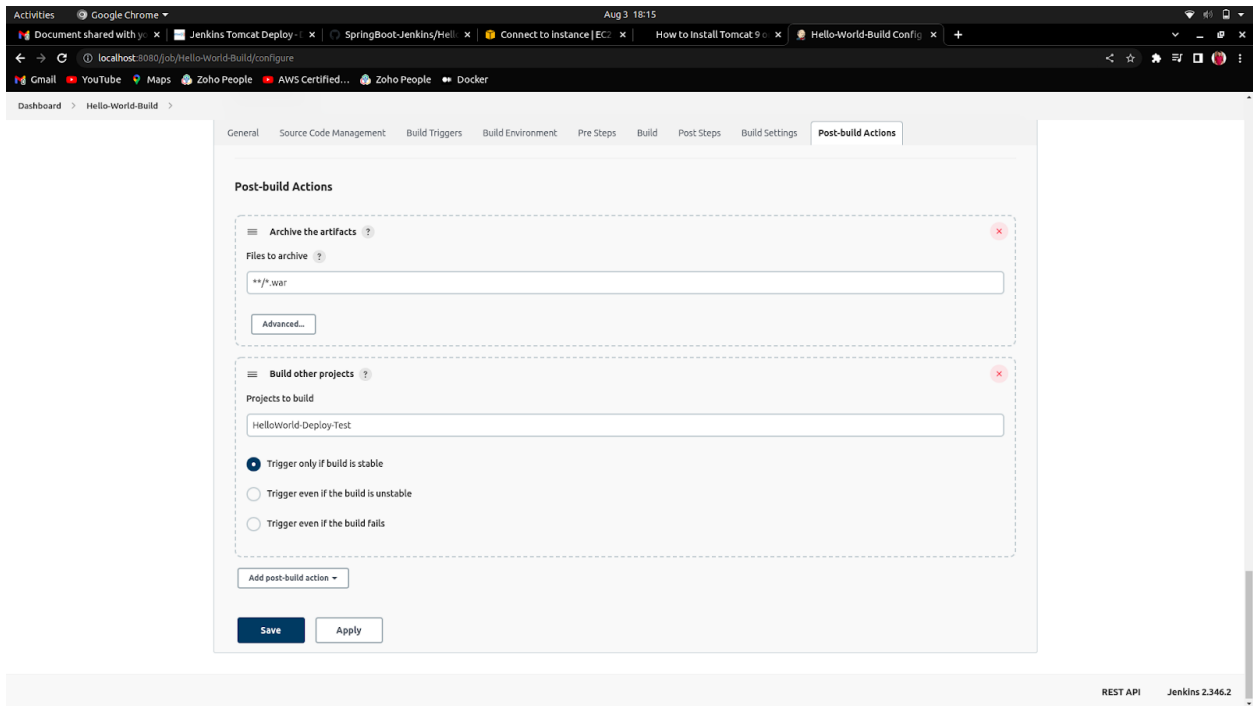
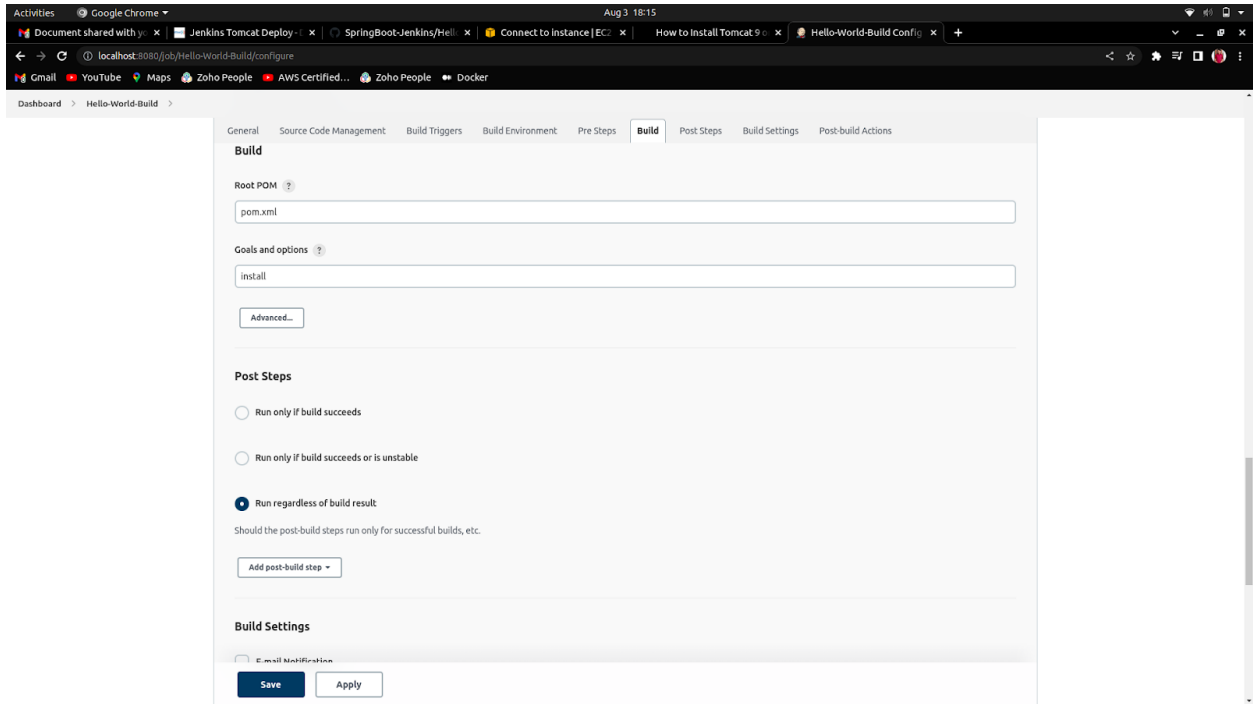
Additional Behaviours

Add

Build Triggers

☒ Build whenever a SNAPSHOT dependency is built ?

Save Apply



c. HelloWorld-Deploy-Test

Activities Google Chrome Aug 3, 18:53

localhost:3080/job/HelloWorld-Deploy-Test/configure

Dashboard > HelloWorld-Deploy-Test >

General Source Code Management Build Triggers Build Environment Build Post-build Actions

[Plain text] [Preview](#)

☐ Discard old builds ?

☐ GitHub project

☒ Permission to Copy Artifact

Projects to allow copy artifacts ?

HelloWorld-Deploy-Prod

☐ This project is parameterised ?

☐ Throttle builds ?

☐ Disable this project ?

☐ Execute concurrent builds if necessary ?

[Advanced...](#)

Source Code Management

☐ None

☒ Git ?

Repositories ?

Repository URL ?

[https://github.com/AmanPathak-dev/SpringBoot-Jenkins.git](#)

[Save](#) [Apply](#)

Activities Google Chrome Aug 3, 18:15

localhost:3080/job/HelloWorld-Deploy-Test/configure

Dashboard > HelloWorld-Deploy-Test >

General **Source Code Management** Build Triggers Build Environment Build Post-build Actions

☒ Git ?

Repositories ?

Repository URL ?

[https://github.com/AmanPathak-dev/SpringBoot-Jenkins.git](#)

Credentials ?

none

[+ Add](#)

[Advanced...](#)

[Add Repository](#)

Branches to build ?

Branch Specifier (blank for 'any') ?

[*/master](#)

[Add Branch](#)

Repository browser ?

[Save](#) [Apply](#)

Activities Google Chrome Aug 3, 18:16

Document shared with y... Jenkins Tomcat Deploy... SpringBoot-Jenkins/Hell... Connect to Instance | EC... How to Install Tomcat 9... HelloWorld-Deploy-Test... +

localhost:8080/job/HelloWorld-Deploy-Test/configure

Gmail YouTube Maps Zoho People AWS Certified... Zoho People Docker

Dashboard > HelloWorld-Deploy-Test >

General Source Code Management **Build Triggers** Build Environment Build Post-build Actions

☐ GitHub hook trigger for GITSCm polling ?

☒ Poll SCM ?

Schedule ?

H/2 * * * *

Would last have run at Wednesday, 3 August, 2022 at 6:14:49 PM India Standard Time; would next run at Wednesday, 3 August, 2022 at 6:16:49 PM India Standard Time.

☐ Ignore post-commit hooks ?

Build Environment

☒ Delete workspace before build starts

Advanced...

☐ Use secret text(s) or file(s) ?

☐ Add timestamps to the Console Output

☐ Inspect build log for published Gradle build scans

☐ Terminate a build if it's stuck

☐ With Ant ?

Build

Copy artifacts from another project

Save Apply

Build

Copy artifacts from another project

Project name ?

Hello-World-Build

⚠️ Artifacts will be copied from all modules of this Maven project; click the help icon to learn about selecting a particular module.

Which build ?

Latest successful build

☐ Stable build only

Activities Google Chrome Aug 3, 18:16

Document shared with y Jenkins Tomcat Deploy - SpringBoot-Jenkins/Hell Connect to Instance | EC How to Install Tomcat 9 HelloWorld-Deploy-Test

localhost:8080/job/HelloWorld-Deploy-Test/configure

Gmail YouTube Maps Zoho People AWS Certified... Zoho People Docker

Dashboard > HelloWorld-Deploy-Test

General Source Code Management Build Triggers Build Environment Build Post-build Actions

Post-build Actions

Archive the artifacts

Files to archive ?

**/*.war

Advanced...

Deploy war/ear to a container

WAR/EAR files ?

**/*.war

Context path ?

/app

Containers

Tomcat 9.x Remote

Credentials

aman/*

+ Add

Save

Apply

General

Source Code Management

Build Triggers

Build Environment

Build

Post-build Actions

aman/******

+ Add

Tomcat URL ?

http://54.89.215.232:8080/

Advanced...

Add Container

☐ Deploy on failure

Build other projects (manual step) ?

Downstream Project Names

HelloWorld-Deploy-Prod

Add Parameters

Add post-build action

Save

Apply

d. HelloWorld-Deploy-Prod

The screenshot shows the Jenkins configuration page for 'HelloWorld-Deploy-Prod'. The 'Source Code Management' tab is selected. Under 'Source Code Management', the 'None' option is selected. Under 'Build Triggers', all options are unchecked. Under 'Build Environment', all options are unchecked. The 'Save' button is highlighted in blue.

Dashboard > HelloWorld-Deploy-Prod >

General | **Source Code Management** | Build Triggers | Build Environment | Build | Post-build Actions

Source Code Management

☒ None

☐ Git ?

Build Triggers

☐ Trigger builds remotely (e.g., from scripts) ?

☐ Build after other projects are built ?

☐ Build periodically ?

☐ GitHub hook trigger for GITscm polling ?

☐ Poll SCM ?

Build Environment

☐ Delete workspace before build starts

☐ Use secret text(s) or file(s) ?

☐ Add timestamps to the Console Output

☐ Inspect build log for published Gradle build scans

☐ Terminate a build if it's stuck

☐ With Ant ?

Save **Apply**

The screenshot shows the Jenkins configuration page for 'HelloWorld-Deploy-Prod'. The 'Build' tab is selected. A dashed box highlights the 'Copy artifacts from another project' section. The 'Project name' is 'HelloWorld-Deploy-Test'. The 'Which build' dropdown is set to 'Latest successful build'. The 'Stable build only' checkbox is unchecked. The 'Artifacts to copy', 'Artifacts not to copy', and 'Target directory' fields are empty. The 'Parameter filters' field is empty. The 'Flatten directories', 'Optional', and 'Fingerprint Artifacts' checkboxes are all unchecked. The 'Save' button is highlighted in blue.

Dashboard > HelloWorld-Deploy-Prod >

General | Source Code Management | Build Triggers | Build Environment | **Build** | Post-build Actions

Build

Copy artifacts from another project

Project name ?

HelloWorld-Deploy-Test

Which build ?

Latest successful build

☐ Stable build only

Artifacts to copy ?

Artifacts not to copy ?

Target directory ?

Parameter filters ?

☐ Flatten directories ☐ Optional ☒ Fingerprint Artifacts ?

Save **Apply**

Activities Google Chrome Aug 3, 18:53

Inbox - aman.pathak@... Jenkins Tomcat Deploy... SpringBoot-Jenkins/Hell... Instance details [EC2 Ma... How to Install Tomcat 9... Hello-World [Jenkins] HelloWorld-Deploy-Prod

localhost:8080/job/HelloWorld-Deploy-Prod/configure

Gmail YouTube Maps Zoho People AWS Certified... Zoho People Docker

Dashboard > HelloWorld-Deploy-Prod >

General Source Code Management Build Triggers Build Environment Build Post-build Actions

Deploy war/ear to a container

WAR/EAR files ?

**/*.war

Context path ?

/app

Containers

Tomcat 9.x Remote

Credentials

aman/*****

+ Add

Tomcat URL ?

http://3.83.235.174:8080/

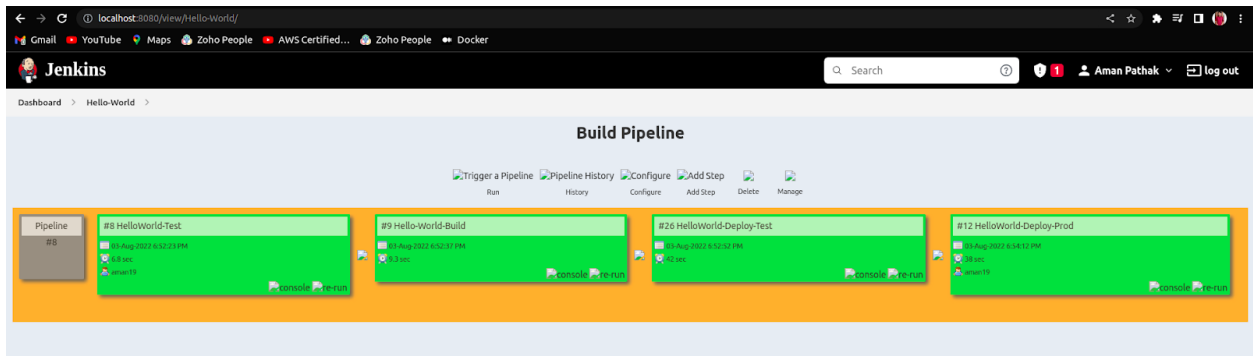
Advanced...

Add Container +

☐ Deploy on failure

Save Apply

The Output



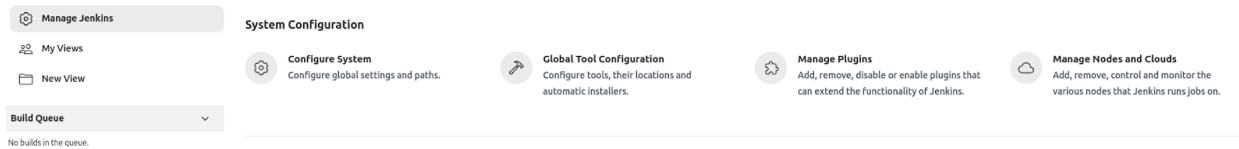
This is All About Spring Boot! This is The Second Version of The Spring Boot



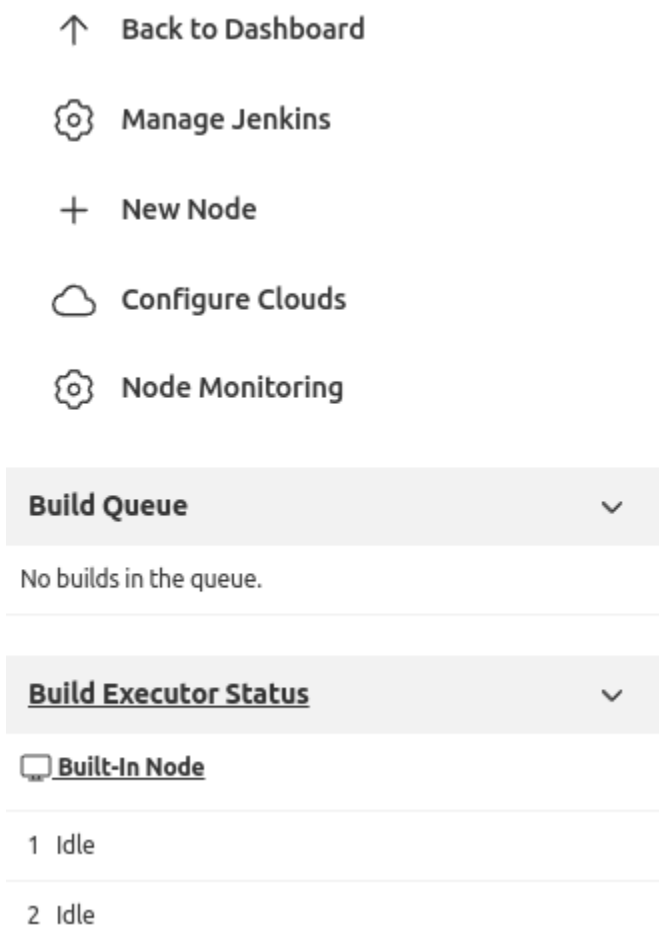
This is All About Spring Boot! This is The Second Version of The Spring Boot

Creating Jenkins Slave

1. First of all, go into Manage Jenkins -> Manage Nodes and Clouds.



2. To create the new Slave, Click on the new node.



3. Give the name to your slave and check the radio button of Permanent Agent then, click on the Create button.

New node

Node name

Linux1

Type



Permanent Agent

Adds a plain, permanent agent to Jenkins. This is called "permanent" because Jenkins doesn't provide higher level of integration with these agents, such as dynamic provisioning. Select this type if no other agent types apply — for example such as when you are adding a physical computer, virtual machines managed outside Jenkins, etc.

4. Before going to the subsequent Configurations of your Jenkins slave,
Follow some steps:
 - a. Create an EC2 Instance with Inbound rule 22(SSH) at least.
 - b. SSH into your instance.
 - c. Install OpenJDK by the command:
sudo apt install openjdk-8-jre-headless -y
 - d. Create a directory named jenkins inside the /var directory.
 - e. Change the owner of that directory by the given command:
sudo chown ubuntu:ubuntu jenkins/
 - f. Now, change the password of your EC2 Instance by the following commands:
sudo passwd ubuntu
sudo vim sshd_config
sudo systemctl restart jenkins

For more reference, follow the link below:

[Change EC2 Instance Password](#)

5. Enter the number of executors as per your choice, then enter the Remote root directory which will be /var/jenkins then, the launch type must be “Launch agents via SSH” Enter the Host Name which will be the Public IP of your instance, then add the credentials of your EC2 Instance (username=ubuntu and password= aman@123) and In the last Host, the Key verification Strategy must be “Non-verifying Verification Strategy”.

The screenshot shows the Jenkins web interface in Google Chrome. The browser tabs include 'Search results - aman.pa...', 'How to Enable Password...', 'Linux1 Configuration [Jenkins]', 'Connect to instance [EC2]', and 'EC2 Management Console'. The address bar shows 'localhost:8080/computer/Linux1/configure'. The Jenkins header shows the user 'Aman Pathak' and a 'log out' button. The left sidebar contains navigation links: 'Back to List', 'Status', 'Delete Agent', 'Configure' (selected), 'Build History', 'Load Statistics', 'Script Console', 'Log', 'System Information', and 'Disconnect'. Below these is a 'Build Executor Status' section showing '1 idle' and '2 idle' executors. The main configuration area for 'Linux1' includes the following fields: 'Name' (Linux1), 'Description' (empty), 'Number of executors' (2), 'Remote root directory' (/var/jenkins), 'Labels' (linux2), 'Usage' (Use this node as much as possible), 'Launch method' (Launch agents via SSH), and 'Host' (54.82.113.239). A 'Save' button is at the bottom.

Field	Value
Name	Linux1
Description	
Number of executors	2
Remote root directory	/var/jenkins
Labels	linux2
Usage	Use this node as much as possible
Launch method	Launch agents via SSH
Host	54.82.113.239

Host ?

54.82.113.239

Credentials ?

ubuntu/*****

+ Add

Host Key Verification Strategy ?

Non verifying Verification Strategy

?

Advanced...

Availability ?

Keep this agent online as much as possible

?

Node Properties

☐ Disable deferred wipeout on this node ?

☐ Environment variables

☐ Tool Locations

Save

6. Now go to that Slave and click on Relaunch agent.

Activities Google Chrome Aug 4 14:42

Search results - aman.pa... x How to Enable Password x Linux1 [Jenkins] x Connect to instance [EC... x EC2 Management Console x +

localhost:8080/computer/Linux1/

Gmail YouTube Maps Zoho People Docker

Jenkins Search Aman Pathak log out

Dashboard > Nodes > Linux1

Back to List

Status

Delete Agent

Configure

Build History

Load Statistics

Log

Build Executor Status

Agent Linux1

This node is being launched. [See log for more details](#)

Relaunch agent

Labels

linux2

Projects tied to Linux1

None

Mark this node temporarily offline

REST API Jenkins 2.346.2

7. As the Slave is connected.

Activities Google Chrome Aug 4 14:45

Search results - aman.pa... x How to Enable Password x Linux1 log [Jenkins] x Connect to instance [EC... x EC2 Management Console x +

localhost:8080/computer/Linux1/log

Gmail YouTube Maps Zoho People Docker

Jenkins Search Aman Pathak log out

Dashboard > Nodes > Linux1

Back to List

Status

Delete Agent

Configure

Build History

Load Statistics

Log

Build Executor Status

Source agent hash is 4B1A46AEF09ED08CE34A2B073FC374F1. Installed agent hash is 4B1A46AEF09ED08CE34A2B073FC374F1

Verified agent jar. No update is necessary.

Expanded the channel window size to 4MB

[08/04/22 14:43:35] [SSH] Starting agent process: cd "/var/jenkins" && java -jar remoting.jar -workDir /var/jenkins -jar-cache /var/jenkins/remoting/jarCache

Aug 04, 2022 9:13:36 AM org.jenkinsci.remoting.engine.WorkDirManager InitializeWorkDir

INFO: Using /var/jenkins/remoting as a remoting work directory

Aug 04, 2022 9:13:36 AM org.jenkinsci.remoting.engine.WorkDirManager setupLogging

INFO: Both error and output logs will be printed to /var/jenkins/remoting

<==[JENKINS REMOTING CAPACITY]==>channel started

Remoting version: 4.13.2

Launcher: SSHLancher

Communication Protocol: Standard in/out

This is a Unix agent

Evacuated stdout

Agent successfully connected and online

The Agent is connected, disconnect it before to try to connect it again.

1 Idle

2 Idle

localhost:8080/computer/Linux1/builds

REST API Jenkins 2.346.2

8. You can check on the left side named Linux1.

↑ Back to Dashboard

Manage Jenkins

+ New Node

Configure Clouds

Node Monitoring

Build Queue

No builds in the queue.

Build Executor Status

Built-in Node

1 Idle

2 Idle

Linux

1 Idle

2 Idle

Linux1

1 Idle

2 Idle

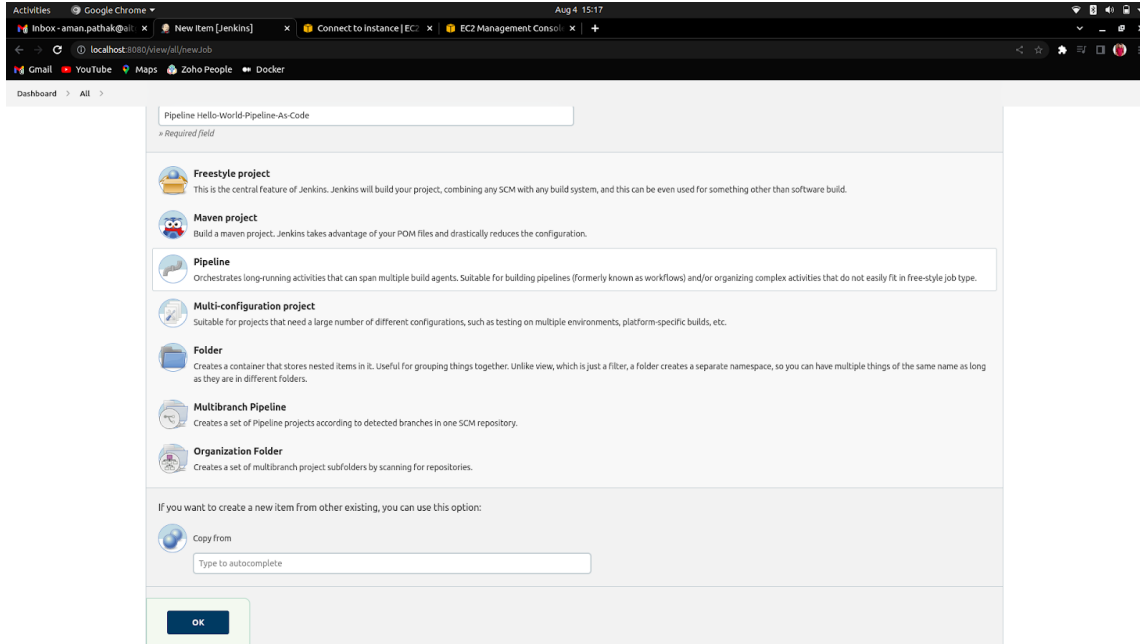
Manage nodes and clouds

Refresh status

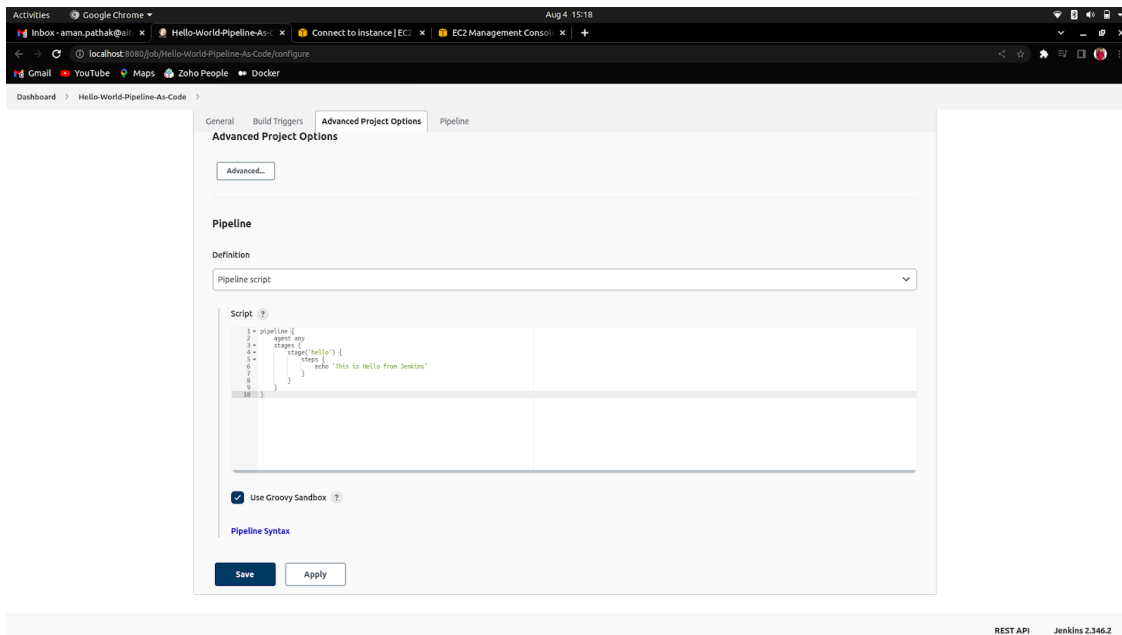
S	Name	Architecture	Clock Difference	Free Disk Space	Free Swap Space	Free Temp Space	Response Time
	Built-in Node	Linux (amd64)	In sync	413.20 GB	1.31 GB	413.20 GB	0ms
	Linux	Linux (amd64)	6.2 sec behind	5.81 GB	0 B	5.81 GB	5190ms
	Linux1	Linux (amd64)	6.2 sec ahead	5.81 GB	0 B	5.81 GB	7238ms
last checked		3 min 33 sec	3 min 23 sec	3 min 21 sec	3 min 23 sec	3 min 21 sec	3 min 29 sec

Create Jenkins Pipeline as a Code

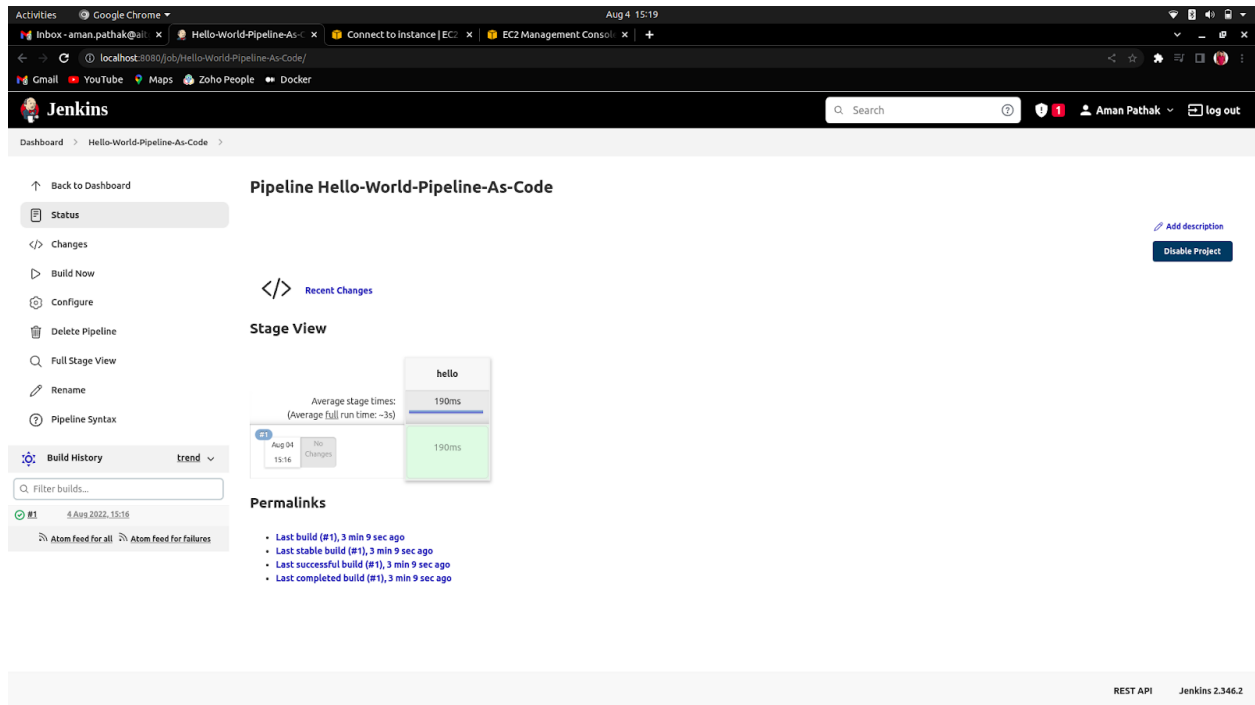
1. Create a Job that will be a Pipeline.



2. Write the Declarative Pipeline to run the job of Hello World.



3. Now, when you build the job you'll see look like this:



The screenshot shows the Jenkins web interface for a pipeline named 'Hello-World-Pipeline-As-Code'. The left sidebar contains navigation links: Back to Dashboard, Status, Changes, Build Now, Configure, Delete Pipeline, Full Stage View, Rename, Pipeline Syntax, Build History, and Filter builds... The main content area displays the 'Stage View' for the 'hello' stage, showing an average stage time of 190ms and an average full run time of ~3s. Below this, the 'Permalinks' section lists build details for build #1. The bottom right corner shows 'REST API' and 'Jenkins 2.346.2'.

4. To see the output hover on the green box and click on the logs.

Stage View



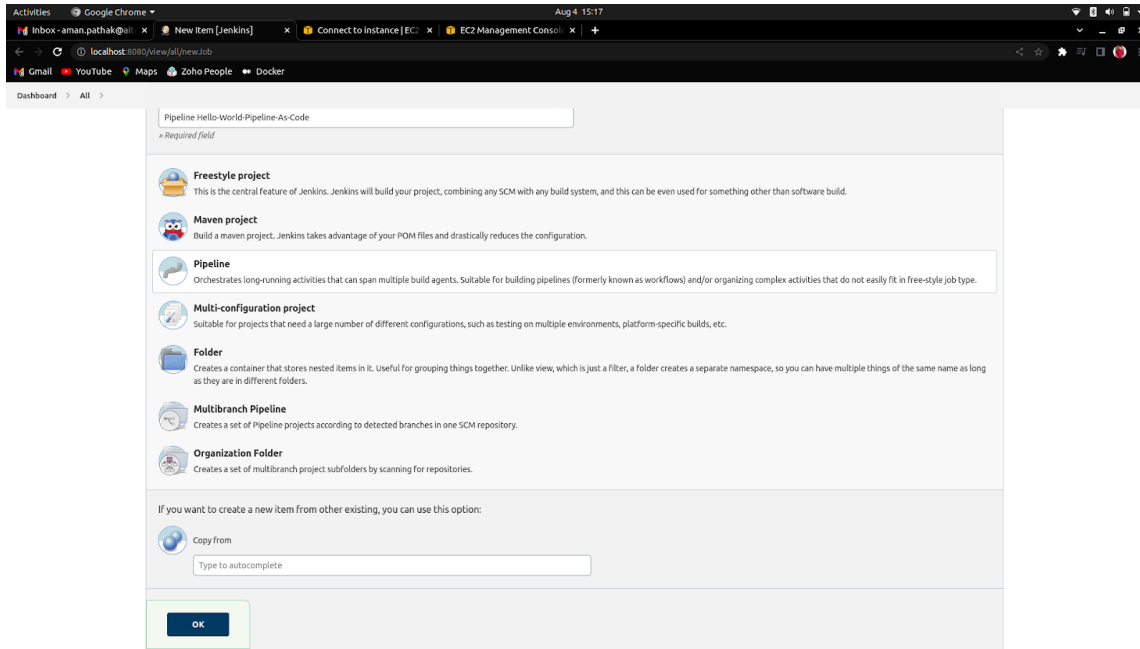
This close-up view of the 'Stage View' for the 'hello' stage highlights the '190ms' duration box. A tooltip with a bar chart icon and the word 'Logs' is visible, indicating where to click to view the build output. The average stage times are also shown: 'Average stage times: (Average full run time: ~3s)'.



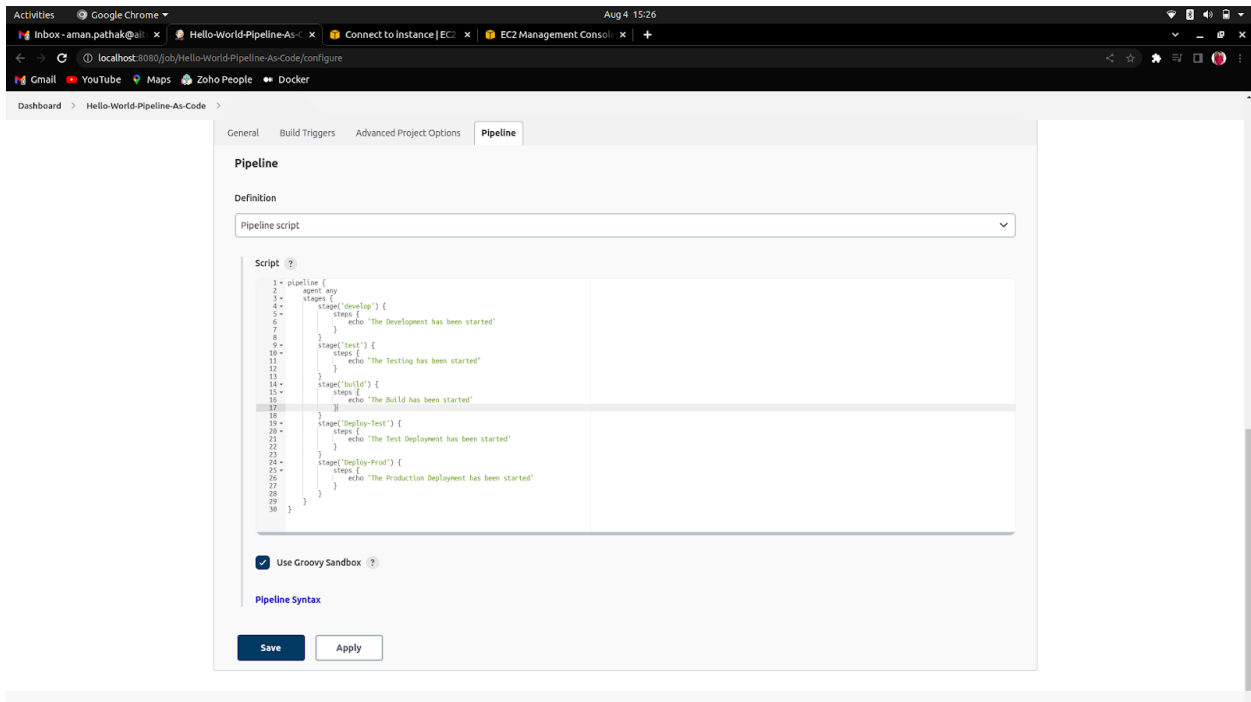
The screenshot shows the 'Stage Logs (hello)' window. It contains a single log entry: 'Print Message - This is Hello from Jenkins (self time 10ms)'. The log entry is displayed in a dark-themed window with a close button in the top right corner.

Multi-Staging Pipeline as a Code

1. Create a Job that will be a Pipeline.



2. Write the Declarative Pipeline.



3. The Output:

The screenshot displays the Jenkins web interface for a pipeline named "Hello-World-Pipeline-As-Code". The interface includes a sidebar with navigation options like "Back to Dashboard", "Status", "Changes", "Build Now", "Configure", "Delete Pipeline", "Full Stage View", "Rename", "Pipeline Syntax", and "Build History". The main content area shows the "Stage View" with a table of stage execution times and a "Permalinks" section with links to build logs.

Pipeline Hello-World-Pipeline-As-Code

Recent Changes

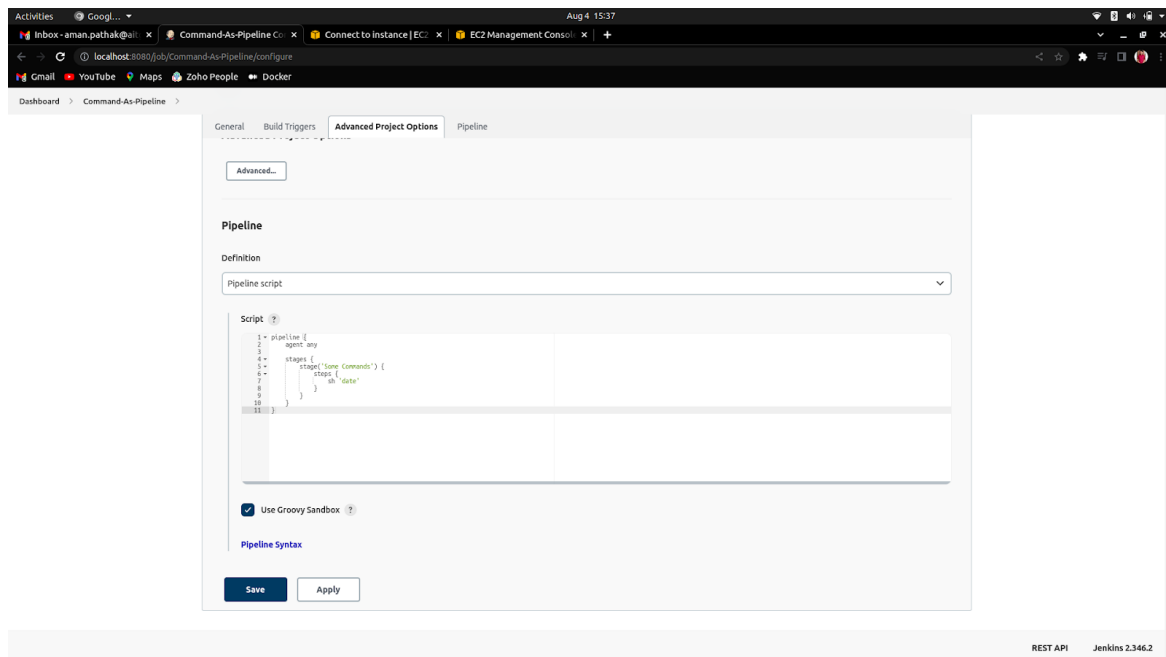
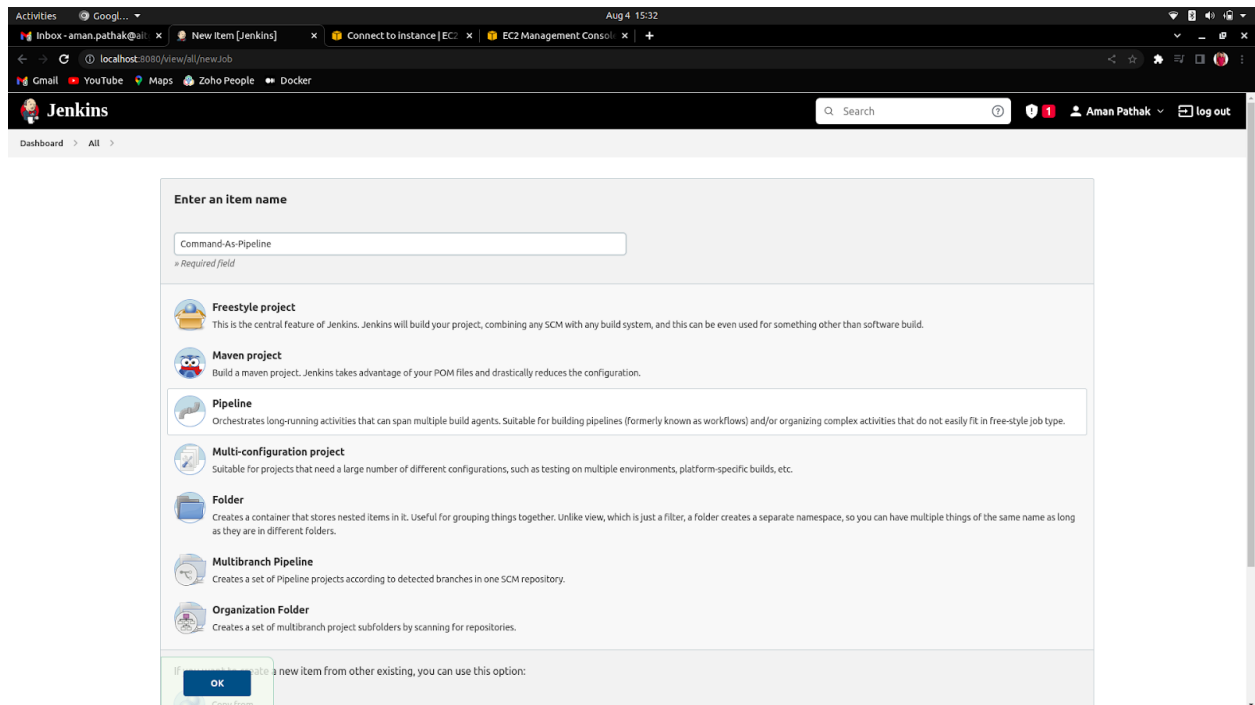
Stage View

	develop	test	build	Deploy-Test	Deploy-Prod
Average stage times: (Average full run time: ~960ms)	96ms	80ms	80ms	81ms	79ms
#3 Aug 04 15:24 No Changes	96ms	80ms	80ms	81ms	79ms
#2 Aug 04 15:23 No Changes					

Permalinks

- Last build (#3), 2 min 3 sec ago
- Last stable build (#3), 2 min 3 sec ago
- Last successful build (#3), 2 min 3 sec ago
- Last failed build (#2), 3 min 2 sec ago
- Last unsuccessful build (#2), 3 min 2 sec ago
- Last completed build (#3), 2 min 3 sec ago

How to run Commands in Pipeline as a Code



How to run multiple commands in Pipeline as a code

The top screenshot shows the Jenkins 'Enter an item name' dialog. The input field contains 'Command-As-Pipeline'. Below the input field, there are several project type options: Freestyle project, Maven project, Pipeline, Multi-configuration project, Folder, Multibranch Pipeline, and Organization Folder. The 'Pipeline' option is highlighted.

The bottom screenshot shows the Jenkins configuration page for 'Command-As-Pipeline'. The 'Pipeline' tab is selected. The 'Definition' section shows a dropdown menu set to 'Pipeline script'. Below this, there is a text area containing a Groovy script:

```
1 pipeline {
2   agent any
3
4   stages('Some Commands') {
5     steps {
6       sh
7       sh
8       sh
9       sh
10      sh
11      sh
12      sh
13      sh
14      sh
15      sh
16    }
17  }
18 }
```

Below the script, there is a checkbox labeled 'Use Groovy Sandbox' which is checked. At the bottom, there are 'Save' and 'Apply' buttons.

The Output:

Activities Google Chrome Aug 4 15:51

Inbox - aman.pathak@ail... Command-As-Pipeline #9 Connect to Instance | EC2... EC2 Management Console

localhost:3080/job/Command-As-Pipeline/9/console

Gmail YouTube Maps Zoho People Docker

Dashboard > Command-As-Pipeline > #9

Restart from Stage
Replay
Pipeline Steps
Workspaces
Previous Build

```
/var/jenkins/workspace/Command-As-Pipeline
+ ls
+ date
Thu Aug 4 10:21:09 UTC 2022
+ cal 2022

      2022
January February March
Su Mo Tu We Th Fr Sa Su Mo Tu We Th Fr Sa Su Mo Tu We Th Fr Sa
1 1 2 3 4 5 1 2 3 4 5
2 3 4 5 6 7 8 6 7 8 9 10 11 12 6 7 8 9 10 11 12
9 10 11 12 13 14 15 13 14 15 16 17 18 19 13 14 15 16 17 18 19
16 17 18 19 20 21 22 20 21 22 23 24 25 26 20 21 22 23 24 25 26
23 24 25 26 27 28 29 27 28 27 28 29 30 31
30 31

April May June
Su Mo Tu We Th Fr Sa Su Mo Tu We Th Fr Sa Su Mo Tu We Th Fr Sa
1 2 1 2 3 4 5 6 7 1 2 3 4
3 4 5 6 7 8 9 8 9 10 11 12 13 14 5 6 7 8 9 10 11
10 11 12 13 14 15 16 15 16 17 18 19 20 21 12 13 14 15 16 17 18
17 18 19 20 21 22 23 22 23 24 25 26 27 28 19 20 21 22 23 24 25
24 25 26 27 28 29 30 29 30 31 26 27 28 29 30

July August September
Su Mo Tu We Th Fr Sa Su Mo Tu We Th Fr Sa Su Mo Tu We Th Fr Sa
1 2 1 2 3 4 5 6 1 2 3
3 4 5 6 7 8 9 7 8 9 10 11 12 13 4 5 6 7 8 9 10
10 11 12 13 14 15 16 14 15 16 17 18 19 20 11 12 13 14 15 16 17
17 18 19 20 21 22 23 21 22 23 24 25 26 27 18 19 20 21 22 23 24
24 25 26 27 28 29 30 28 29 30 31 25 26 27 28 29 30
31

October November December
Su Mo Tu We Th Fr Sa Su Mo Tu We Th Fr Sa Su Mo Tu We Th Fr Sa
1 1 2 3 4 5 1 2 3
2 3 4 5 6 7 8 6 7 8 9 10 11 12 4 5 6 7 8 9 10
9 10 11 12 13 14 15 13 14 15 16 17 18 19 11 12 13 14 15 16 17
16 17 18 19 20 21 22 20 21 22 23 24 25 26 18 19 20 21 22 23 24
23 24 25 26 27 28 29 27 28 29 30 25 26 27 28 29 30 31
30 31

[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

REST API Jenkins 2.346.2

How to set the Environment variable Globally in the Code Pipeline

Pipeline

Definition

Pipeline script

Script ?

```
1 pipeline {
2   agent any
3   environment {
4     name = "jenkins_user"
5   }
6   stages {
7     stage('Global Variable') {
8       steps {
9         sh 'echo $name'
10      }
11    }
12  }
13 }
```

☒ Use Groovy Sandbox ?

Stage Logs (Global Variable)

Shell Script -- echo \$name (self time 3s)

+ echo jenkins user
jenkins_user

How to set the Environment variable Locally in Code Pipeline

Definition

Pipeline script

Script ?

```
1 pipeline {
2   agent any
3   environment {
4     name = "jenkins_user"
5   }
6   stages {
7     stage('Global Variable') {
8       steps {
9         sh 'echo $name'
10        sh 'echo $BUILD_ID'
11      }
12    }
13    stage('Local Variable') {
14      environment {
15        user_id = '34'
16      }
17      steps {
18        sh 'echo $user_id'
19      }
20    }
21    stage('Accessing Local Variable') {
22      steps {
23        sh 'echo $user_id'
24      }
25    }
26  }
27 }
```

☒ Use Groovy Sandbox ?

The Output

Activities

Google...

Aug 4 16:07

Inbox - aman.pathak@all x Environment_Variable-Code-Pipeline x Connect to Instance | EC2 x EC2 Management Console x +

localhost:8080/job/Environment_Variable-Code-Pipeline/5/console

Gmail YouTube Maps Zoho People Docker

Dashboard > Environment_Variable-Code-Pipeline > #5

</> Changes

Console Output

View as plain text

Edit Build Information

Delete build '#5'

Restart from Stage

Replay

Pipeline Steps

Workspaces

Previous Build

Started by user Aman Pathak

[Pipeline] Start of Pipeline

[Pipeline] node

Running on Linux1 in /var/jenkins/workspace/Environment_Variable-Code-Pipeline

[Pipeline] {

[Pipeline] withEnv

[Pipeline] {

[Pipeline] stage

[Pipeline] { (Global Variable)

[Pipeline] sh

+ echo jenkins_user

jenkins_user

[Pipeline] sh

+ echo 5

5

[Pipeline] }

[Pipeline] // stage

[Pipeline] stage

[Pipeline] { (Local Variable)

[Pipeline] withEnv

[Pipeline] {

[Pipeline] sh

+ echo 34

34

[Pipeline] }

[Pipeline] // withEnv

[Pipeline] }

[Pipeline] // stage

[Pipeline] stage

[Pipeline] { (Accessing Local Variable)

[Pipeline] sh

+ echo

[Pipeline] }

[Pipeline] // stage

[Pipeline] }

[Pipeline] // withEnv

[Pipeline] }

[Pipeline] // node

[Pipeline] End of Pipeline

Finished: SUCCESS

How to take input from the users in the Code Pipeline

Pipeline

Definition

Pipeline script

Script ?

```
1 pipeline {
2   agent any
3   parameters {
4     string(name: 'username', defaultValue: 'Jenkins', description: 'Who is the Current User')
5     booleanParam(name: 'Graduated?', defaultValue: 'true', description: 'Are you Graduated')
6     choice(name: 'hobby', choices: ['Cricket', 'Chess', 'Football', 'Hockey'], description: 'Favourite Sport')
7   }
8   stages {
9     stage('String Value') {
10      steps {
11        sh 'echo $username'
12      }
13    }
14    stage('Boolean Value') {
15      steps {
16        sh 'echo $Graduated?'
17      }
18    }
19    stage('Choices Values') {
20      steps {
21        sh 'echo $hobby'
22      }
23    }
24  }
25 }
```

The Output:

Activities Google Chrome Aug 4 16:26

Inbox - aman.pathak@all Parameterized-Value-Code Connect to Instance | EC2 x EC2 Management Console x

localhost:3080/job/Parameterized-Value-Code-Pipeline/#console

Gmail YouTube Maps Zoho People Docker

Dashboard > Parameterized-Value-Code-Pipeline > #6

Back to Project

Status

Changes

Console Output

View plain text

Edit Build Information

Delete build '#6'

Parameters

Restart from Stage

Replay

Pipeline Steps

Workspaces

Previous Build

Console Output

Started by user Aman Pathak

[Pipeline] Start of Pipeline

[Pipeline] node

Running on Jenkins in /var/lib/jenkins/workspace/Parameterized-Value-Code-Pipeline

[Pipeline] {

[Pipeline] stage

[Pipeline] { (String Value)

[Pipeline] sh

+ echo Groovy Script

Groovy Script

[Pipeline] }

[Pipeline] // stage

[Pipeline] stage

[Pipeline] { (Boolean Value)

[Pipeline] sh

+ echo ?

?

[Pipeline] }

[Pipeline] // stage

[Pipeline] stage

[Pipeline] { (Choices Values)

[Pipeline] sh

+ echo Hockey

Hockey

[Pipeline] }

[Pipeline] // stage

[Pipeline] }

[Pipeline] // node

[Pipeline] End of Pipeline

Finished: SUCCESS

REST API Jenkins 2.346.2

Pipeline Script of the Last Job

```
pipeline {
  agent any
  parameters {
    string(name: 'username', defaultValue: 'Jenkins', description: 'Who is the Current User')
    booleanParam(name: 'Graduated?', defaultValue: 'true', description: 'Are you Graduated')
    choice(name: 'hobby', choices: ['Cricket','Chess','Football','Hockey'], description: 'Favourite Sport')
  }
  stages {
    stage('String Value') {
      input {
        message "Do we start?"
        ok "Yes, Let's do this"
      }
      steps {
        sh 'echo $username'
      }
    }
    stage('Boolean Value') {
      steps {
        sh 'echo $Graduated?'
      }
    }
    stage('Choices Values') {
      input {
        message "Want to Continue?"
        ok "Yeah! Definitely"
      }
      steps {
        sh 'echo $hobby'
      }
    }
  }
  post {
    always {
      echo 'The Jenkins!!!!'
    }
    success {
      echo 'The Final Achievement is Here!!!'
    }
    failure {
      echo 'Seriously! We got the Failure from the other End!!!!'
    }
  }
}
```

In **conclusion**, the above notes provide a beginner-level introduction to Jenkins and its role in enabling Continuous Integration and Continuous Delivery (CI/CD) practices. By understanding the fundamental concepts and features of Jenkins, beginners can kickstart their journey towards automating their software development processes and achieving faster, more reliable project deliveries.

For those seeking to advance their knowledge and skills in Jenkins, there is a world of possibilities to explore. Jenkins offers a vast array of plugins and integrations with various tools, allowing advanced users to tailor their CI/CD pipelines to meet specific project requirements. By delving deeper into Jenkins' capabilities, experienced users can optimize their workflows, integrate advanced testing and deployment strategies, and harness the full potential of CI/CD for continuous improvement.

Moreover, Jenkins Pipelines offers an exciting avenue for advanced users to adopt a "pipeline-as-code" approach, enabling better version control, collaboration, and scalability. With declarative pipeline scripts, developers can encapsulate complex build and deployment logic, making it easier to manage and maintain their CI/CD workflows effectively.

While the notes provide a solid foundation for beginners, it is essential to continue exploring Jenkins and other related technologies to stay up-to-date with the latest advancements in CI/CD practices. Online tutorials, documentation, forums, and community resources are valuable sources to deepen one's expertise and stay ahead in the rapidly evolving landscape of DevOps and CI/CD.

Whether at the beginner or advanced level, embracing Jenkins as a pivotal tool in the software development lifecycle empowers teams to automate, collaborate, and deliver high-quality software efficiently. By leveraging Jenkins' capabilities and staying curious, developers can continually refine their CI/CD processes and drive innovation in the dynamic world of modern software development.

Did you find it Useful?

Leave a **comment!**



ABD UR REHMAN NAWAZ

**@ABD-UR-REHMAN-
NAWAZ**

REHMANWARRAICH8282@GMAIL.OCM

FOLLOW FOR MORE

Like



Comment



Repost

